



Towards Long-Horizon Agents: A Survey

Foundation, Evolution, Harness, Optimization, Application, and Frontier

Guanting Dong^{1,*}, Xiaoshuai Song^{1,*}, Yuyang Hu^{1,*}, Jiajie Jin^{1,*}, Chenghao Zhang^{1,*}, Yifei Chen¹, Xiaoxi Li¹, Huaying Yuan¹, Xinyu Yang¹, Tongyu Wen¹, Jiejun Tan¹, Hongjin Qian², Shijue Huang⁵, Junting Lu², Zhenyu Li³, Wanjun Zhong⁴, Yutao Zhu¹, Tat-Seng Chua⁶, Zhicheng Dou^{1,†}, Ji-Rong Wen¹

* Core Contributors † Corresponding author

¹Renmin University of China, ²Peking University, ³Tsinghua University, ⁴Sun Yat-Sen University, ⁵Hong Kong University of Science and Technology, ⁶National University of Singapore

With the rapid advancement of LLM capabilities, expectations for AI agents are shifting from solving simple, single-turn tasks toward carrying out long-horizon tasks in the real world. We refer to such systems as **long-horizon agents**: agents that plan over extended horizons, interact with real-world environments, recover from their own mistakes, and adapt their strategies during execution. This capability is rapidly becoming the central bottleneck for practical agent intelligence. Despite this progress, the field still lacks a shared definition and taxonomy for long-horizon agents. Related concepts such as “self-evolving” or “autonomous” agents are often used interchangeably, none of which clearly captures what it means to build a *long-horizon agent*. This gap leaves the community without a principled way to attribute advances in long-horizon competence.

This paper offers a unified landscape by framing long-horizon agency as the co-evolution of an **externalized harness** and an **internalized optimization**, and organizes the paper around six connected perspectives: **Foundation, Evolution, Harness, Optimization, Application, and Frontier**. We first formalize long-horizon agency as a harness-coupled decision process and distinguish it from neighboring concepts such as long-running execution, autonomy, and self-evolution. We then trace the field’s evolution from prompt-level control to runtime agent systems, classifying existing work through the complementary lenses of externalized harnesses and internalized optimization. Building on this view, we organize five application forms of long-horizon agents by their interfaces, and review the corresponding benchmarks and resources. Finally, we discuss key challenges and frontier directions. Looking ahead, we hope this paper serves not only as a reference for existing work, but also as a foundation for building the next generation of capable, reliable long-horizon agents.

✉ dongguanting@ruc.edu.cn, dou@ruc.edu.cn

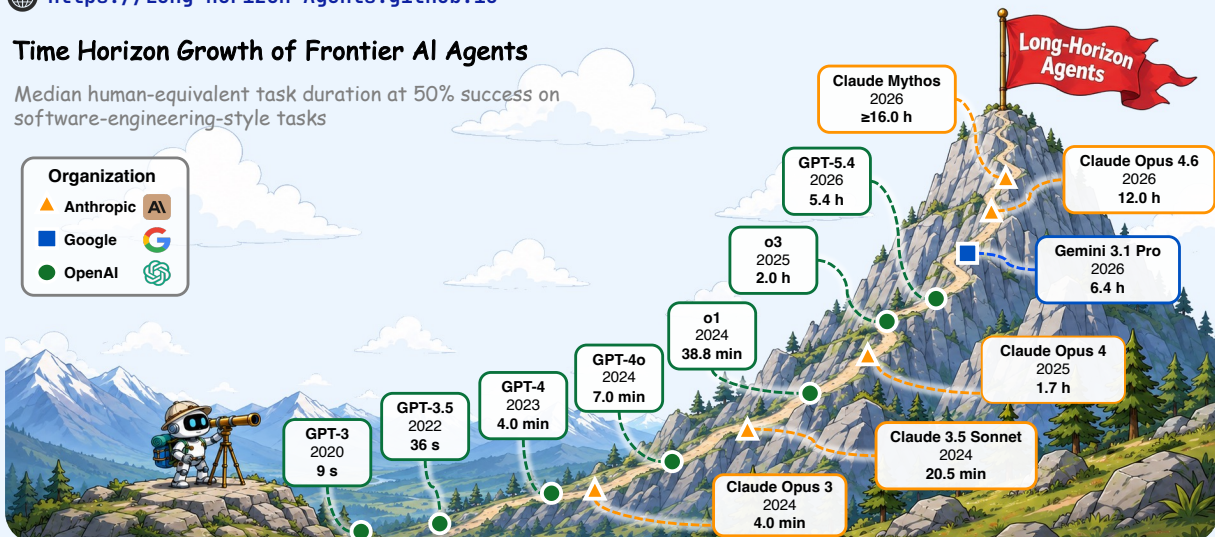
🌐 <https://github.com/RUC-NLPIR/Awesome-Long-Horizon-Agents>

🌐 <https://Long-Horizon-Agents.github.io>



Time Horizon Growth of Frontier AI Agents

Median human-equivalent task duration at 50% success on software-engineering-style tasks



*The empirical time-horizon trend of frontier agents: the task-completion horizon has climbed from seconds to over twelve hours, doubling roughly every seven months (<https://metr.org/time-horizons>).

Contents

1	Introduction	4
2	Foundation: Formalizing Long-Horizon Agents	7
2.1	Definition	7
2.2	Three Levels of Long-Horizon Tasks and Capabilities	10
2.3	Scaling the Time Horizon of Frontier-Agent Execution	11
2.4	Long-Horizon Agent vs. Neighboring Concepts	13
2.4.1	Long-Horizon Agent vs. Long-Running Agent	13
2.4.2	Long-Horizon Agent vs. Autonomous Agent	13
2.4.3	Long-Horizon Agent vs. Self-Evolving Agent	14
3	Evolution: From Prompting to Runtime	15
3.1	Stage I: Prompt Engineering (2020–2023)	16
3.2	Stage II: Context Engineering (2023–2025)	17
3.3	Stage III: Runtime Harnesses (2025–Present)	18
4	Harness: Externalizing Long-Horizon Capability	20
4.1	Loops and Workflows	20
4.1.1	Linear Workflows	21
4.1.2	Plan-Execute Workflows	21
4.1.3	Branching Workflows	22
4.2	Context and Memory	23
4.2.1	Working Context	23
4.2.2	Persistent Memory	24
4.3	Tools, MCP, and Skills	26
4.3.1	Tool Interfaces and Protocols	27
4.3.2	Active Tool Discovery	27
4.3.3	Skill Libraries	28
4.4	Orchestration	29
4.4.1	Decomposition and Roles	29
4.4.2	Coordination Topologies	30
4.4.3	Orchestration Optimization	30
4.4.4	Agent Protocols	30
4.5	Hooks and Middleware	31
4.5.1	Pre-defined Rule-based Hooks	31
4.5.2	Custom User-defined Hooks	32
4.5.3	Runtime-adaptive Hooks	33
4.6	Verification	34
4.6.1	Assessment Targets	34
4.6.2	Verification Levels	36
4.6.3	Verifier Strategies	36
5	Optimization: Internalizing Long-Horizon Capability	37
5.1	Architectural Substrate	37
5.1.1	Explicit-Context Architectures	38
5.1.2	Compressed-State Architectures	39
5.1.3	Hybrid Architectures	40
5.1.4	High-Throughput Mechanisms	40
5.2	Data and Environment Synthesis	40
5.2.1	Task Synthesis	41
5.2.2	Environment Synthesis	41
5.2.3	Trajectory Synthesis	42
5.3	Pre-training and Mid-training	43
5.3.1	Reasoning Priors	44
5.3.2	Long-Context State	44
5.3.3	Multimodal Perception	44
5.3.4	Data Mixture and Training Strategies	44

5.4	Fine-tuning	45
5.4.1	Instruction Selection and Mixing	45
5.4.2	Curriculum Learning	46
5.4.3	Distillation	46
5.5	Agentic Reinforcement Learning	47
5.5.1	Credit Assignment	48
5.5.2	Policy Optimization	50
5.5.3	Sampling Strategy	50
5.5.4	Interaction Patterns	51
5.6	On-Policy Distillation	52
5.6.1	Teacher-guided OPD	52
5.6.2	Self-improving OPD	53
5.7	Self-evolution	53
5.7.1	Offline Self-evolution	54
5.7.2	Online Self-evolution	54
5.7.3	Agent–Environment Co-evolution	55
6	Application: Long-Horizon Agents in Practice	55
6.1	Software Engineering	56
6.1.1	Repository Grounding	56
6.1.2	Workflow-level Planning	57
6.1.3	Feedback-driven Repair	58
6.2	Information Seeking	58
6.2.1	Deep Search	58
6.2.2	Wide Search	59
6.2.3	Multimodal Grounding	59
6.2.4	Research Synthesis	59
6.3	Computer Use	60
6.3.1	Browser Agents	60
6.3.2	Desktop GUI Agents	60
6.3.3	Mobile Agents	60
6.4	Multimodal Agents	61
6.4.1	Multimodal Understanding	61
6.4.2	Multimodal Generation	61
6.4.3	Omnimodal Agency	62
6.5	General-purpose Agents	62
6.5.1	Personal Assistants	62
6.5.2	Embodied Agents and World Models	63
6.5.3	Productive Agents	63
6.6	Benchmarks and Resources	64
7	Frontier: Open Challenges and Outlooks	67
7.1	Evolution: Generalization and Learning Over Time	67
7.1.1	Self-evolving Harness and Agents	67
7.1.2	Harness Generalization and Transferability	69
7.1.3	Continual and Lifelong Learning	69
7.2	Effectiveness: Acting in Realistic Environments	70
7.2.1	Real-world Environment Interaction	70
7.2.2	From Digital to Embodied Agents	71
7.3	Efficiency: Spending Compute, Context, and Modality Budgets	71
7.3.1	Cost- and Budget-aware Agency	72
7.3.2	Multimodal and Omni Harnesses	72
7.4	Trustworthiness: Robust and Governed Autonomy	73
7.4.1	Reflection and Error Robustness	73
7.4.2	Safety and Governance	74
8	Conclusion	74

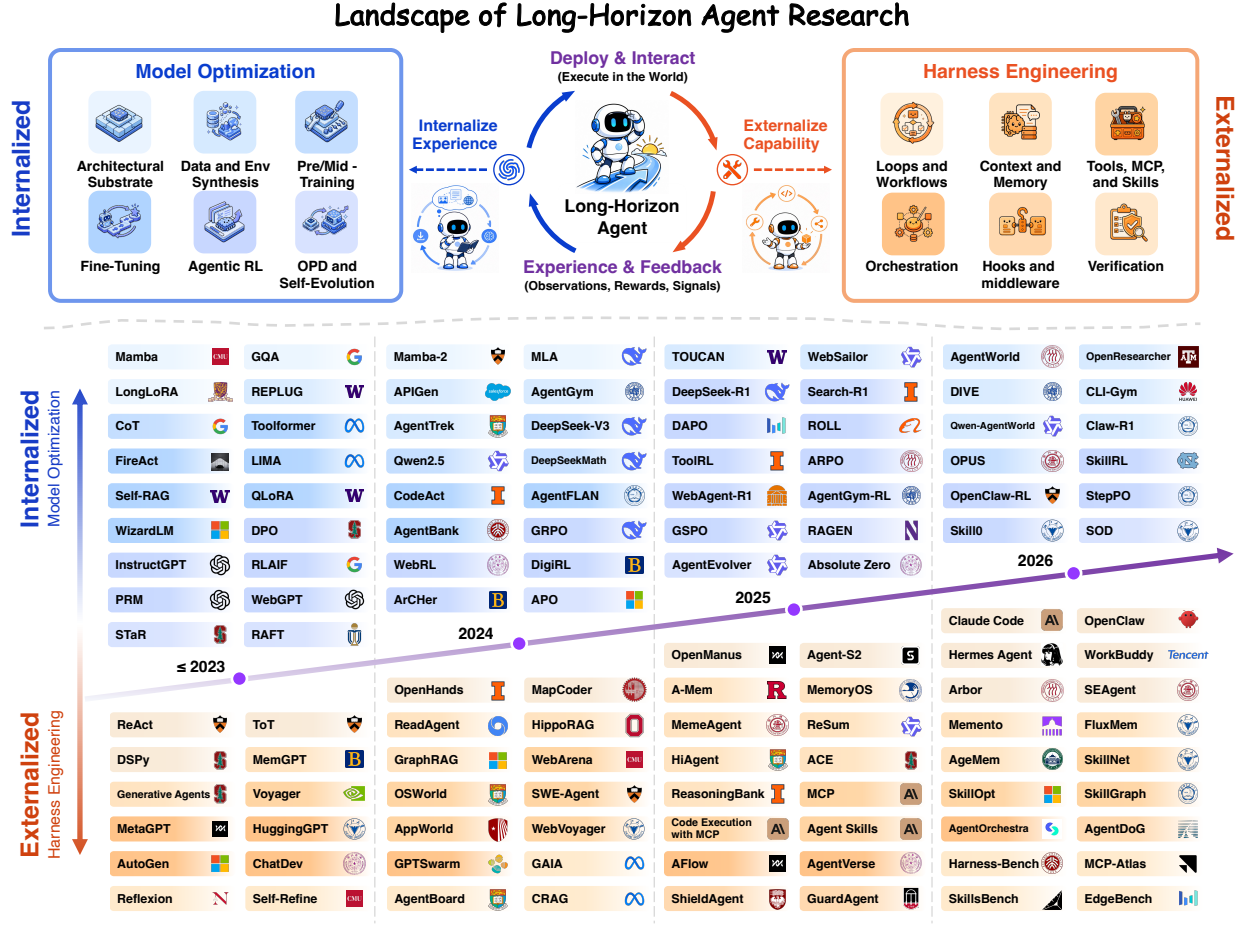


Figure 1 The landscape of long-horizon agent research. This paper adopts a co-evolutionary view of long-horizon agents, organized around *externalized harness engineering* (bottom) and *internalized model optimization* (top).

1 Introduction

Over the past few years, large language models (LLMs) have moved from single-turn chatbots to the decision-making core of autonomous agents in software engineering [260, 747], general-purpose assistance [431], scientific discovery [678], computer use [716, 885], and multimodal interaction [141]. These domains differ on the surface, but they share one decisive requirement that we call *long horizon*: they require persistent iteration across reasoning, tool use, observation, and revision over many interdependent steps—from those completable within a single context window to those that span windows, sessions, or open-ended task streams. To meet this requirement, *long-horizon agents* have emerged, expanding the boundaries of agent applications through continual, sustained interaction with real-world environments.

Long-horizon competence is now advancing rapidly and increasingly marks the practical line for delegating productive work to agents. For instance, frontier coding agents are already able to run for hours on a single project, occasionally close to a full day [14, 456]. Moreover, METR [286] reports that the time horizon of advanced agentic task execution has doubled roughly every seven months over the past several years, accelerating to about every four months for the most recent models. For example, GLM-5.1 is reported to sustain independent execution on a single task for up to eight hours, covering planning, implementation, testing, iterative refinement, and final delivery [179]. Notably, this capability cannot be obtained *simply* by optimizing and scaling the model’s internal parameters or enlarging its context window alone; its capability boundary also varies sharply with the *external* harness surrounding the model [123, 378, 468, 511]. Therefore, the central thesis of this paper is that *long-horizon agency is a system-level capability jointly shaped by two*

co-evolving components: externalized harness engineering and internalized model optimization.

Why long-horizon task execution is hard for agents. The difficulty of long-horizon task execution does not reduce to per-step accuracy: it arises from the *composition* of many tightly coupled steps into a single trajectory, rather than from any individual decision. Across the systems we organize, three representative failure modes recur throughout long-horizon execution and structure much of the technical discussion.

- **Goal drift and compounding error.** Over thousands of steps an agent can slowly diverge from its original objective, and because no step is perfectly reliable, local errors accumulate along the trajectory and eventually steer it onto a wrong reasoning path [20, 549].
- **Context rot and context-limit pressure.** As the working context fills, an agent tends to degrade abruptly once it passes a certain context-utilization threshold [378, 511]; conversely, an agent that perceives its context limit may terminate prematurely and declare partial progress a success [14].
- **Sparse, delayed rewards and irreversibility.** Many long-horizon tasks return a reward only at the end, leaving the agent with an extremely sparse learning signal over its intermediate decisions [808]; moreover, the longer an agent acts, the more likely it is to take a risky, irreversible action, so monitoring its execution safety and recoverability becomes essential.

Why long-horizon agents need a new taxonomy. Recent works have begun to clarify parts of this landscape, but its taxonomies still exhibit two structural limitations that motivate a new one.

- **Limited and unsystematic coverage.** Existing papers typically examine a *single* technical component of long-horizon agents in isolation, such as memory [694], context engineering [425, 841], agent harnesses [468, 876], self-evolution [165, 709], or agentic reinforcement learning [808]. Each is a genuine piece of the long-horizon agent, but none offers a comprehensive account of what such an agent *is*, nor of how to build an efficient and trustworthy long-horizon agent that stays reliable across a full trajectory.
- **Unclear definitions and overlapping terminology.** As interest in capable agents grows, the concept itself has become broad and blurred. Related notions such as long-context, long-running, autonomous, and self-evolving agents are routinely used interchangeably, even though each emphasizes a different facet, and reported gains entangle changes in the model, the harness, the data pipeline, and the evaluation protocol. Most works cannot say precisely what counts as long-horizon agency or how these techniques relate, which leaves improvements hard to attribute and hard to transfer.

These findings highlight the need for a unified taxonomy for long-horizon agent research.

We therefore organize the long-horizon agent field in this paper through a *co-evolutionary* view (Figure 1). Our organizing principle is that long-horizon agency is realized through the co-evolution of **externalized harness engineering** and **internalized model optimization**, two complementary halves that together constitute a long-horizon agent. Concretely, **externalized harness engineering** supplies runtime structure such as loops and workflows, context and memory, tools, orchestration, hooks and middleware, and verification; **internalized model optimization** improves the agent policy through architecture, data, environments, fine-tuning, reinforcement learning, and self-evolution. Neither half is sufficient on its own; long-horizon agency emerges only through their co-evolution, as harness mechanisms are gradually internalized into the policy and a stronger policy in turn unlocks more advanced harnesses.

Building on this co-evolutionary view, this paper aims to reconcile the fragmented definitions, connect the emerging trends, and articulate the foundational principles of long-horizon agents. Concretely, we organize the discussion around the following key questions:

Key Questions

- Q1. Foundation.** How should long-horizon agency be formalized, and how does it differ from long-running execution, autonomy, and self-evolution?
- Q2. Evolution.** How did the control surface widen from prompts, to context, to runtime agent systems?
- Q3. Harness.** How does externalized harness engineering keep long-horizon agent execution specifiable, steerable, verifiable, and recoverable at runtime?
- Q4. Optimization.** How can internalized model optimization build long-horizon competence across the agent’s architectural substrate, data and environment synthesis, and full training lifecycle?
- Q5. Application.** What forms do long-horizon agents take across domains, and how should their long-horizon capability be systematically evaluated?
- Q6. Frontier.** What open problems define the next stage as horizons grow?

To address these questions, the paper follows this co-evolutionary decomposition through six connected perspectives. **Foundation** formalizes the agent and separates it from adjacent concepts, while **Evolution** traces how the locus of control widened from prompts to context to runtime systems. The two halves of the argument are then treated in turn: **Harness** studies how long-horizon capability is *externalized* into a runtime layer, and **Optimization** studies how it is *internalized* into the policy. Finally, **Application** shows how the same long-horizon pressures recur across domains and how such systems are evaluated, and **Frontier** identifies what remains open. Crucially, these two halves are not independent: capability continually migrates between them, so throughout we treat externalized harness engineering and internalized model optimization as coupled rather than as separate tracks.

Contributions and Roadmap. This paper makes the following contributions, each answering one of the key questions and mapping to one part of the paper. Figure 2 presents an overview of our taxonomy.

1. **Foundation (Section 2).** We formalize long-horizon agency as a coupled model–harness decision process and organize its difficulty into three nested task levels (H1–H3) with matching capabilities (C1–C3): intra-context reasoning, cross-context memory, and cross-task experience. Building on METR’s data, we note that the frontier time horizon is scaling with an accelerating doubling time. We further separate long-horizon agency from long-running execution, autonomy, and self-evolution, giving the field a shared vocabulary for attributing where long-horizon competence comes from.
2. **Evolution (Section 3).** We trace how the control surface migrated from prompts, to context, to runtime systems, framing today’s long-horizon agents as the current endpoint of a capability-driven trajectory and clarifying why control first moved outward before it could be internalized.
3. **Harness (Section 4).** We organize externalized harness engineering by component: loops and workflows, context and memory, tools, MCP, and skills, orchestration and multi-agent control, hooks and middleware, and verification. Together these components show how a runtime layer keeps long execution specifiable, steerable, verifiable, and recoverable.
4. **Optimization (Section 5).** We organize internalized model optimization along the pipeline, from the architectural substrate and data and environment synthesis to agentic pre/mid-training, fine-tuning, reinforcement learning with long-horizon credit assignment, on-policy distillation, and self-evolution, tracing how capabilities first scaffolded through externalized harness engineering are progressively internalized into the policy.
5. **Application (Section 6).** Organized by the agent–environment interface, we map long-horizon pressures across software engineering, information seeking, computer use, multimodal agents, and general-purpose agents, revealing a common long-horizon core beneath surface-level domain differences, and consolidate horizon-aware evaluation resources.
6. **Frontier (Section 7).** We organize open problems into four axes, namely evolution, effectiveness, efficiency, and trustworthiness, spanning nine concrete directions that range from self-evolving and transferable

harnesses, to real-world environments and error robustness, to cost- and modality-aware efficiency, and to safety, governance, and embodiment, marking what must be solved as horizons keep growing.

Taken together, these contributions serve three purposes: a *clear definition* of long-horizon agency; a *high-quality taxonomy* (Figure 2) built around the paper’s six perspectives; and a *roadmap* sketching the directions toward capable long-horizon agents. Beyond organizing existing work, this co-evolutionary view also offers a vantage point for reasoning about how the balance between externalized harness and internalized optimization may shift as horizons grow, and thus for anticipating the frontier forms that long-horizon agents may take.

Scope and Boundaries. We delimit our scope along four boundaries. **(1)** This is not a general investigation of LLM agents, memory, or reinforcement learning: we treat memory, reinforcement learning, and self-evolution through the co-evolution of externalized harness engineering and internalized model optimization, and only as they bear on long-horizon competence. **(2)** We exclude general ML platform infrastructure (e.g., container orchestration, GPU-cluster scheduling), and consider inference and decoding only where they materially affect long-trajectory execution. **(3)** We address model-architecture internals only for their effect on horizon, i.e., making long trajectories representable, trainable, and affordable to decode, while still treating reasoning and long-context modeling as foundational capabilities. **(4)** We draw on official documentation, open-source implementations, and emerging specifications alongside indexed academic publications, prioritizing documented engineering practice.

2 Foundation: Formalizing Long-Horizon Agents

Overview. Section 1 positioned long-horizon agency as a system-level capability, jointly shaped by the internalized model optimization and the externalized harness engineering that surrounds it. This section makes that claim precise and lays the conceptual groundwork for the rest of the paper, organized around four questions that follow the section’s structure:

- **What is a long-horizon agent? (Section 2.1)** We define it by the coupled logical dependencies of a task rather than by elapsed time, and cast the agent as a base policy coupled to a surrounding harness, so long-horizon agency becomes a property of the model–harness system rather than of the foundation model alone.
- **How does its difficulty decompose? (Section 2.2)** We use how far execution stretches beyond one working context as an observable proxy for the underlying dependency burden, giving three nested levels (H1–H3) and their capabilities (C1–C3).
- **How fast is the frontier moving? (Section 2.3)** We quantify the rising time horizon of frontier agents as an empirical scaling trend and compare its full-period and recent-window doubling times.
- **How does it differ from neighboring notions? (Section 2.4)** We separate long-horizon agency from long-running execution, autonomy, and self-evolution, showing that it integrates these rather than reducing to any one.

2.1 Definition

We begin with the two objects of study: the task and the agent that seeks to complete it. As introduced in Section 1, we characterize long-horizon agency not by elapsed time or token count, but by the *coupled logical dependencies* that distribute a task across many interdependent steps. It is this chain of decisions, together with the interaction it entails, rather than duration alone, that determines the length of the horizon.

Long-horizon Tasks. A **long-horizon task** is a goal that may be under-specified at the outset and may require progressive clarification during solving, and whose solution path admits many possibilities and typically couples several complex subtasks together. As a result, it cannot be answered by a single-turn response in isolation, but only by *composing many interdependent decisions* into a single coherent trajectory [286, 811]. Concretely, a long-horizon task requires the agent to reason over many steps while interacting with the environment and, where relevant, the user: it progressively narrows down the solution path and continually tests and revises its reasoning direction until the goal is reached. Such a task may run from minutes to days

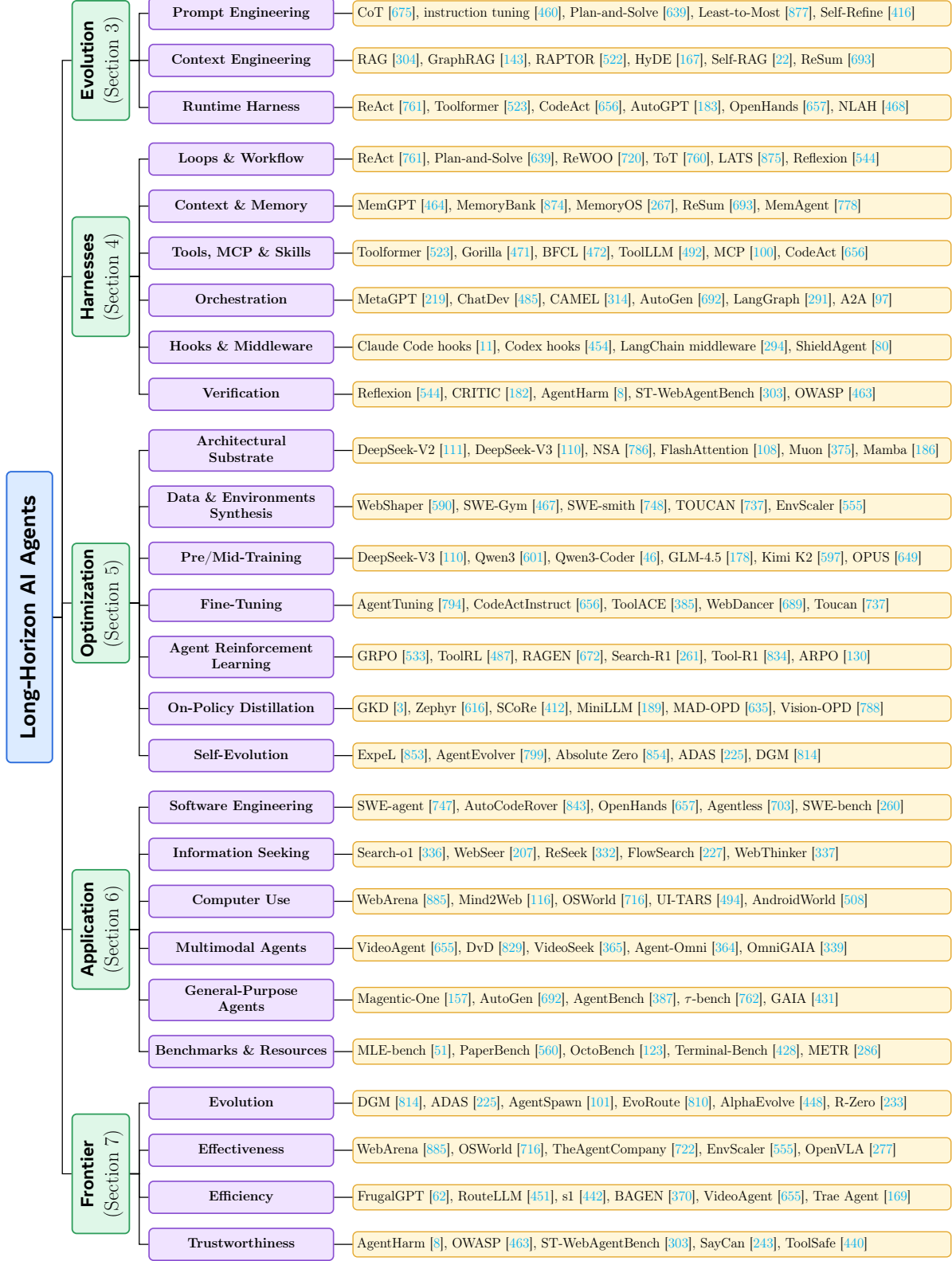


Figure 2 A taxonomy of long-horizon agent research. The paper is structured according to this taxonomy, progressing from **Evolution** (Section 3) through **Harnesses** (Section 4), **Optimization** (Section 5), and **Applications** (Section 6), to open **Frontiers** (Section 7). Each leaf lists representative works or systems discussed in the corresponding part of the paper.

and may fit within one context window or stretch far beyond it; its defining feature is not duration itself but this sustained, feedback-driven dependence among steps and the continual interaction required to manage it.

Long-horizon Agents. A **long-horizon agent** is a system designed to solve such tasks in an environment with state, feedback, and side effects [640]. Although the stated intent may be simple, successful execution requires resolving a large and tightly coupled chain of logical dependencies. Unlike a single-turn responder, it does not produce one answer and stop; instead, it must sustain reasoning through repeated cycles of planning, action, observation, and revision, progressively clarifying an under-specified goal as the trajectory unfolds. As argued in Section 1, this competence is not a property of the foundation model alone. It arises from the joint design of an **externalized harness engineering**, which supplies runtime structure such as tools, memory, orchestration, and checks, and an **internalized model optimization** process, which improves the policy through data, training, and experience.

Formalization: Long-Horizon Agent

We model a long-horizon agent as a foundation-model *base policy* π_θ coupled to a surrounding structure $\mathcal{H}$ that closes the loop around it:

$$\text{Agent} = \pi_\theta \oplus \mathcal{H} \quad (1)$$

Here π_θ proposes actions, while $\mathcal{H}$ comprises the loops and workflows, system prompts, tools and skills, context and memory, orchestration logic, hooks, and verification mechanisms that govern *when* the model is invoked, *how* its actions are validated and executed, and *what* state persists across steps. The operator $\oplus$ denotes runtime coupling rather than simple addition: $\mathcal{H}$ closes the action–observation–memory loop around π_θ , so long-horizon agency is a property of the coupled system and its training, not of π_θ alone. Equation (1) describes the *deployed* coupling that produces long-horizon behavior at run time; how it *evolves* over time, as π_θ internalizes competence through training and the system accumulates experience, is the co-evolving recipe we develop in Section 5 and Section 3. We refer to these two halves as **Pillar I**, the externalized harness $\mathcal{H}$ (Section 4), and **Pillar II**, the internalized optimization of π_θ (Section 5).

The Long-horizon Agent–Environment Loop. Since a long-horizon agent makes progress precisely by interacting with its environment step after step, its execution is most naturally described as a continual loop in which it acts, observes the consequences, and updates its state before acting again. Following the standard LLM-based agent formulation [228, 640, 761], we place the coupled system in an environment. Let $\mathcal{S}$ be an environment state space, $\mathcal{O}$ an observation space, and $\mathcal{A}$ an action space. The environment evolves by a transition kernel $s_{t+1} \sim \Psi(\cdot \mid s_t, a_t)$ and emits observations through an observation kernel $o_t \sim Z(\cdot \mid s_t)$, which is partial and in general stochastic. We distinguish the base policy π_θ from the *agent policy* $\pi = \pi_\theta \oplus \mathcal{H}$ that the harness closes around it: at each step the harness assembles a contextual signal c_t and the base policy acts on it,

$$c_t = \mathcal{H}(o_{0:t}, a_{0:t-1}, \mathcal{Q}), \quad a_t \sim \pi_\theta(\cdot \mid c_t),$$

where $\mathcal{Q}$ is the task specification and c_t is the contextual signal that the harness assembles at step t from the interaction history: the prompt window with retrieved memories, prior tool outputs, reasoning traces, and other state. Execution induces a trajectory $\tau = (s_0, o_0, a_0, s_1, o_1, a_1, \dots, s_T)$ that runs until termination, i.e. until the first step $T = \min\{t : s_t \in \mathcal{S}_{\text{term}}\}$ at which the state enters a terminal set $\mathcal{S}_{\text{term}}$ (the goal is reached, a budget is exhausted, or the agent halts). Because o_t is only a partial view of s_t and c_t is a lossy summary of history, the coupled system is naturally modeled, following standard POMDP formulations for sequential decision making under partial observability [264], as a partially observable Markov decision process (POMDP). Much of what $\mathcal{H}$ does is the machinery of acting under partial observability. The task assigns each trajectory a utility $R(\tau) \in \mathbb{R}$, typically sparse and delayed and often revealed only at T , so that credit for the intermediate decisions that actually drive success or failure must be inferred from a single terminal signal. This is the long-horizon credit-assignment problem that is central to reinforcement learning methods [579].

Three Levels of Long Horizon Tasks & Capabilities

H1 $\subset$ H2 $\subset$ H3 | C1 $\subset$ C2 $\subset$ C3

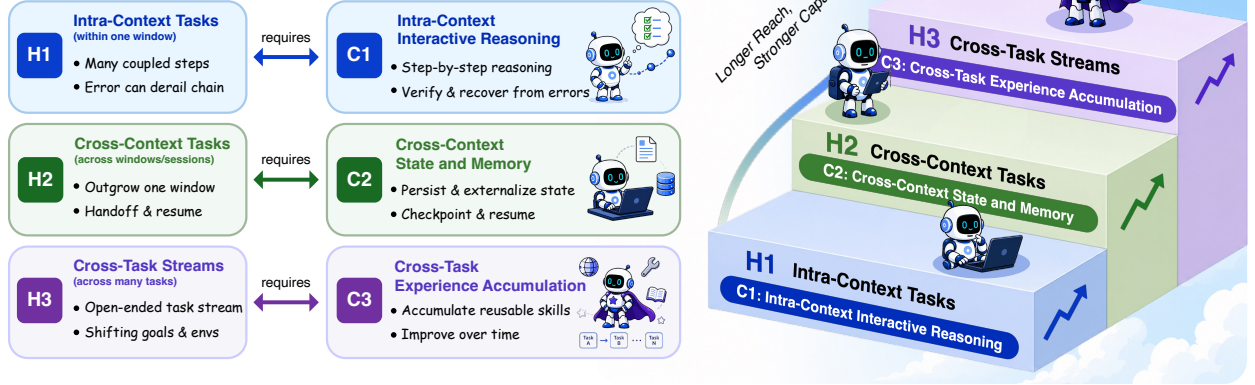


Figure 3 Three levels of long-horizon tasks and the capabilities they demand. As the required horizon widens from intra-context tasks (H1), to cross-context tasks (H2), to cross-task streams (H3), the agent must add intra-context interactive reasoning (C1), cross-context state and memory (C2), and cross-task experience accumulation (C3) capabilities.

2.2 Three Levels of Long-Horizon Tasks and Capabilities

Although the notion of a *long-horizon* agent is by now widely used, it remains conceptually vague, and prior work still lacks a principled way to partition long-horizon competence into distinct levels. Building on the definition above, we organize long-horizon tasks by an operational axis: *how far execution must stretch beyond a single working context*, ranging from intra-context tasks completable within one window (H1), to cross-context tasks that span windows or sessions (H2), to cross-task streams in which an overarching goal unfolds over many episodes (H3). This axis is a proxy for, rather than the definition of, logical dependency: a context boundary matters when coupled decisions and task state must survive it, whereas a long but decomposable batch process need not be long-horizon. The proxy is useful because no step is perfectly reliable and errors compound along the dependency graph, so trajectory-level reliability rather than per-step accuracy governs success [286]. It yields three nested operational levels of increasing reach [14, 228], and we pair each level with the capability an agent needs to succeed at it (Figure 3). Because every level adds a new failure mode on top of the one below, the corresponding demands accumulate. The time ranges below are therefore descriptive of current systems, not definitional thresholds.

Three Levels of Long-Horizon Tasks (H1 $\prec$ H2 $\prec$ H3)

H1 — Intra-context, within one window (~minutes).

A single task can be completed within one working context window, yet it is far from a one-shot query: it requires composing many interdependent decisions into one coherent trajectory, with repeated interaction with the environment and, where relevant, the user to test and revise the reasoning direction until the goal is reached [286, 544].

H2 — Cross-context, across windows/sessions (~hours to days).

A single task far exceeds one context window or session and must be carried across multiple context windows or by several agents working in concert. The agent(s) must therefore actively compress and externalize historical context, track interaction state in real time, and resume progress without assuming that the full history remains visible at every step [14, 378, 511].

H3 — Cross-task, open-ended task stream (~lifelong).

Tasks arrive as an open-ended stream rather than a closed episode. A single overarching goal may therefore span many tasks, each of which may itself require multiple context windows to complete. In deployment, such settings often demand persistent, and sometimes lifelong, operation across heterogeneous environments, tools, and sessions [165, 228, 580, 808].

So far we have organized long-horizon *tasks* into three difficulty levels; we now turn from tasks to *agents* and identify, for each level, the capability that its difficulty demands. Each level exposes a distinct insufficiency. H1 shows that per-step accuracy alone is insufficient: the agent must sustain reasoning through continual interaction rather than produce a single response. H2 shows that a larger context window is insufficient: once a task spans multiple windows or agents, state must be compressed, externalized, and tracked even when the full history is not continuously in view. H3 shows that episodic success is insufficient: an agent that completes an individual task may still fail to accumulate reusable competence across an open-ended stream of subsequent tasks. Each level therefore demands a capability of matching reach, denoted C1–C3 respectively, and these capabilities compose cumulatively: C2 presupposes C1, and C3 presupposes both.

Capabilities Demanded by Each Level (C1 < C2 < C3)

C1 — Intra-context interactive reasoning (for H1).

Within a single context, sustain continuous reasoning through interaction with the environment and the user: take actions, absorb feedback, and revise the trajectory so that many interdependent decisions compose into one coherent path toward the goal [286, 544].

C2 — Cross-context state and memory (for H2).

Building on C1, maintain task state when execution spans windows or agents: compress and externalize historical context, read and write memory, checkpoint progress, and faithfully resume after hand-offs [14, 378, 511, 694].

C3 — Cross-task experience accumulation (for H3).

Building on C2, turn experience into lasting competence across an open-ended task stream: switch among complex tasks, each of which may itself span multiple context windows, accumulate reusable skills [630], generalize from past experience [462], and continually improve as goals, environments, and data distributions shift [165, 228, 580].

Because the levels nest, so do the capabilities, in that an agent operating at H3 must still satisfy C1 and C2 within every task it touches. This is where the paper’s central question enters. Each capability can be supplied either from *outside* the model, through the harness, or from *inside* it, through training. H1 pressures are often met by C1 within a single context, or absorbed into the model weights; H2 pressures push C2 out of the window into durable external state, namely compressed context, memory, and checkpoints; and H3 pressures demand C3, which turns deployment experience into lasting competence. In many H3 settings, however, neither route is likely to suffice on its own, and the highest level of long-horizon competence is therefore inherently a *system-level* property, jointly constructed by internalized model weights and the externalized harness rather than achievable by either in isolation (Figure 1). We attach each method to the level it primarily serves.

2.3 Scaling the Time Horizon of Frontier-Agent Execution

In the previous subsection, we organized long-horizon difficulty and its corresponding capabilities into a stair-stepped ladder *qualitatively*; here, we quantify how an observable proxy for the frontier scales over time. We build on the measurements of METR [286], which define the *50%-task-completion time horizon* as the human-expert completion time for the class of agentic engineering tasks that a given agent is predicted to complete with 50% reliability. These tasks span software engineering, machine learning engineering, and computer use, providing a more objective proxy for task difficulty than task labels alone.

Figure 4 plots METR’s live estimates through May 8, 2026 [286, 429]¹. Because several early models were not re-evaluated under the newer TH1.1 methodology, the full historical series combines TH1.0 estimates for those models with TH1.1 estimates for later models. Under this stitched series, the time horizon of frontier agents rises from seconds for early models (e.g. GPT-2) to approximately twelve hours for Claude Opus 4.6 and at least sixteen hours for an early Claude Mythos Preview by May 2026, spanning nearly four orders of magnitude within only a few years.

¹METR live dashboard, <https://metr.org/time-horizons/>, accessed May 8, 2026.

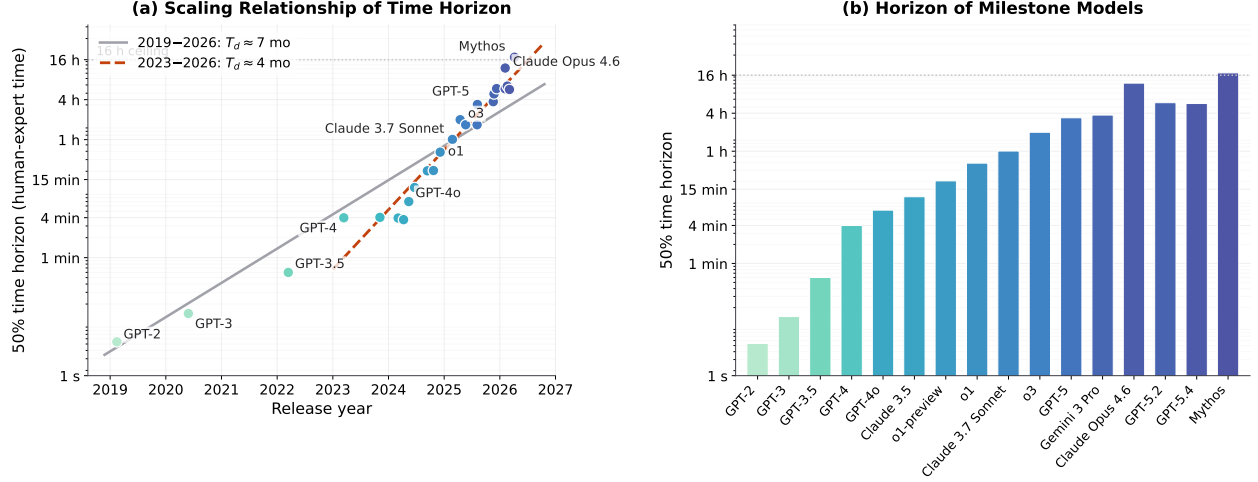


Figure 4 Empirical growth of frontier-agent time horizons. Frontier agents’ 50%-task-completion time horizons over model release dates, shown for the full fitted series in (a) and selected milestone agents in (b).

The fitted trend in Figure 4 summarizes this rapid expansion. Across the stitched full-period series, the estimated doubling time is approximately 196.5 days, or 6.5 months. Restricting the fit to models released from 2023 onward under TH1.1 yields a shorter doubling time of approximately 130.8 days, or 4.3 months [286, 429, 683]. The dotted line marks the approximately sixteen-hour region above which METR considers estimates unreliable under the current task suite. Consequently, both individual estimates and fitted trends near or beyond this boundary carry substantial uncertainty.

To summarize the trend formally, we borrow the functional spirit of the compute-optimal scaling law of Hoffmann et al. [217] and model the frontier horizon in terms of two quantities: the release date of a frontier agent and the human-expert duration of tasks that the evaluated system can complete. We write this relationship as a compact descriptive summary:

$$\log_2 H(t); \approx; \log_2 H_0 + \frac{t - t_0}{T_d}, \quad \text{equivalently} \quad H(t); \approx; H_0 \cdot 2^{(t-t_0)/T_d}, \quad (2)$$

where $H(t)$ is the estimated time horizon of a frontier agent whose base model was released at date t , t_0 is a reference date, $H_0 = H(t_0)$ is the fitted horizon at that date, and T_d is the *doubling time*. Under this functional form, the full-period and 2023-onward fits correspond to $T_d! \approx 196.5$ days and $T_d! \approx 130.8$ days, respectively. Although these estimates are derived from self-contained, automatically scorable software-style tasks and become less precise near the upper boundary of the benchmark, these limitations primarily constrain the exact fitted values rather than the overall direction of the trend.

The decisive signal is that the task duration frontier agents can reliably handle has increased approximately exponentially. The shorter doubling time obtained for the 2023-onward subset further suggests that this expansion may have accelerated, although differences in evaluation coverage and methodology prevent treating the two fitted values as definitive evidence of acceleration. The trend nevertheless aligns directly with our capability hierarchy: as completable task horizons expand from minutes to hours and then to tens of hours, frontier agents move beyond intra-context H1 tasks, increasingly address H2 tasks that require cross-context state and memory, and begin to approach the boundary of cross-task, open-ended H3. In this sense, the sustained expansion of the measured time horizon is an observable projection of long-horizon competence climbing the H1→H2→H3 ladder. At the current pace, many multi-hour tasks that remain at the frontier today may become routine in the foreseeable future. That said, the trend characterizes *progress over time*; it does not explain its *source*. In particular, it does not distinguish whether the gains arise from a stronger policy π_θ , a better harness $\mathcal{H}$, or the co-evolution of the two. Attributing observed progress among these factors is the central question of this paper. We therefore turn from measuring the frontier to asking what drives it.

2.4 Long-Horizon Agent vs. Neighboring Concepts

Long-horizon agency is often conflated with three adjacent notions: long-running execution, autonomy, and self-evolution. They are related because all appear in modern agent systems, but they answer different questions. Long-running asks how long execution persists, autonomy asks how much human control remains, and self-evolution asks whether the agent improves itself over time. Long-horizon agency asks a different question: can the coupled system sustain interactive reasoning within a context (C1), maintain state and memory across contexts when needed (C2), and accumulate experience across tasks when deployment demands it (C3)? Figure 5 summarizes the overlap and separation.

2.4.1 Long-Horizon Agent vs. Long-Running Agent

A long-running agent is defined by duration: it keeps making progress over hours to weeks across many windows, sessions, or sandboxes, with checkpointing and resumption [14]. It often appears in long-horizon settings, but duration and difficulty are not the same.

Overlap. Many genuinely long-horizon tasks are also long-running, because their trajectories outgrow one context window or one execution session. In these cases the same engineering mechanisms that instantiate C2 matter: compressing and externalizing historical context, durable memory, progress logs, checkpoints, and faithful resumption. Long-running engineering is therefore a common implementation requirement in H2 settings, where task state must survive resets, context boundaries, and hand-offs without assuming that full history remains visible.

Distinctions. The key distinction is that duration and *logical coupling* are independent properties. A task may run for days while consisting mostly of logically decoupled, independently checkpointable steps; a script that loops over a large dataset and applies the same operation runs for a long time, yet its steps carry few interdependencies, so it demands neither dense, sustained interactive reasoning (C1) nor live management of cross-context state (C2). Conversely, a task may complete within a single window while still requiring many interdependent decisions whose direction must be continually tested and revised through interaction with the environment. Long-running engineering thus optimizes persistence, throughput, and cost over time, whereas long-horizon agency concerns whether an agent can sustain reasoning through continual interaction and, when the task demands it, maintain the state and memory needed to carry that reasoning across contexts [286]. Accordingly, execution duration captures temporal persistence, whereas long-horizon agency denotes the capacity to sustain coupled reasoning and state management across the task’s dependency structure.

2.4.2 Long-Horizon Agent vs. Autonomous Agent

An autonomous agent is defined by how much it can act without user involvement, often formalized as roles ranging from operator to observer [155]. Autonomy and long-horizon agency are strongly related, but they describe different properties of a system.

Overlap. The two overlap because long-horizon tasks make constant human supervision expensive, so most long-horizon agents are largely automated: they interact with external environments and tools on their own, and autonomously decide when to revise their strategy. If an agent must take many steps, interact with tools and the environment, absorb feedback, and revise its trajectory, then requiring approval at every step can destroy the usefulness of the system, even when the task itself fits within H1. At the same time, autonomy raises the safety stakes of long-horizon execution, since more unattended steps and more coupled side effects mean a larger blast radius, so permission gates and oversight remain central.

Distinctions. The distinction is that autonomy is only a partial, governance-oriented dimension of a system, not a measure of its capability. A strong long-horizon agent can be run with low autonomy behind approval gates, whereas a highly autonomous agent may execute only a short, single-window task, such as a single web action [452]. Moreover, long-horizon agents need not be fully autonomous: their harnesses can introduce permission gates, approval checkpoints, and escalation policies that selectively involve humans at high-impact or uncertain steps. Autonomy therefore asks who remains in the loop, whereas long-horizon agency asks

Long-Horizon Agent vs. Neighboring Concepts

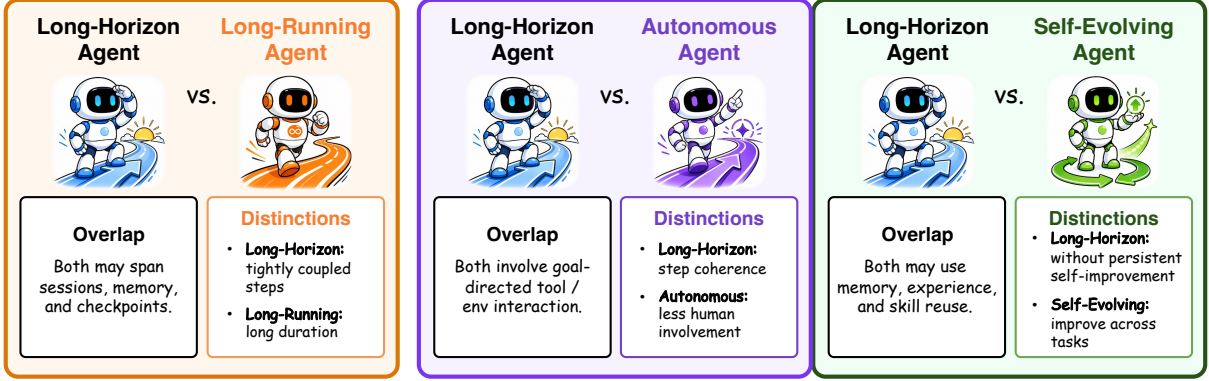


Figure 5 Long-horizon agent and three neighboring concepts. Long-running execution, autonomy, and self-evolution each emphasize a different facet; long-horizon agency asks whether the coupled model–harness system can sustain interactive reasoning (C1), maintain state and memory across contexts when needed (C2), and accumulate experience across tasks when deployment demands it (C3).

whether the coupled system can sustain the reasoning, state management, and, at H3, experience accumulation that the task horizon demands. In short, autonomy controls the degree of human involvement, whereas long-horizon agency measures whether execution can be carried through the required horizon.

2.4.3 Long-Horizon Agent vs. Self-Evolving Agent

A self-evolving agent is defined by experience-driven, *persistent* change to its own parameters, memory, tools, or topology, with the goal of improving future performance. It is the closest neighbor to long-horizon agency because both concern competence beyond a single model call, but they still differ in scope.

Overlap. At the cross-task (H3) scale the two overlap strongly. A long-horizon agent that switches among complex tasks, accumulates reusable skills, consolidates memory, and reuses experience across episodes begins to approach self-evolution. Both also fit the “era of experience” view that agent competence should grow from interaction rather than only from a frozen pretraining corpus [580]. In this paper, self-evolution is one internalized mechanism for realizing C3, namely persistent cross-task experience accumulation.

Distinctions. The distinction is whether experience persistently changes the agent for the future. Self-evolution requires such persistent change and is therefore only one facet of the C3 capability, cross-task experience accumulation, rather than the whole of long-horizon agency, which also rests on core capabilities such as interactive reasoning, tool use, and long-context reasoning. Consequently, many long-horizon agents operating at H1 or H2 complete difficult tasks with a fixed policy and temporary state, finishing a trajectory without updating their weights, tools, or lasting memory. Such agents lack any self-evolution capability, yet they remain long-horizon agents.

Self-evolution therefore concerns persistent improvement *between* tasks, while long-horizon agency additionally concerns reliably carrying each task *through* the extended interaction, state management, tool use, and iterative reasoning that its dependency horizon demands.

Overall, this section has laid the conceptual foundation on which the rest of the paper builds. We defined a long-horizon agent as a coupled model–harness system, $\pi_\theta \oplus \mathcal{H}$, rather than as the foundation model alone (Section 2.1); organized its difficulty and matching capabilities into three nested levels, H1–H3 and C1–C3 (Section 2.2); quantified the empirical growth of a frontier-agent time-horizon proxy and compared alternative fitting windows (Section 2.3); and separated long-horizon agency from long-running execution, autonomy, and self-evolution (Section 2.4). The methodological takeaway is that long-horizon agency is a high-level, system-level capability that integrates these adjacent properties rather than reducing to any one of them. An agent may exhibit sustained running, autonomy, and self-evolution and yet fail an H1 task when it cannot

Table 1 Three stages in the co-evolution of long-horizon agents. Each stage widens the *control unit*, opens a new *control surface*, and advances a set of *key technical routes*. The graded *long-horizon profile* on the right scores each stage against the three accumulating capabilities defined in Section 2.2, namely C1 intra-context interactive reasoning, C2 cross-context state and memory, and C3 cross-task experience accumulation (● full, ◐ partial, ○ absent).

Stage (era)	Control unit & surface	Key technical routes	Long-horizon profile		
			C1	C2	C3
I. Prompt engineering (2020–2023)	Single query <i>language of the prompt</i>	• Chain-of-thought reasoning • Decomposition, planning, and search • Prompt optimization and internalization	◐	○	○
II. Context engineering (2023–2025)	Conditioned call <i>information in the context</i>	• Retrieval-augmented generation • Tool use and function calling • Long context, memory, and context engineering	◐	◐	○
III. Runtime harnesses (2025–Present)	Whole trajectory <i>runtime around the model</i>	• Agent loops (reason–act–observe) • Harness engineering • Internalizing the loop	●	●	◐

sustain interactive reasoning through environment feedback, or fail an H2–H3 task when it lacks the state management or experience accumulation that those horizons require. Competence therefore shifts from the model alone to the coupled system $\pi_\theta \oplus \mathcal{H}$ and its training, and evaluation must track task completion, robustness over interaction, cross-context recovery, cross-task transfer, cost, and safety rather than static per-call accuracy. Having framed long-horizon agency as a coupled model–harness property organized along H1–H3 and C1–C3, we now turn to a historical question, namely how the field progressively moved capability from prompts, to context, to persistent runtime systems. Section 3 traces that progression.

3 Evolution: From Prompting to Runtime

As stated in the previous sections, we define a long-horizon agent as the joint product of *internalized model optimization* and *externalized harness engineering*, and reading the field through the lens of evolution shows why this coupling is not a recent design choice but a recurring necessity. Across the history of LLMs, the base model and the engineering structure around it have advanced together, each repeatedly compensating for the other’s limits. A base model is a powerful but fragile next-step predictor that, on its own, does not reliably decompose a goal, ground its steps in the world, sustain reasoning through environment interaction, or maintain the cross-context state that longer horizons require (Sections 1 and 2.2), and supplying this missing structure has always been the task of the surrounding engineering. Following this co-evolutionary thread, this section traces the development of long-horizon agents along its two driving forces, model optimization and harness engineering, and organizes the representative work that defines each stage.

As shown in Figure 6, we trace this co-evolution through three nested, chronologically and thematically coherent stages, each opened by an expansion in the base model’s own capability, organized around a new control surface exposed by the surrounding harness, and closed by the long-horizon limit that forces the next stage. **(1) Prompt Engineering** (2020–2023) steers behavior through the *language* of a single query, first eliciting multi-step reasoning from a fixed model; **(2) Context Engineering** (2023–2025) extends control to the *information* supplied to the model, grounding the agent in retrieved evidence, tools, and memory; and **(3) Runtime Harnesses** (2025–present) make the unit of control the whole *trajectory*, so that the long-horizon agent proper emerges as a model coupled to a runtime that sustains one goal over many dependent steps. The three are not successive replacements but nested layers, in which each new control surface absorbs the previous one rather than discarding it. Throughout this chapter, the years attached to each stage denote the period in which its control surface became the *dominant* paradigm, not the publication date of any individual work; a stage may therefore rest on foundational methods that predate it and preview ideas that mature later. Table 1 summarizes the three stages along a common set of axes.

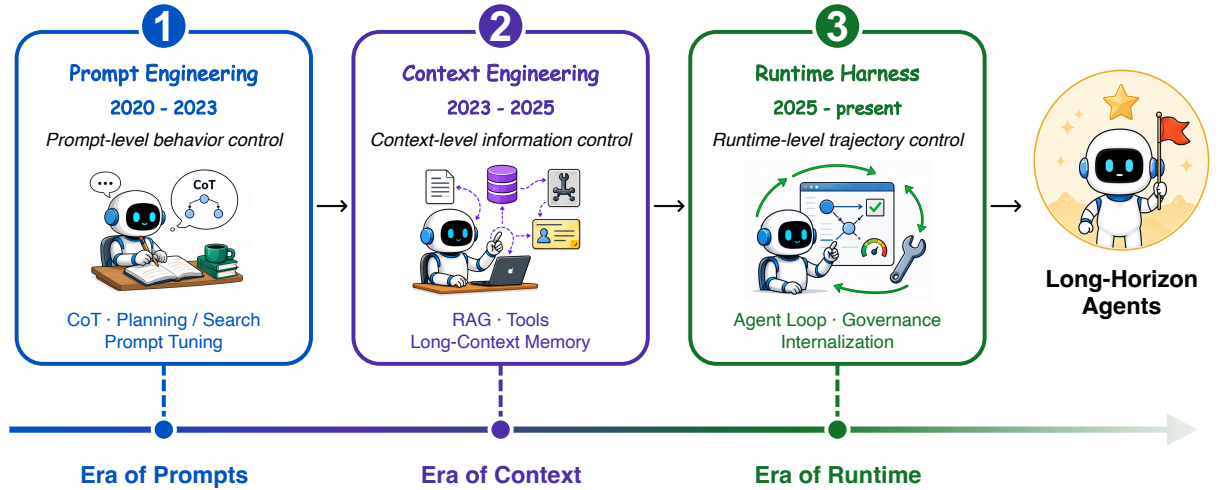


Figure 6 The co-evolution of long-horizon agents across three stages. Control widens from the *language* of a single prompt (2020–2023), to the *information* supplied to the model before it acts, to the whole *trajectory* sustained by a runtime harness (2025–present); each stage absorbs the previous rather than replacing it.

3.1 Stage I: Prompt Engineering (2020–2023)

The prompting stage is the prototype of long-horizon agency, in which the model carries most task capability in its weights and the only surface a builder can touch is the language of the query. The stage opens with *prompt learning*, made widely visible by GPT-3 [40] and consolidated by dedicated papers on prompting and in-context learning [132, 379], which let a pre-trained model perform a new task from a handful of in-prompt examples with no weight update and thereby moved the focus of system building from training the model to shaping its input at inference time. Its central discovery is that the right prompt can *elicit* latent abilities the base model does not show by default, including the embryonic forms of planning and execution that long-horizon tasks demand. Concretely, the prompting stage surfaces this latent competence along three representative threads, which we take up in turn: chain-of-thought reasoning, decomposition and planning, and the optimization and internalization of prompts themselves.

Chain-of-Thought Reasoning. The first thread showed that a fixed model reasons far better when prompted to externalize intermediate steps rather than answer directly. Initially, the scratchpad [450] emitted intermediate computation for multi-step problems; chain-of-thought (CoT) prompting [675] then turned this into a few-shot recipe that elicits a reasoning trace before the answer; and zero-shot CoT [281] further showed that the same behavior could be triggered by a single instruction, confirming that the capacity was latent in the weights and merely needed surfacing. Building on CoT, a family of variants quickly followed that automate demonstration construction, scale reasoning effort with problem complexity, and modularize the prompt into reusable sub-steps [160, 275, 851]. The shared lesson, that the same model exhibits qualitatively different competence depending on how its input is phrased, recurs throughout the paper.

Decomposition, Planning, and Search. A second thread widened the single chain into the structures that prefigure an agent, using prompting not merely to reason but to make a fixed model *decompose* a goal, *plan* a course of action, and *execute* it step by step, which is the earliest recognizable form of an agent. Self-consistency [659] samples many chains and aggregates them by majority vote, trading one fragile path for a more reliable distribution; least-to-most prompting [877] made decomposition explicit by solving ordered subproblems; and Plan-and-Solve prompting [639] split an explicit planning step from execution, producing the first clean “plan then act” template that later agent loops inherit. A parallel line grounded execution in code, with PAL [168] and Program of Thoughts [70] offloading computation to an interpreter rather than the model’s own arithmetic, while Tree-of-Thoughts [760] recast reasoning as deliberate search over a tree of partial solutions with look-ahead and backtracking. A closely related line then pushed the

same plan-then-act template into *interactive* environments, where SayCan and Inner Monologue grounded language plans in affordances and environment feedback [236, 243], and AdaPlanner, ADaPT, and DEPS closed the loop with adaptive re-planning from execution feedback in embodied and text worlds such as ALFWorld [482, 545, 569, 673]. Read together, this line forms the embryonic agent skeleton, which decomposes a goal, proposes and executes steps, evaluates progress, and revises. Prompt learning alone cannot yet yield a long-horizon agent, since all of this unfolds without a persistent runtime harness, durable state, or robust recovery. What the stage does provide is a soft external constraint on model behavior and an early exploration of the harness-mediated control that later stages systematize, together with the planning-and-execution skeleton here and the elicitation-then-internalization dynamic below.

Prompt Optimization and Internalization. A third thread responded to a property this stage exposed and the rest of the paper inherits, namely that elicited capability is *fragile*, swinging with wording, example order, example choice, or output format [403, 433, 862], which made prompt-based systems hard to reproduce, test, and compose. The community answered by making prompt construction systematic rather than handcrafted. One line recast prompt design as an explicit search or optimization problem [483, 889], while another treated prompts as composable, program-like artifacts [514]. In parallel, what began as a prompting trick started to be *internalized*, as instruction tuning together with reinforcement learning from human feedback let models follow intent without elaborate templates, most visibly in InstructGPT [460], and the same dynamic would later fold full chain-of-thought traces into the weights, which we revisit in Section 5. This is the first concrete instance of the harness→policy migration the paper tracks across all three stages.

Although prompt engineering by itself does not constitute a long-horizon agent, its core idea, shaping a model’s reasoning behavior through external constraints rather than weight updates, remains a guiding principle for long-horizon agents to this day. Yet prompting controls only *language* over a fixed knowledge base. A single call can use only the text in the prompt and the knowledge already stored in the parameters [266, 479, 516], so no phrasing can supply information the model never saw. As applications moved toward current events, private data, large repositories, and changing APIs, the binding question shifted from how the instruction is worded to what the model is allowed to see, pushing engineering one layer outward, from the language of the prompt to the information assembled into it.

3.2 Stage II: Context Engineering (2023–2025)

Where Stage I engineered the model’s *behavior* through the language of the query, Stage II turns to its *information*, that is, the evidence, tools, and history that a single call is allowed to condition on. Broadening this input, rather than rewording the instruction, becomes the primary means of extending the model’s capability boundary. The representative directions of this stage are retrieval-augmented generation, tool use and function calling, and context management and memory, which we detail in turn below.

Retrieval-Augmented Generation. Retrieval-augmented generation (RAG) loosened the boundary between the model and the world by attaching external evidence to the prompt at inference time [304], building on a line of retrieval- and memorization-augmented models that coupled parametric knowledge with an external corpus [38, 200, 246, 247, 270, 273] and organized systematically in dedicated RAG reviews. Its core result is that grounding generation in retrieved text reduces hallucination and improves factuality on knowledge-intensive tasks. Initially a single retrieve-then-read pass, the technique soon evolved into a richer family. HyDE-style hypothetical-document embeddings first improved recall [167], and hierarchical indexing summarized corpora into multi-resolution trees [522]. Retrieval was then interleaved with reasoning, through multi-hop methods that chain retrieval across steps [612] and active retrieval that lets the model decide *when* to look something up [258]. More recently, self-reflective retrieval further lets the model critique and re-retrieve its own evidence [22].

Tool Use and Function Calling. Where retrieval lets a model *read* the world, tool use lets it *query and act on* systems beyond text. Early work established the mechanics. Toolformer [523] showed that a model could teach itself, in a self-supervised way, when and how to call APIs; ToolAlpaca [585] and ToolLLM [492], with the StableToolBench testbed [198], then scaled tool use to thousands of real-world APIs; Gorilla [471] grounded API selection in documentation to curb hallucinated calls; and WebGPT [445] grounded long-form

answering in live browsing. As the practice consolidated, function calling and structured outputs standardized the interface so that model decisions could be parsed and dispatched reliably [385, 472], a maturation charted by dedicated tool-learning papers [493, 497]. The decisive bridge is ReAct [761], which interleaves reasoning with tool calls so that each observation feeds the next thought, while HuggingGPT [537] further showed how an LLM could route tasks across specialized models in a tool ecosystem. Both sit at the boundary of two stages, turning one-shot tool use into the beginnings of the loop that Stage III makes central.

Long Context, Memory, and Context Engineering. The third development enlarged the container itself. Efficient-attention kernels such as FlashAttention [108] improved the practicality of long-sequence serving, while sparse-attention architectures such as Longformer [32] and BigBird [792] explored algorithmic routes to longer inputs; later frontier systems exposed hundred-thousand- to million-token windows [593], and state-space architectures offered near-linear alternatives to attention [186]. This made it physically possible to condition on whole documents, codebases, and histories rather than summarizing them away, which shifted the question from whether the model can reach information to whether it can use the information well once present. LongBench [28] was among the first benchmarks to quantify how poorly models handle genuinely long-context tasks, a gap that dedicated long-context suites continue to make precise [7, 220, 833]. Because no window is unbounded, a complementary line externalized state into persistent memory, with MemGPT treating context as a paging hierarchy over external storage [464] and generative agents maintaining a long-running memory stream with retrieval and reflection over remembered experience [470]. This abundance, however, created its own problem, since more context is not better context. Long inputs are read unevenly, with material in the middle systematically ignored [378]; irrelevant evidence actively distracts reasoning [541]; and utilization past a threshold degrades sharply, a failure measured as *context rot* [511]. The methodological response, named by practitioners *context engineering*, is the systematic selection, organization, compression, and refresh of the model’s input under hard token budgets [13, 425], with a toolkit spanning prompt compression [254, 351, 773], working-memory mechanisms [224, 268, 778], and rolling summarization for long runs [693, 890]. More recently, this curation is itself becoming agentic, as *agentic context engineering* lets the agent treat its own context as an evolving object that it revises and improves over successive steps [824]. Together, retrieval, tools, longer windows, memory, and context engineering shifted the relevant capacity from how much knowledge sits inside the model to how effectively the surrounding structure can selectively surface, organize, and use relevant information at each step; here too capability was partly internalized, as retrieval, tool use, and long-context handling increasingly became native, trainable model skills rather than bolted-on calls.

For all its reach, context engineering keeps the structural assumption it inherited from prompting, in that it optimizes the input to a *single* model call. The unit is wider, but it remains a one-shot, open-loop conditioning step that shapes what the model sees *before* it acts, not what happens after. Once deployments began chaining many such calls into consequential actions on tools, files, browsers, and operating systems, the dominant failures became properties of the *trajectory*, as errors compounded across steps, belief state drifted from the world, and side effects became irreversible. Closing that loop requires control over the runtime itself, which is the surface Stage III opens.

3.3 Stage III: Runtime Harnesses (2025–Present)

In Stage III, capability expansion manifests as a shift from answering questions to acting in environments, and the long-horizon agent takes its recognizable form. This also marks the transition of agents from chatbots toward genuinely productive settings, in which they carry out consequential work rather than merely converse. The transition was made concrete by executable environments and benchmarks such as WebShop [759], WebArena [885], SWE-bench [260], OSWorld [716], and Terminal-Bench [428], which score completed actions in web, code, desktop, and terminal settings rather than static answers, and thereby supply the trajectory-level feedback long-horizon agents are optimized against. The unit of engineering is no longer a prompt or a single conditioned call but a *trajectory*, comprising repeated cycles of reasoning, action, observation, verification, and recovery. We summarize the stage in three movements, namely the loop that makes the agent act, the harness that governs the loop, and the internalization that absorbs harness capabilities back into the model.

Agent Loops. The core engineering unit is a *trajectory* that unfolds repeatedly in an environment, in which the agent reasons, acts, reads feedback, and decides what to do next, following the widely adopted

thought–action–observation cycle. Early work such as ReAct, Reflexion, and Self-Refine established the basic pattern of interleaving reasoning with tool use and verbal reflection after failure [416, 544, 761], while language-agent tree search wrapped the loop in value-guided search with backtracking [875]; around the same time, open-source autonomous agents such as AutoGPT and BabyAGI popularized the self-directed loop for a broad audience [183, 444]. Since 2025, the loop has tightened along three axes rather than inventing a new paradigm. First, *action representation* became more executable and modular, as CodeAct unified step-level actions as code [656] and ReWOO separated planning from tool execution [720]. Second, *reliability* moved inward to the loop itself, as ReflAct grounds decisions in goal-state reflection [276] and test-and-repair loops such as CRITIC introduce tool-interactive verify-then-correct steps that catch and revise errors mid-trajectory [182], complementing the reflect-and-retry pattern of Reflexion and the value-guided search of LATS above. Third, *task form* widened from QA-style interaction to open-ended, multi-tool trajectories in benchmarks such as GAIA and AssistantBench [431, 774], to vision–language control in SeeAct [864], and to multi-stage research pipelines that interleave planning, retrieval, and writing [258, 613, 871]. Alongside these free-form loops, a complementary line re-emphasizes structured *agent workflows*, in which recurring procedures are captured as reusable graphs or memories and replayed across tasks, as in Agent Workflow Memory [674] and automated workflow generation [353]. The loop has thus become a runtime primitive that sustain one goal across many coupled steps while continuously changing external state, and the remaining question is how to govern it.

Harness Engineering. Once a loop stretches over a horizon, failures become properties of the *execution process* rather than of any single input, as errors compound, goals drift, success is declared too early, state distorts as context fills, and side effects escape authorization. The engineering response must therefore enter the runtime itself. *Harness engineering* names the organization of prompts, context, tools, state, permissions, verification, and recovery into a system-level control surface. The field’s first response was to make multi-step execution explicit. Communicative multi-agent efforts led by ChatDev and MetaGPT cast long-horizon software work as role-specialized message passing [219, 485], AutoGen turned that pattern into a programmable conversation runtime [692], and Magentic-One added dual ledgers that separate outer replanning from inner dispatch [157]. As trajectories grew longer and more inspectable, control migrated from ad hoc dialogue to graph-structured runtimes exemplified by LangGraph and packaged into deep-research systems such as Deep Agents and Flash-Searcher, where state can be resumed, branched, and audited [291, 293, 491]. In parallel, the interface to the outside world was standardized, as MCP unified heterogeneous tools behind a discoverable, authorizable service layer [99], while AGENTS.md and Agent Skills externalized project rules as portable harness configuration [16, 98]. Long-running deployment then forced a third concern, what an agent is allowed to do, into the foreground through hooks, middleware, pre-action gates, sandboxes, and policy layers [17, 18, 41, 463, 617]. These layers are what product systems ultimately integrate, including repository-oriented harnesses such as OpenHands and SWE-agent, whose Agent–Computer Interface showed that interface design alone can determine long-horizon success [657, 747], local personal-agent runtimes such as Claude Code and OpenClaw [10, 457, 513], and GUI-facing systems such as Operator [12, 452]. A later turn makes the harness itself an object of search and improvement, as in workflow discovery by AFlow and ADAS [225, 383, 662, 816, 897] and self-editing harness code in the Darwin Gödel Machine [814]. Reliable long-horizon agency is therefore a system property to be engineered and learned. Practitioners increasingly describe this outward shift as *loop engineering*, a move from writing one-shot prompts to designing loops that dispatch, monitor, verify, and recover agents over a trajectory.

Internalizing the Loop. Stage III is also where the paper’s two halves become visible together, as behaviors first organized by the harness are increasingly absorbed into model weights through trajectory-level SFT and RL. Post-2024 work no longer pursues a single generic agent-tuning recipe; instead, internalization *tracks harness type*, because each runtime exposes a different action grammar, feedback channel, and failure mode. Search-centric harnesses are distilled by agentic-search RL such as Search-R1 [261]; repository harnesses by software-engineering RL such as SWE-RL, increasingly trained inside neural environments like SWE-World [572, 680]; terminal harnesses by systems such as RLAnything that unify shell, tool-use, and coding rollouts [663]; GUI harnesses by UI-TARS-2 [632]; and personal runtimes by Claw-R1, which routes rich OpenClaw trajectories into RL backends through step-level middleware [457, 627]. Agent-World pushes this further by coupling environment–task discovery with continual self-evolving training, so that internalization is no longer confined to fixed benchmarks [131]. Evaluation moves in the same direction,

being less concerned with a single end-to-end score under one hand-built harness and more with whether a model natively carries the capability a harness would otherwise supply. Recent benchmarks accordingly probe cross-industry computer-use workflows [575], long-horizon tool and MCP use [322, 700], local control and multi-day collaboration [426, 764], and productive office reasoning [458]. Internalization does not make the harness disappear; it shifts the field from hand-assembling a runtime for every model toward distilling runtime experience into weights and validating transfer with harness-native benchmarks, a theme developed systematically in Section 5.

This is the co-evolution at the heart of the paper. Long-horizon capability is supplied *jointly* by an externalized harness and an internalized optimization process, with experience traces carrying capability between them. The next two chapters take up these halves in turn, treating the externalized harness in Section 4 and the internalized optimization in Section 5.

4 Harness: Externalizing Long-Horizon Capability

To analyze long-horizon capability, we start from the externalization perspective. Long-horizon capabilities need not originate in weights: before a model learns to plan, remember, or recover through training, these capacities can be *externalized*, that is, engineered into the runtime system through the harness that wraps the model at inference time and mediates every interaction it has with the world. It contains the machinery of loops, memory, tools, coordination, and oversight that extends a model into a long-horizon agent without changing what the model knows. Where Section 5 asks what training can absorb, this chapter asks what harness can supply. As shown in Table 2, we organize the harness into six components that recur across essentially every contemporary agent, ordered from the control loop outward to the systems that govern it:

The Six Components of an Agent Harness

1. **Loops and Workflows** (Section 4.1): specifies how execution moves across states, including when the model is called, how actions are taken, and how control returns after observations.
2. **Context and Memory** (Section 4.2): manages the state available to the model, from the active context of the current run to persistent memory across windows, tasks, and sessions.
3. **Tools, MCP, and Skills** (Section 4.3): defines how the agent accesses external capabilities, represents actions, and reuses acquired procedures.
4. **Orchestration** (Section 4.4): organizes work beyond a single agent loop, including task decomposition, role assignment, routing, and inter-agent coordination.
5. **Hooks and Middleware** (Section 4.5): provides predefined interception points in the runtime where execution can be inspected, modified, paused, or handed off.
6. **Verification** (Section 4.6): checks whether states, actions, outputs, or side effects satisfy task, safety, and quality constraints before execution continues or results are accepted.

4.1 Loops and Workflows

The innermost layer of a long-horizon agent harness is the workflow that advances execution from one state to the next. At this layer, the harness specifies the control pattern by which model calls, tool calls, state updates, checks, and feedback are sequenced. Recent workflow papers describe agent workflows as execution structures in which agents perceive state, reason, decide, act, receive feedback, and adapt under a workflow engine [777]. An example of a long-horizon run can be found in Figure 7.

For long-horizon agents, the central design question is how much of the trajectory is governed by this external control pattern rather than by the model’s next-token policy alone. A workflow may simply repeat an observe-act loop, maintain an explicit plan across steps, or branch over multiple candidate trajectories before committing. We therefore classify harness workflows into three structures: *linear workflows*, which advance through a single trajectory; *plan-execute workflows*, which externalize task decomposition and progress tracking; and *branching workflows*, which search over alternatives using comparison, pruning, or backtracking.

Table 2 The agent harness by component. *Harness responsibility* states what the component must supply at runtime; *representative mechanisms* lists the component’s sub-mechanisms, and *representative systems* pairs each sub-mechanism with two concrete systems on the same line.

Component (§)	Harness responsibility	Representative mechanisms	Representative systems
Loop & Workflow (§4.1)	Schedules model calls, action execution, and observation feedback across steps	• Linear workflows • Plan–execute • Branching	ReAct [761], Self-Refine [416] ReWOO [720], Arbor [262] Tree-of-Thoughts [760], LATS [875]
Context & Memory (§4.2)	Bounds in-run state and persists what must survive across windows and sessions	• Working context • Factual memory	ReSum [693], MemAgent [778] Mem0 [88], MemoryBank [874] ExpeL [853], ReasoningBank [462]
Tools, MCP & Skills (§4.3)	Interfaces model decisions with external capabilities and reusable procedures	• Tool interfaces & MCP • Active discovery • Skill libraries	Toolformer [523], MCP [100] AnyTool [140], ToolGen [647] Voyager [630], SkillNet [360]
Orchestration (§4.4)	Decomposes goals, assigns roles, and coordinates work beyond a single loop	• Decomposition • Topologies • Routing & protocols	MetaGPT [219], CAMEL [314] Magenti-One [157], LangGraph [291] GPTSwarm [897], A2A [97]
Hooks & Middleware (§4.5)	Enforces allow, block, or hand-off at predefined points on the action path	• Rule-based hooks • User-defined policies • Adaptive guards	AEGIS [783], IsolateGPT [698] AgentSpec [633], ShieldAgent [80] AdaptiveGuard [754], AGrail [408]
Verification (§4.6)	Scores states and outputs for correctness, safety, and task fit before continuing or accepting results	• Assessment targets • Verification levels • Verifier strategies	SelfCheckGPT [418], VeriGuard [432] PRMs [363], LLM-as-a-Judge [867] Math-Shepherd [643], RLV [521]

4.1.1 Linear Workflows

Linear workflows are the simplest way to keep an agent moving: the harness repeatedly renders the current state to the model, executes the selected action, records the resulting observation, and feeds the updated state into the next call. The workflow follows one evolving trace rather than maintaining a separate plan or search tree. This makes it suitable for interactive tasks where the agent must react to fresh evidence, such as web navigation, debugging, tool use, and exploratory search. ReAct [761] is the canonical instance of this pattern, where reasoning and acting are interleaved and each observation conditions the next model invocation.

The strength of a linear workflow is responsiveness, but its weakness is that all progress depends on a single trace. Early mistakes can contaminate later steps, repeated reasoning can exhaust the context window, and the model may loop, drift, or stop prematurely. Many variants therefore wrap feedback, critique, retrieval, memory, or self-correction around the basic chain [22, 416, 544, 567]. In this sense, a bare linear workflow is only the inner loop, while the emerging practice of *loop engineering* designs the recurring outer system that automatically prompts, monitors, verifies, persists, and resumes the agent. This outer system adds external stop conditions, persistent state, verification gates, cost limits, and recovery policies, turning the workflow into an unattended long-horizon run in which a human need not decide every next prompt.

4.1.2 Plan-Execute Workflows

Plan-execute workflows address the short-sightedness of purely reactive loops by making task structure explicit. The harness first creates or maintains a plan, then uses it to guide later actions through subgoals, dependencies, progress markers, budgets, and failure conditions. Early forms ask the model to plan before solving, while more structured variants plan tool calls, decompose tasks recursively, or revise the plan during execution [482, 639, 720]. Across these systems, the plan becomes an execution object that the harness can inspect, update, resume, invalidate, or assign to different modules.

This structure is especially useful for long-horizon loops because an unattended agent cannot safely run on a vague instruction such as “improve the project”. It needs a goal that can be decomposed, monitored, and verified. A maintained plan provides this control surface: it translates an underspecified objective into concrete checkpoints and failure conditions, while allowing the model to make local decisions under persistent task commitments. Recent long-horizon systems therefore couple planning with execution monitoring, verification, replanning, or learning-based policy improvement [763, 832]. Arbor [262] extends this paradigm to autonomous research by maintaining a persistent hypothesis tree across coordinator–executor cycles, where each branch

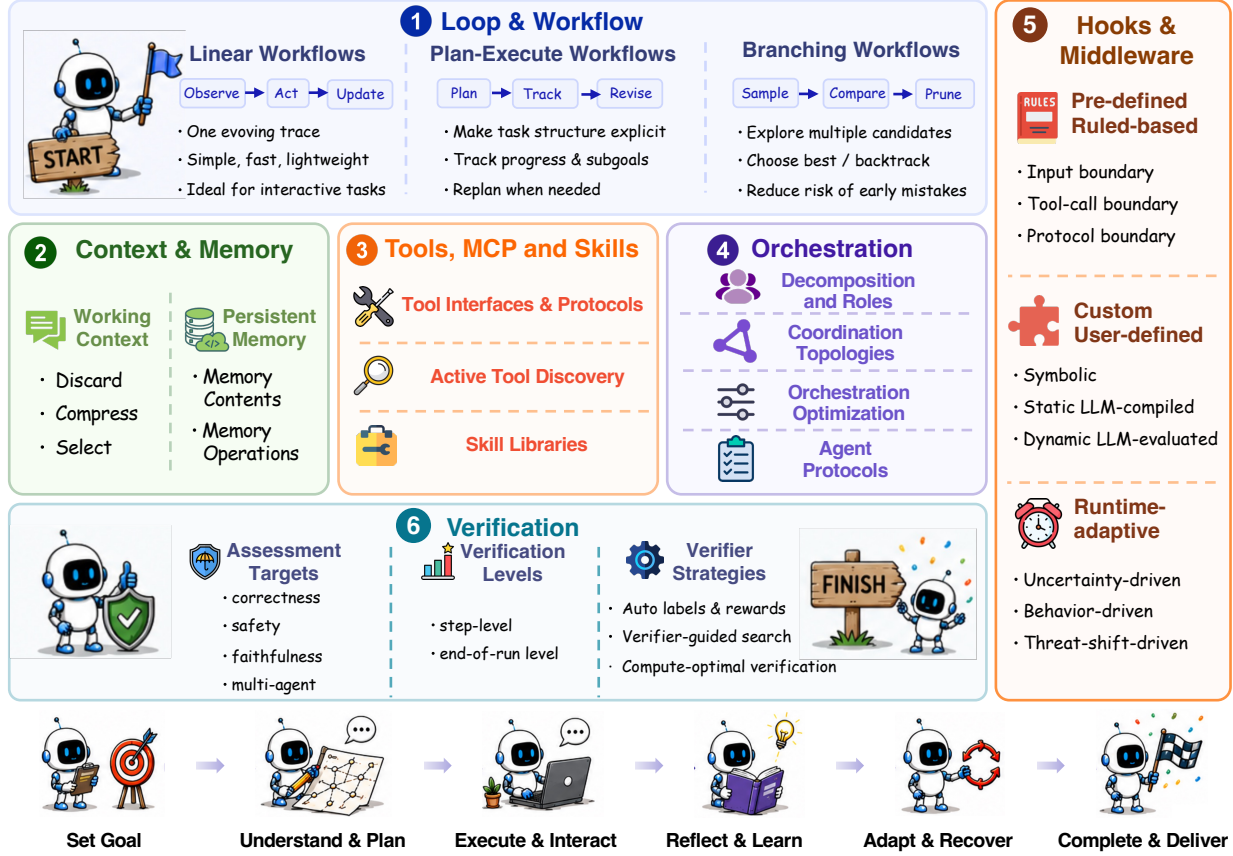


Figure 7 Overview of the agent harness. Six core harness components support the end-to-end agent lifecycle, from goal formulation and planning to execution, adaptation, verification, and delivery, enabling coherent long-horizon execution across interdependent steps.

records hypotheses, experimental artifacts, evidence, and distilled insights that propagate across iterations, turning isolated attempts into a cumulative strategy-driven search.

4.1.3 Branching Workflows

Branching workflows are useful when one wrong commitment can derail the whole task. Instead of immediately following the first plausible action, plan, or answer, the harness generates several alternatives and evaluates them before deciding which one to pursue. This pattern appears in reasoning trees, tree-search-based acting, graph-based thought, code-generation search, and memory-augmented search variants [33, 91, 760, 875]. The defining feature is evaluation before commitment: the harness controls the branching factor, scoring function, pruning rule, backtracking policy, and final selection procedure [319].

For long-horizon agents, branching reduces the risk of spending a large trajectory budget on a weak early decision [280]. It is also closely related to loop engineering, where unattended loops need reliable verification rather than repeated self-approval. A branching workflow can separate generation and checking: one branch proposes a patch or plan, another critiques it, a verifier tests constraints, and the harness decides whether to continue, revise, or stop. This maker-checker separation helps reduce self-evaluation bias and Goodhart-style failures, where the agent satisfies the visible completion condition while violating the user’s underlying intent.

Branching is most useful when partial states can be evaluated cheaply and meaningfully. It is less attractive when feedback is weak, delayed, or expensive, because search can multiply token and tool costs, create incompatible partial worlds, and make provenance harder to audit [515]. Thus, branching workflows are best

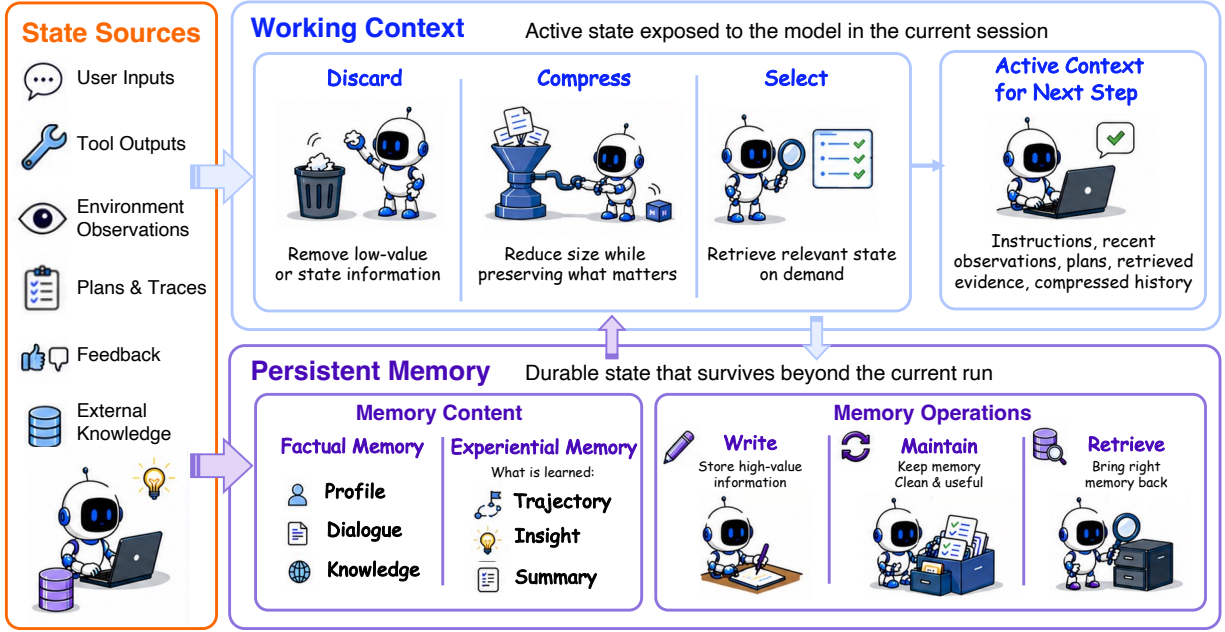


Figure 8 Context and memory in an agent harness. The harness governs the state interface between the workflow and the model: it selects, compresses, or discards active context under a token budget, and reads from and writes to persistent memory that outlives a single window, task, or session.

viewed as selective search mechanisms invoked when the cost of a bad commitment is higher than the cost of evaluating alternatives [343]. Across linear, plan-execute, and branching workflows, the common requirement is that the harness must maintain the state on which these workflows operate. We therefore turn next to the state machinery that supports long-horizon execution.

4.2 Context and Memory

If workflows define how an agent proceeds, context and memory define what state it proceeds with [228]. For long-horizon agents, the harness cannot simply pass along an ever-growing transcript. It must decide what the next model call can see, what should be compressed or discarded when the context window fills, and what information should persist beyond the current window, task, or session [876]. In this sense, context and memory are the state interface between the workflow and the model: the harness reads from prior observations, tool outputs, plans, traces and stored knowledge, then writes the selected state back into the next step.

We distinguish context and memory by their time horizon and role at runtime. *Context* is the active state exposed to the model in the current run, including instructions, recent observations, intermediate reasoning, retrieved evidence, and compressed history. *Memory* is the durable state that survives beyond the immediate window and can be retrieved or updated over time. We organize this component accordingly. Section 4.2.1 covers working state, namely the in-window or near-window state maintained within a run; Section 4.2.2 covers persistent state, namely the durable memory that survives across windows, tasks, and sessions. In Table 3, we list representative works for each sub-section. The overview of this section is shown in Figure 8.

4.2.1 Working Context

Every step of an agent run produces new state: model outputs, tool returns, environment observations, partial plans, and intermediate artifacts. Runtime context maintenance is the harness process that decides which of these states should condition the next model call. Some work refers to this in-window or near-window state as the agent’s *working memory*, emphasizing its role as a limited workspace for keeping task-relevant information available within a single episode [228]. For long-horizon agents, the central challenge is not only that the context window is finite, but that useful, stale, and distracting information accumulate together over time.

Too much context can degrade performance through *context rot* and *lost-in-the-middle* effects, while too little context removes constraints, evidence, or partial progress that the agent still needs [378, 511, 541]. The harness must therefore turn an expanding trajectory into a bounded, high-signal workspace. In practice, long-running agents combine three operations: *discard*, *compress*, and *select* [13, 14, 293].

Context Discard. Discarding removes information from the active window [113]. Common policies include resetting context at task boundaries, dropping bulky tool responses after they have been parsed or summarized, and keeping only the most recent turns [597]. For short interactions, discard mainly saves tokens; for long-horizon runs, it is also a reliability mechanism. Stale observations, obsolete plans, verbose logs, and irrelevant tool outputs can continue to bias later decisions if they remain visible.

Discard can also happen before information enters the trajectory. Tool or schema pruning, for example, prevents rarely useful capabilities or verbose specifications from occupying the prompt in the first place [27, 622]. This matters because noise compounds over long runs: a small amount of irrelevant state repeated across many steps can become a first-order source of drift. Discard is therefore the most direct harness-level way to enforce forgetting, rather than relying on the model to ignore low-value context.

Context Compression. Compression preserves task-relevant information while reducing its token and attention footprint. A useful operational distinction is between compressing the current payload and compressing the accumulated trajectory. The first case handles large immediate inputs, such as retrieved passages, tool outputs, web pages, logs, or DOM trees [57, 254, 773]. In agent harnesses, similar operations selectively rewrite verbose observations into compact, task-oriented state descriptions.

The second case is trajectory consolidation: replacing a growing interaction history with a maintained state. This state may be a summary, a list of open constraints, a table of facts, a plan with completion markers, or a graph of entities and dependencies. For long-horizon agents, the goal is not just to shorten the transcript, but to preserve what still matters for future action: completed decisions, remaining subgoals, unresolved failures, and produced artifacts. Recent methods explore learned, task-aware, budget-aware, and hierarchical forms of such compression [224, 268, 573, 693, 696, 768, 778, 824, 890]. This also marks a shift from hand-written context rules toward learned policies for when and how to fold state.

Context Selection. Selection keeps richer state outside the active window and retrieves only what is relevant to the current decision. The indexed state may include documents, events, artifacts, plans, tool outputs, or previous observations. Unlike ordinary RAG, selection in an agent harness often retrieves from the agent’s own trajectory, not only from an external knowledge base [202, 669]. Typically, CL-bench is designed to target and evaluate a model’s long-context comprehension and selection capabilities [133]. The goal is to keep the current context small while preserving access to the evidence trail.

From the harness perspective, selection involves four choices: when to retrieve, how to form the query, which store to search, and how to filter or aggregate the results before injection. Retrieving too often increases cost and noise; retrieving too rarely risks losing dependencies that were established many steps earlier. Long-running agents therefore usually combine selection with discard and compression: bulky outputs are removed, completed subtasks are folded, and relevant past state is retrieved on demand [13, 230, 231, 824, 846]. These operations keep the active context bounded without forcing the agent to forget the full task history.

4.2.2 Persistent Memory

Working context keeps a single run usable; persistent memory lets the harness carry state beyond the current window, trajectory, or session [228, 583, 694, 848]. This is crucial for long-horizon agents because useful information often outlives the step in which it was produced. A failed attempt may become a lesson for later recovery, a completed subtask may define constraints for downstream actions, and a discovered procedure may be reused in future tasks. Without persistent memory, the agent must repeatedly rediscover the same facts, strategies, and skills as the context window turns over. We use a simple harness-oriented view. *Memory Contents* asks what persistent state is stored; *Memory Operations* asks how that state is written, maintained, retrieved, and evaluated.

Table 3 Representative context and memory systems in the agent harness. Rows follow the split of this subsection: *working context* (Section 4.2.1) is bounded within a single run, while *persistent memory* (Section 4.2.2) survives across windows, tasks, and sessions. **Type** names what the method does with state. **State form** is the representation it maintains. **Train-free** (✓) marks methods that plug into a frozen model as pure harness machinery, versus (✗) those whose policy must be trained or RL-tuned. **Trigger** is when the operation fires. **Key mechanism** briefly describes the core operation.

Method	Type	State form	Train-free	Trigger	Key mechanism
Working Context					
Effective Harnesses [14]	Discard	Turns	✓	Boundary	Reset context, drop parsed tool output
Deep Agents [293]	Discard	Files	✓	Boundary	Offload state to files
Tool pruning [622]	Discard	Schemas	✓	Start-of-run	Prune tools/schemas before prompt
LLMLingua [254]	Compress	Summary	✗	Per-payload	Token-level prompt compression
CompAct [773]	Compress	Summary	✗	Per-payload	Actively compress retrieved docs
ReSum [693]	Compress	Summary	✓	Boundary	Learned trajectory summarization
MEM1 [890]	Compress	Summary	✗	Per-step	RL fuses memory and reasoning
MemAgent [778]	Compress	Summary	✗	Per-step	RL over fixed-size memory
ACON [268]	Compress	Summary	✓	Boundary	Optimizes context-compression policy
ContextBudget [696]	Compress	Summary	✗	Per-step	Budget-aware context allocation
HiAgent [224]	Compress	Plan	✓	Boundary	Subgoal-scoped working memory
Context-Folding [573]	Compress	Summary	✗	Per-step	Folds history under a budget
AgentFold [768]	Compress	Summary	✗	Per-step	Proactive history folding
ACE [824]	Select	NL-note	✓	On-demand	Retrieves, evolves context entries
LEGOMem [202]	Select	Summary	✓	On-demand	Modular per-subtask procedural memory
Memex(RL) [669]	Select	Vector	✗	On-demand	RL over indexed experience memory
Memory-as-Action [846]	Mixed	Summary	✗	Per-step	Memory read/write as actions
Persistent Memory					
CLAUDE.md [10]	Factual	NL-note	✓	Start-of-run	Project-rule file loaded per run
Cursor Rules [104]	Factual	NL-note	✓	Start-of-run	Persistent project rules per run
AGENTS.md [98]	Factual	NL-note	✓	Start-of-run	Portable project-knowledge spec
MemoryBank [874]	Factual	Vector	✓	On-demand	Store with forgetting-curve updates
Mem0 [88]	Factual	Graph	✓	On-demand	Linked long-term fact store
MemoryOS [267]	Factual	Mixed	✓	On-demand	OS-style tiered memory
HippoRAG [199]	Factual	Graph	✓	On-demand	Graph memory for associative recall
A-MEM [731]	Factual	Graph	✓	On-demand	Self-organizing memory notes
Reflexion [544]	Experiential	NL-note	✓	After-failure	Verbal lessons from failures
ExpeL [853]	Experiential	NL-note	✓	On-demand	Reuses insights from trajectories
Synapse [868]	Experiential	Summary	✓	On-demand	Trajectory-as-exemplar prompting
SAGE [359]	Experiential	NL-note	✓	On-demand	Reflective memory-augmented evolution
Buffer of Thoughts [750]	Experiential	NL-note	✓	On-demand	Reusable thought templates
ReasoningBank [462]	Experiential	NL-note	✓	On-demand	Distills reusable reasoning strategies
AWM [674]	Experiential	Code-skill	✓	Boundary	Induces workflows from traces
Agent KB [587]	Experiential	NL-note	✓	On-demand	Cross-domain experience knowledge base
Voyager [630]	Experiential	Code-skill	✓	On-demand	Reusable executable skill library
MemGPT [464]	Mixed	Mixed	✓	On-demand	OS-style context/memory paging
Generative Agents [470]	Mixed	Summary	✓	On-demand	Memory stream with reflection

Memory Contents. Persistent memory mainly stores two kinds of state. *Factual memory* stores declarative information that the agent should not have to rediscover: user preferences, project rules, commitments, environment state, document or codebase facts, and the capabilities of external systems [228]. For a long-horizon harness, these facts support continuity: user-level facts keep behavior consistent across sessions, while environment-level facts prevent the agent from repeatedly re-inferring stable constraints. Practical examples include project-rule files such as `CLAUDE.md`, `Cursor Rules`, and `AGENTS.md`, which provide stable project knowledge outside the immediate prompt [10, 98, 104]. User memory systems and graph-structured stores further extend this idea by maintaining profiles, preferences, and linked factual records that can be consolidated and audited over time [88, 199, 229, 267, 507, 731, 874].

Experiential memory stores what the agent has learned from prior trajectories. It may contain failure lessons, successful cases, state-action exemplars, reusable reasoning patterns, plans, workflows, or executable procedures. The common purpose is to reduce long-horizon cost by replacing repeated rediscovery with retrieval of prior structure. Some systems store verbal reflections or lessons from past failures; others maintain experience pools, reusable reasoning templates, workflow memories, or cross-domain solution paths [44, 262, 359, 462, 544, 587, 674, 750, 807, 853, 868]. At the most operational end, experiential memory can be distilled into skills, scripts, API wrappers, or tool-like procedures, as in skill-library agents such as Voyager [446, 630]. Once stored in this form, memory begins to overlap with the tool layer: the harness can retrieve, version, validate, reuse, or retire a learned procedure without changing model weights.

Memory Operations. A persistent store is useful only if the harness manages how memory is written, maintained, and read back into context. *Memory writing* decides what should be saved from a long run. Tool outputs, partial plans, reasoning traces, user feedback, observations, and intermediate artifacts are often too noisy to store as-is. The harness therefore extracts higher-value units such as facts, summaries, cases, lessons, workflows, or skills, using summarization, distillation, or structured construction [221, 407, 464, 470, 674]. The key question is what level of experience should survive: an event, a workflow, or an executable procedure.

Memory maintenance keeps the store usable as new information arrives. Over long horizons, memories can become stale, repeated, or inconsistent. The harness must merge related entries, update facts when users or environments change, and remove outdated or low-value material before retrieval becomes noisy. Existing systems use memory streams, graphs, linked notes, or other structured stores to support this process [199, 470, 731]. The model may suggest what to remember, but the harness decides whether a memory is inserted, merged, linked, archived, or forgotten.

Memory retrieval decides how stored state re-enters the active context. Some memories are loaded at the start of a run, such as project rules or user preferences [664]; others are retrieved at subtask boundaries, after failures, before tool use, or when the model asks for them. Retrieval requires choices about the query, the store to search, the ranking or filtering method, and how much content to inject [488, 568, 610, 694]. Too little retrieval makes the agent repeat mistakes or miss long-range dependencies; too much retrieval turns memory back into context overload. Recent work therefore treats memory read/write as a policy problem, learning when to remember, update, or retrieve during inference [778, 846].

Persistent memory is also hard to evaluate. End-task success often mixes memory quality with model strength, tool reliability, and benchmark variance. New benchmarks test long-term recall, cross-session consistency, and tasks that depend on earlier sessions, but recent analyses still find saturation, judge sensitivity, and limited coverage of update cost and memory maintenance [235, 253, 686]. For long-horizon harnesses, the key question is not whether an agent can recall an isolated fact, but whether it can save, update, retrieve, and inject the right memory when it changes the next action. We collect evaluation resources in Section 6.6.

4.3 Tools, MCP, and Skills

Where context and memory settle what the agent *knows*, this component settles what it can *do*. Without tools that return observations, even a capable model can only emit text; tools are what let it query a search engine, run code, or read and write a database. We organize this capability interface in three parts: the tools exposed by the runtime and the protocol layer, most prominently the Model Context Protocol (MCP), that standardizes them (Section 4.3.1); the discovery mechanisms that keep a large tool surface usable (Section 4.3.2); and the skills an agent accumulates on top of them (Section 4.3.3). This interface matters more as the horizon grows,

because extended execution becomes a long chain of interdependent calls whose state must be carried, verified, and recovered across steps.

4.3.1 Tool Interfaces and Protocols

Function Calling. Tool interfaces let a language model act beyond its parametric knowledge. Foundational work established when, what, and how to call: Toolformer self-supervises where API calls should be inserted, Gorilla maps requests onto large API surfaces while suppressing hallucinated calls, and ToolLLM scales instruction-tuned tool use to thousands of real-world REST APIs [471, 492, 523]. Papers consolidate this line into a workflow of task planning, tool selection, tool calling, and response generation within a controller–tools–environment view [497, 724]. Single-call competence is by now well covered by mature benchmarks (BFCL, StableToolBench) and data-synthesis pipelines (APIGen, ToolACE) [198, 385, 392, 472].

Tool Protocols and MCP. The dominant engineering response to a growing, heterogeneous tool surface is protocolization. The Model Context Protocol (MCP) defines a host–client–server lifecycle with capability negotiation, authorization, and tool annotations, turning ad hoc integrations into a uniform, discoverable layer; its ecosystem, architecture, and security are covered in dedicated and broader protocol papers [100, 758]. Because it lets agents compose independently maintained servers and exposes the tool space uniformly, MCP also sets up the scaling techniques of Section 4.3.2. MCP-grounded benchmarks such as MCP-Universe reveal what single-call leaderboards miss: once tasks span many servers and tools in stateful sandboxes, even the strongest models succeed on only a minority of them, with input tokens rising rapidly over steps and failures becoming increasingly cognitive rather than syntactic [31, 350, 411, 700]. Beyond MCP, a broader wave of tool-use benchmarks probes schema discipline, state mutation, conversational tool legality, and messy real-world user behavior, from constrained tool-legality suites such as τ -bench to long-horizon, cross-application, and in-the-wild benchmarks such as Toolathlon [213, 322, 399, 539, 548, 762, 779].

Multi-turn Tool Use. Function calling and MCP standardize the *interface* of an individual call, but tool use at scale is inherently multi-turn: an agent chains many dependent calls, each conditioned on the previous observation. As these sequences grow to tens or hundreds of invocations, the bottleneck migrates from isolated-call correctness to sustaining the whole chain [724], and even strong agents complete only a fraction of realistic tasks, degrading through context saturation and memory decay as the turn count rises [322, 325, 406]. The limiting constraint is increasingly the state and context accumulated across turns rather than the mechanics of any single call. This is a property of the execution process rather than the call interface; the control-flow responses to it, namely decomposing, scheduling, and coordinating many turns, are orchestration, which we treat separately, so this subsection stays on the tool surface itself.

4.3.2 Active Tool Discovery

A fixed, prompt-resident tool list does not scale. Once catalogs reach thousands of tools, their descriptions crowd out the task, token cost tracks registry size rather than step demand, and selection accuracy degrades; the tool-retrieval saturation diagnosed above motivates active discovery [350, 411]. The remedy is to stop placing the entire surface in context and instead let the agent actively discover the few tools each step needs.

Tool Retrieval. The first move is retrieval-based discovery: index tool metadata and inject only the top-ranked schemas at each step. RAG-MCP, AnyTool, and MCP-Zero develop this idea with plain, hierarchical, and agent-initiated retrievers respectively, sharply reducing token use on large toolsets [140, 153, 162]. A complementary, supply-side approach generates MCP servers directly from OpenAPI specifications, and Anthropic’s Tool Search Tool demonstrates the same retrieval principle in production, scaling to thousands of tools with large token savings and higher tool-selection accuracy [9, 423]. Pushing this further, DeepAgent makes tool search an action inside a single reasoning process, dynamically discovering tools from registries of tens of thousands as the task unfolds, while ToolGen removes the separate retriever altogether by representing each tool as a vocabulary token the model emits during generation [338, 647].

Programmatic Tool Use. A second move treats the tool surface itself as something the agent can read and program against. CodeAct treats executable code as the action space, letting a single step compose

calls, branch, and loop [656]; in MCP settings, code-execution patterns present servers as importable APIs and coordinate many calls inside one code block, cutting token consumption by orders of magnitude [9, 15], and Code-Execution MCP formalizes the shift from tool-by-tool orchestration to a single model-generated program, keeping context consumption nearly constant in the call count and tool-ecosystem size at the cost of a larger executable attack surface [154].

As coding agents strengthen, tool *selection* itself becomes a workspace activity: rather than picking from a static registry, the agent inspects tool definitions, documentation, or source on demand, using progressive disclosure to keep a large ecosystem cheap in context [730]. The same insight drives agentic search. Direct corpus interaction has a strong agent search a raw corpus with terminal tools (grep, find, read) instead of a top- k retriever, beating sparse, dense, and reranking baselines [355], and agentic keyword search with only grep-style tools matches vector-database RAG [566]. Applied to code, a terminal-only RL-trained agent localizes code as well as far larger models [578], turning discovery from a static input choice into an agent-driven search. Together, code execution and workspace-mediated discovery bound a tool surface that would otherwise grow with every call.

4.3.3 Skill Libraries

Once tools and workspace state are in place, the next layer of externalized capability is the *skill*: a reusable, parameterizable procedure that packages a validated combination of tools, control flow, and domain knowledge. Recent papers cast this as the step from episodic tool use to persistent, colleague-like agents under a “Workspace + Skill” paradigm, where a persistent workspace supplies state and a curated skill library supplies reusable operational knowledge [841]. We unpack it along three axes: what a skill is, how skills are acquired and reused, and how the library evolves.

Procedural Skills. On top of tool interfaces, the *skill* abstraction packages validated combinations of tools, control flow, and domain knowledge into named, reusable procedures. Rather than re-deriving a tool sequence each time, the agent recalls a procedure that already works, shifting the interface from tools to skills [888]. Papers frame skills as procedural artifacts with a representation, acquisition, retrieval, and evolution lifecycle, and as a layer orthogonal to MCP: where MCP supplies connectivity, skills supply the procedural knowledge for interpreting outputs and recovering from failure. The current landscape converges on three representation families: natural-language or Markdown procedures that condition a frozen agent, realized as a progressively disclosed file tree by Anthropic’s Agent Skills [16]; executable code the agent can call, inspect, and edit, as in Voyager [630]; and structured relational libraries, where SkillNet weaves over 200,000 skills into a typed skill graph (depend_on, compose_with, ...) [360]. The shared move is from flat, similarity-retrieved entries toward curated, relationally organized procedures owned by the harness as first-class assets.

Skill Acquisition. Work on skill acquisition and reuse shows the payoff. Voyager, the canonical instance, discovers executable Minecraft skills and reloads them on later tasks, and later systems extend the idea to inference-time tool synthesis, cumulative scientific-reasoning skills, and skills for unfamiliar software [4, 237, 400, 577, 630]. Related systems evaluate higher-order tool-composition skills and convert episodic narratives into executable skills with explicit activation and termination conditions [65, 430]. Across these systems the recurring benefit is lower token cost, higher reliability, and shorter effective horizons through reuse.

Skill Library Evolution. What distinguishes a skill from a tool is that the library is a first-class object the harness owns and curates: skills can be versioned, retrieved, refined, retired, and reused, so where tools supply flexibility, skills supply efficiency and reliability [340]. Because the library lives outside the weights, it can evolve without gradient updates: Memento treats skill acquisition as memory-based reinforcement learning, Alita generates and reuses MCP servers on demand, and observability-driven harness evolution finds most of its gains in rewriting the tool, middleware, and memory layers rather than the system prompt [456, 495, 881]. A first line optimizes the skill text directly: SkillOpt trains a frozen agent by editing its natural-language skill document, accepting a change only when it improves a held-out validation score [757]. A second structures the library as a graph rather than a flat list: the prerequisite-skill graph originates in data-efficient training (Skill-it) [63], and is imported into agent libraries by SkillGraph, which co-evolves a typed skill graph with the policy via reinforcement learning [341], by GraSP, which compiles retrieved skills into an executable

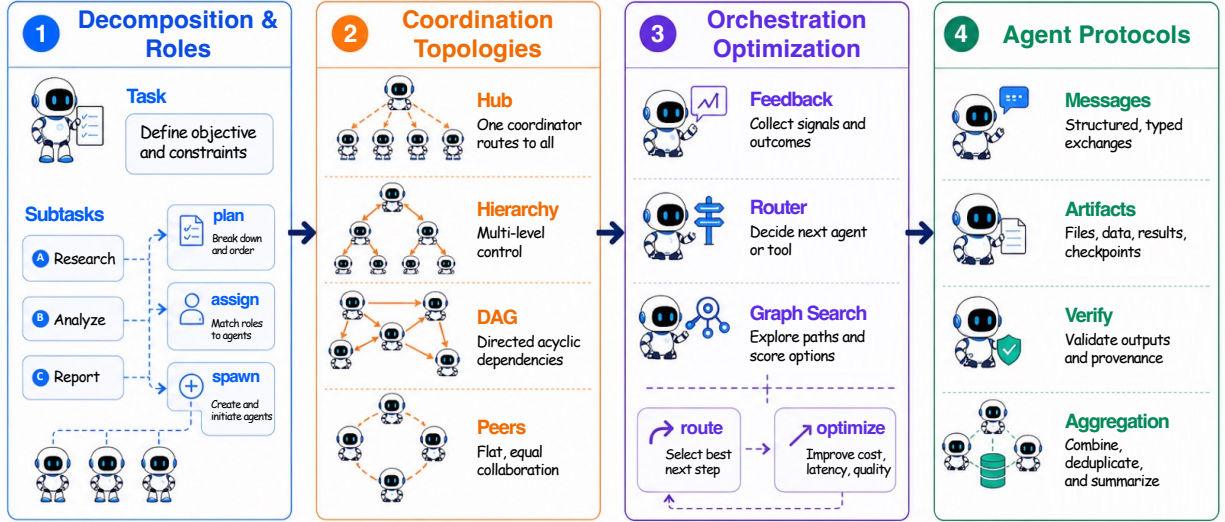


Figure 9 Overview of agent orchestration. Tasks are decomposed into roles, coordinated through different topologies, optimized via feedback-driven routing, and integrated through communication and aggregation protocols.

DAG with locality-bounded repair [707]. A growing line couples skill libraries directly with reinforcement learning: Skill0, for instance, trains with an in-context curriculum that progressively withdraws skill files until the policy runs skill-free, beginning to fold the library into the weights themselves [405]. This library-level self-improvement is the externalized form of evolution; its internalized counterpart is taken up in Section 5.

4.4 Orchestration

A single agent loop can attend to only one thing at a time; many long-horizon tasks are better served by composing several loops, whether as subtasks of one agent or as a team of specialized agents. Multi-agent collaboration can be described along a few dimensions: the participating agents, the interaction type (cooperation, competition, or a mix), the communication structure, the coordination strategy, and whether the arrangement is static or dynamic [197, 611]. We adopt a pragmatic version of this taxonomy and ask four questions: how work is decomposed and assigned, what topology the resulting computation takes, whether that topology is fixed or learned, and how the agents communicate. Orchestration is the externalized form of long-horizon control structure [468]: once a task outgrows what a single loop can sustain, how the work is split and coordinated often affects end-to-end success more than swapping the underlying model does. Composition is not free, however. A systematic study of multi-agent failures attributes most breakdowns to under-specified roles, inter-agent misalignment, and missing verification rather than to weak base models, a reminder that orchestration must be designed rather than merely assembled [48]. The overall framework of agent orchestration is shown in Figure 9.

4.4.1 Decomposition and Roles

The first decision is how a long task is split into units and assigned to roles. The earliest harnesses fix this split statically, hard-coding the roles and their interaction pattern, whether as software-team standard operating procedures, planner-executor role-play, staged role-conditioned dialogues, or a general programmable substrate in which roles and their conversations are first-class, as in frameworks such as software-team SOPs [219], role-play collaboration [314], staged dialogues [485], and a programmable substrate [692]. A predefined subtask graph is simple but brittle, since an early failure propagates downstream and a hand-coded role set scales poorly; this motivates dynamic decomposition that revises the split from feedback and can spawn a subtask-tailored subagent on the fly, as in TDAG [660]. Either way the harness must keep the split well-formed: agent-oriented planning, for example, holds the decomposing meta-agent to solvability, completeness, and non-redundancy, re-planning unresolvable or duplicated subtasks before any agent runs [305]. How fine to

decompose is then a granularity trade-off: too coarse and a subtask overwhelms its agent, too fine and coordination overhead dominates, and that overhead grows with the horizon. Much of the benefit usually attributed to role personas in fact comes from the structural decomposition they induce, namely parallelism, context isolation, and tool specialization, rather than from the personas themselves [48].

4.4.2 Coordination Topologies

Once units and roles are set, the way agents are wired together fixes the shape of the computation. These topologies fall into centralized forms built around a hub or orchestrator, decentralized or distributed forms such as peer debate and dynamic graphs, and hierarchical forms [611]. Centralized designs, exemplified by Magentic-One, place a hub or orchestrator that drives a team, in some variants stacking layers that successively refine one another or dynamically recruiting collaborators [69, 157, 638]. Decentralized designs instead let peers select and prune one another at inference time or scale collaboration over large network structures, as in DyLAN and MacNet [390, 486]. The dominant programmable abstraction across both is the directed acyclic graph: LangGraph defines workflows as DAGs whose nodes are agents or functions and whose edges carry typed state, while Agint and Flash-Searcher compile instructions into typed DAGs and recast web research as a parallel DAG to cut step counts [90, 491]. On long-horizon coding tasks this design space can matter more than the choice of model: on SWE-bench Verified, a fixed frontier model varies far more across scaffolds than different models do under a single fixed scaffold [468].

The wirings above coordinate agents toward a shared objective, but a complementary family makes the interaction itself *competitive*, pitting two roles in a GAN-like loop where one proposes ever-harder problems and the other strives to solve them, so the pair co-evolves an automatic curriculum with no external data. R-Zero instantiates a Challenger rewarded for posing tasks at the edge of a Solver’s competence and a Solver rewarded for cracking them, while Absolute Zero folds both roles into a single model that proposes and solves verifiable tasks in one self-play loop [233, 854]; earlier, SPAG showed that setting a model against a copy of itself in a two-player adversarial language game steadily sharpens reasoning [84]. Because this adversarial coordination improves the participants through training rather than routing a fixed roster of models, its payoff is ultimately internalized (Section 5); yet the arrangement itself remains a distinctly competitive orchestration pattern, the counterpart to the cooperative composition that dominates the rest of this section.

4.4.3 Orchestration Optimization

A topology can be hand-designed or searched. AFlow treats workflow design as program-space search with Monte-Carlo tree search, and GPTSwarm represents a multi-agent system as an optimizable graph and learns both node prompts and edges [816, 897]. A related line learns who executes each unit. At the model level, routers learn to select among models under performance, cost, latency, and congestion constraints [447, 810, 880]; at the agent level, MasRouter jointly chooses collaboration mode, roles, and the model per role, while other systems spawn subagents on the fly or pair central planning with a tool–environment–agent protocol [519, 791, 828]. Taken to its limit, the coordination structure itself is allowed to emerge: a central orchestrator can be trained with reinforcement learning to induce compact coordination at lower cost, and related systems refine functionality and coordination from scratch or couple shared memory with asynchronous execution [107, 496, 836]; at scale, mixed self-organizing coordination can even outperform hand-designed hierarchies [127]. Like skill-library growth (Section 4.3.3), these black-box optimizations of the orchestration graph are an externalized form of evolution.

4.4.4 Agent Protocols

Finally, composed agents must exchange messages reliably. The A2A protocol foregrounds Agent Cards, Tasks, and Artifacts over JSON-RPC with streaming, IBM’s Agent Communication Protocol offers a lighter REST-native alternative, and MCP, ACP, A2A, and ANP have been compared across architecture, discovery, communication, and security [97, 144, 512, 758]. Framework-level messaging conventions coexist with richer schemes that add decentralized identity and verifiable credentials, switch between structured protocols and natural language according to communication frequency, or perform decentralized discovery over encrypted peer-to-peer channels [219, 314, 422, 501, 504, 692]. A complementary harness duty is aggregation: when parallel trials or subagents run, their outputs must be reduced to a single result, through best-of- N or

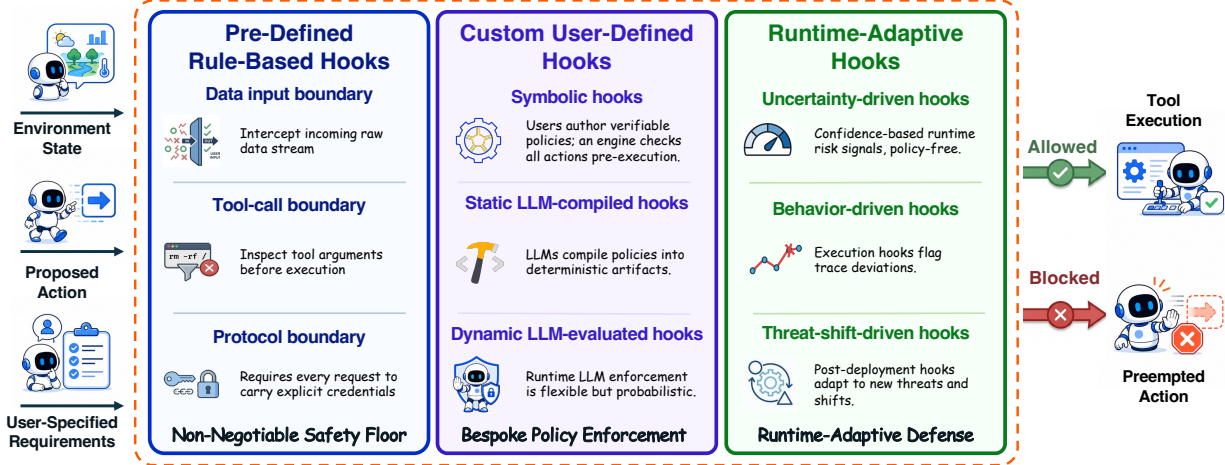


Figure 10 Overview of hooks and middleware. We categorize hooks into three paradigms based on the origin and adaptiveness of their safety criteria: pre-defined rules, user-defined policies, and runtime-adaptive signals.

voting for verifiable tasks [323, 442, 659] and through confidence-aware agentic aggregation for open-ended ones [296, 298]. This shift from prescribed hierarchies toward protocol-mediated coordination stands in productive tension with the deterministic governance we turn to next, and remains one of open problems.

4.5 Hooks and Middleware

The preceding subsections establish the full execution infrastructure of an agent harness: loops govern control flow, context and memory manage information state, tools extend capabilities, and orchestration composes agents into collaborative systems. Yet the very expressiveness that makes these mechanisms powerful also amplifies the risk surface: an agent equipped with file-system access, API credentials, and multi-step autonomy can cause irreversible harm through hallucinated commands or adversarial jailbreaks [1]. Prompt-level instructions alone are insufficient to constrain such behavior, as they remain vulnerable to the same failure modes they aim to prevent. Recent architectures [421, 529] have therefore shifted toward programmable boundaries enforced at the harness level, which we collectively term *hooks*. Hooks perform enforcement, gating the action path with allow, block, or hand-off decisions, whether through rule-based evaluation or model judgment. As illustrated in Figure 10, based on who defines the safety criterion and how the trigger condition is determined, we identify three complementary paradigms: (1) pre-defined rule-based hooks enforce universal safety invariants hard-coded by framework designers; (2) custom user-defined hooks enforce domain-specific policies authored by deployers; and (3) runtime-adaptive hooks derive their trigger conditions from runtime signals such as model uncertainty and behavioral patterns rather than from any predetermined specification.

4.5.1 Pre-defined Rule-based Hooks

At the foundation of agent safety, harnesses employ immutable, hard-coded middleware to intercept or block actions that violate universal safety invariants without requiring user configuration. These hooks are triggered at fixed lifecycle points such as immediately before a shell command or API call, and because they operate on the action path rather than the reasoning path, they run outside the LLM’s context window and are therefore less directly exposed to prompt-level manipulation; their coverage, however, still depends on policy completeness and adversarial robustness, since encoded payloads, obfuscated arguments, or gaps in the policy can still slip past them. Concretely, pre-defined hooks manifest at three successive boundaries along the agent execution pipeline:

- **Input Boundary.** Operating before data ingestion by the core language model, this mechanism intercepts incoming raw data streams, including user instructions, prompt templates, and retrieved external context. Its primary objective is to filter out adversarial prompt injections, jailbreaks, and malformed

system directives designed to compromise the model’s behavioral alignment. Systems implementing this boundary [529] deploy independent, token-level or semantic static filters to reject non-compliant inputs before they can influence the model’s internal attention state.

- **Tool-call Boundary.** Acting as a pre-execution firewall, this mechanism inspects tool arguments before execution to detect dangerous options (e.g., `rm -rf`) and common attack patterns such as code injection or unauthorized file access. AEGIS [783] instantiates this design by interposing a policy engine on every tool call, validating arguments against a curated set of known vulnerability patterns, including path traversal, with negligible latency overhead.
- **Protocol Boundary.** Enforced at the inter-component communication layer, this boundary requires every request to carry explicit credentials and receive per-action authorization, ensuring no message is implicitly trusted regardless of its origin. Representative systems include Authenticated Workflows [503], which attaches cryptographic tokens to protocol messages for authenticity verification, and MagenticUI [441], which mandates explicit user approval before dispatching agent actions.

These three boundaries form independent, complementary defense layers: because each targets a distinct attack surface, the failure of any single layer does not compromise the others [421]. This defense-in-depth property is further strengthened by architectural isolation techniques such as IsolateGPT [698], whose hub-and-spoke design routes all tool interactions through an isolated mediator to prevent cross-tool information leakage. Together, these pre-defined mechanisms establish a non-negotiable safety floor that holds irrespective of the model’s reasoning quality, the user’s configuration choices, or the particular task being executed.

4.5.2 Custom User-defined Hooks

Beyond immutable rules, modern harnesses expose programmable interfaces that allow developers to define domain-specific policies enforced at runtime. While pre-defined hooks target universally dangerous operations, custom hooks address the far broader space of application-specific requirements: **workflow constraints, regulatory compliance, access control scoping, and safety boundaries that vary across deployment contexts.** The defining characteristic is the locus of policy authorship: in every approach below, it is the user or developer, not the framework designer, who specifies the substantive safety criterion. “User-defined” thus refers to who writes the policy, not how it is enforced; the enforcement mechanism ranges from deterministic symbolic engines to probabilistic model evaluators, but the policy content originates with the deployer in all cases.

Existing approaches vary along two orthogonal dimensions: (i) how the user specifies a policy (formal language vs. natural language), and (ii) what enforces it at runtime (a deterministic symbolic engine vs. a probabilistic model). The interplay of these two dimensions yields three distinct categories. Formal-language policies pair naturally with symbolic enforcement. Natural-language policies split by whether an LLM translates them into executable artifacts offline and exits the enforcement path, or remains on the critical path to evaluate compliance at every step. The resulting spectrum trades specification effort against enforcement strength.

(1) Symbolic Hooks. Users author policies in a formal, machine-verifiable language, and a deterministic engine evaluates every action against the policy before execution. Because the LLM is entirely absent from the enforcement path, compliance guarantees are hard: a compliant action is provably within policy, and no prompt injection can subvert the check.

Several systems design event-triggered DSLs whose predicate guards and enforcement actions are executed by lightweight symbolic runtimes [509, 633]. Others go further by translating temporal-logic or JSON-schema specifications into first-order constraints and using SMT solvers or constrained token generation to make non-compliant tool calls physically impossible to emit [265, 542]. For multi-agent settings, PCAS [466] models inter-agent state as a causal dependency graph and enforces Datalog-derived policies via Differential Datalog with a reference monitor intercepting every action. A complementary line draws on operating-system security primitives such as capability labels and cryptographic audit records to enforce information-flow control and provenance tracking at the tool-call boundary [109, 617]. Across all variants, the enforcement engine is a formally deterministic program whose correctness can be audited independently of the LLM it guards.

(2) Static LLM-compiled Hooks. A second paradigm accepts policies written in natural language or semi-structured documents but compiles them into executable artifacts that are then enforced deterministically

or quasi-deterministically. The LLM participates at compile time but is absent from the enforcement path, yielding guarantees bounded by compilation quality but independent of runtime model reasoning.

The compilation targets range from executable guardrail code that runs deterministically to admit or block each action [708], to probabilistic rule circuits derived from regulatory documents and verified by formal model checkers [80], to formally verified policy code generated from Hoare-triple specifications and validated by an offline verifier before deployment [432]. The common thread is a two-phase architecture: an LLM-powered compilation step that transforms human-readable intent into machine-executable policy, followed by a symbolic or quasi-symbolic enforcement step. Errors can be introduced at compile time, but once compiled, enforcement proceeds without further LLM involvement.

(3) Dynamic LLM-evaluated Hooks. A third paradigm also starts from user-authored natural-language policies but, unlike static compilation, keeps the LLM on the enforcement critical path: the model directly evaluates each action or trajectory against the user-supplied policy at runtime. The deployer must provide domain-specific safety criteria without which the system has nothing to enforce, so the “user-defined” property holds in the same sense as the preceding categories. This maximizes expressiveness and minimizes specification effort at the cost of replacing hard guarantees with probabilistic ones.

One approach deploys a guardian LLM that interprets natural-language policies with contextual reasoning across multiple control points [285]. A classifier-based alternative fine-tunes or prompts language models as configurable safety classifiers whose risk taxonomies or policy definitions can be adapted without retraining [244, 274]. A third variant composes multiple heterogeneous scanners into a layered pipeline that users can extend with custom scanners tailored to their threat model [87]. Because the enforcer is itself a language model, these approaches can handle nuanced, context-dependent policies that resist formalization, but compliance is probabilistic and susceptible to the same failure modes that motivate guardrails in the first place.

Collectively, these three categories trace a design-space frontier between specification cost and enforcement strength: symbolic hooks demand the most expertise to author but provide the hardest guarantees; LLM-compiled hooks offer a middle ground by lowering the authoring barrier while keeping the LLM off the enforcement path; and LLM-evaluated hooks are the most accessible but the least formally bounded, trading deterministic compliance for the ability to express nuanced, context-dependent policies. In practice, production deployments increasingly layer multiple paradigms (e.g., symbolic access-control checks composed with model-based content classifiers) to exploit the complementary strengths of each.

4.5.3 Runtime-adaptive Hooks

While rule-based and user-defined hooks operate on predetermined conditions, a third, runtime-adaptive paradigm derives dynamic, context-sensitive triggers from runtime signals rather than fixed rules. The defining characteristic is that neither the trigger condition nor the enforcement threshold is fully specified before deployment; instead, these hooks continuously monitor the agent’s internal confidence, task progress, and external behavioral patterns in real time, activating timely protective measures when the system detects that the agent is operating outside its reliable competence range. We distinguish runtime-adaptive hooks from user-defined hooks by the locus of criterion definition: user-defined hooks enforce human-authored policies, whereas runtime-adaptive hooks let the system itself discover what to check from various data.

Existing approaches can be organized by the type of runtime signal to which the hook responds: the model’s own uncertainty, deviations in the agent’s behavioral patterns, or shifts in the external threat distribution.

(1) Uncertainty-driven Hooks. The most direct form of runtime adaptation monitors the model’s own confidence in its reasoning. These hooks extract calibrated risk scores from the inference process and surface them to downstream systems for intervention decisions, without themselves enforcing any fixed policy. Representative methods estimate semantic entropy over sampled completions to detect confabulations [151], propagate verbalized confidence scores across reasoning steps and trigger targeted recomputation when accumulated uncertainty exceeds a threshold [815], or model situational weights via hidden Markov models to extend uncertainty propagation to multi-step agentic reasoning [858]. Because these signals are derived from the model’s internal state at each inference step, the effective trigger condition adapts continuously without requiring any external training data or human-authored rules.

(2) Behavior-driven Hooks. A second class of hooks monitors the agent’s observable execution behavior and intervenes when action patterns deviate from learned expectations. Unlike uncertainty-driven hooks that read internal model states, these approaches build structural models of normal agent behavior from execution traces and use them to detect anomalies at runtime. One line of work learns formal behavioral models from historical traces and applies model checking to anticipate violations before they occur, enabling proactive intervention [634]. Another line operates at the trajectory level, training compact verifiers on synthetically perturbed trajectories for step-level error localization and targeted rollback [388], or constructing dynamic execution graphs from distributed traces to detect structural irregularities such as infinite loops, prompt injections, and collusive delegation across multi-agent interactions [214]. The common thread is that the definition of “normal behavior” is learned from data rather than specified by a human, making these hooks adaptive to the particular agent and task at hand.

(3) Threat-shift-driven Hooks. The preceding two categories respond to signals that arise within a fixed threat model; a third category addresses the scenario where the threat landscape itself evolves after deployment. These hooks detect that previously unseen attack types or domain shifts have rendered the current safety model inadequate, and update their own detection criteria accordingly. AdaptiveGuard [754] identifies novel attack types as out-of-distribution inputs on guardrail-model activations and rapidly adapts via continual learning, maintaining high detection performance where static guardrails degrade sharply. AGrail [408] takes a lifelong-learning approach: two cooperative LLMs generate and iteratively refine safety checks per action type via test-time adaptation, accumulating them in a memory module that generalizes across tasks. DuoGuard [119] employs adversarial co-evolution between a generator and a classifier during training to produce a robust safety classifier; notably, its adaptiveness is a training-time property and the deployed model is static, placing it at the boundary between this category and fixed guardrail classifiers.

Benchmarks such as Agent-SafetyBench [849] and ToolEmu [520], which reveal that even frontier agents struggle to achieve satisfactory safety scores, underscore the necessity of these signal-driven mechanisms. By deriving their trigger conditions from learned models rather than human-authored rules, runtime-adaptive hooks complement user-defined hooks: the former discovers what to watch for, while the latter enforces what the developer already knows.

4.6 Verification

Hooks establish deterministic boundaries that intercept or block actions, but interception alone is not sufficient: the system must also *evaluate* the quality, correctness, and safety of agent behavior at every stage. Verification mechanisms serve this complementary role, acting as the harness’s assessment engine that judges whether an intercepted action should proceed, whether intermediate reasoning is on track, and whether final outputs meet task requirements. Together with hooks, they close the governance loop by shifting the harness from a passive execution wrapper to an active controller. Table 4 provides a structured overview of representative verification and evaluation methods discussed in this subsection, organized by assessment target, verification level, and verifier strategy.

4.6.1 Assessment Targets

Verification mechanisms can be taxonomized by the property they certify: the factual correctness of outputs, the operational safety of actions, faithfulness to the user’s instructions, or agreement across verifiers.

(1) Correctness Verification. Correctness verification certifies the factual accuracy of agent outputs. SelfCheck-GPT [418] detects hallucinations through sampling consistency alone, providing a zero-resource baseline for factual self-assessment without any external knowledge source.

(2) Safety Verification. Safety verification certifies that agent actions do not cause harm before they take effect. VeriGuard [432] generates both functional code and formal verification artifacts for each action, achieving near-zero attack success. A rapidly growing line builds dedicated safety verifiers and diagnostic guardrails: AgentDoG [287] organizes moderation around a three-dimensional risk taxonomy (source, failure mode, consequence) for fine-grained, trajectory-level judgments with root-cause diagnosis beyond binary labels, and its successor AgentDoG 1.5 [288] reports strong moderation from a small amount of training data while supporting real-time online deployment. ToolSafe [440] targets tool-invocation safety with a step-level guardrail

Table 4 Representative verification and evaluation methods for LLM agents. Rows follow the three dimensions of this subsection: assessment targets, verification levels, and verifier strategies. **Train-free** (✓) marks methods that work with a frozen model, versus (✗) those requiring dedicated training.

Method	Train-free	Key mechanism
Assessment Targets		
SelfCheckGPT [418]	✓	Sampling consistency for factuality verification
VeriGuard [432]	✓	Verified code generation for action safety
ToolSafe [440]	✗	RL-trained proactive tool-invocation safety guardrail
NSVIF [563]	✓	Neuro-symbolic constraint checking for instruction faithfulness
Multiagent Debate [139]	✓	Inter-model deliberation for factual agreement
Multi-Agent Verification [362]	✓	Independent aspect verifiers for cross-verifier agreement
Verification Levels		
Let’s Verify Step by Step [363]	✗	Human-supervised process rewards for intermediate steps
AgentPRM [702]	✗	Per-decision process rewards for agent trajectories
CRITIC [182]	✓	Tool-interactive stepwise verify-then-correct loop
LLM-as-a-Judge [867]	✓	Model-based evaluation of final outputs
Agent-as-a-Judge [898]	✓	Tool-augmented assessment after task completion
Reflexion [544]	✓	End-of-trial verbal reflection with episodic memory
Verifier Strategies		
Math-Shepherd [643]	✗	Monte Carlo process labels for verifier training
Implicit PRM [787]	✗	Process rewards from policy logits without extra training
Self-Consistency [659]	✓	Majority voting over diverse reasoning paths
LATS [875]	✓	Tree search with LLM-generated value estimates
ReST-MCTS* [805]	✗	PRM-guided self-training via iterative tree search
Compute-optimal scaling [550]	✗	Difficulty-adaptive compute allocation with process verifiers

(TS-Guard) trained via multi-task reinforcement learning that flags unsafe calls before execution, substantially reducing harmful invocations. These verifiers are increasingly stress-tested by dedicated safety benchmarks. The need for them is underscored by AgentHarm [8], a motivating benchmark showing that leading LLMs remain surprisingly compliant with explicitly malicious multi-step tasks; complementary benchmarks then probe specific settings: R-Judge [790] measures safety-risk awareness over multi-turn interaction records, RedCode [193] probes risky code execution and generation for code agents, ST-WebAgentBench [303] evaluates policy-constrained actions for web agents, and OS-Sentinel [571] (distinct from the SentinelAgent runtime monitor of §4.5) combines formal and neural validation for mobile GUI agents in realistic workflows.

(3) Faithfulness Verification. Faithfulness verification certifies that an agent follows the given instructions and stays on task, rather than drifting toward a superficially plausible but misaligned objective. Neuro-symbolic checkers translate natural-language instructions into formal constraints that an output can be mechanically checked against [563], while instruction-following evaluations for multi-turn, tool-using web agents test whether constraints stated early in a session are still honored many steps later [118]. This concern is sharpened by evidence of *goal drift*: agents can gradually abandon their original objective under accumulated context or contextual pressure, so faithfulness verification must track adherence across the whole trajectory rather than at a single step [20, 427]. In agentic settings it is thus the natural counterpart to the maker-checker separation of Section 4.1.3: an independent checker guards against completions that satisfy the visible goal while violating the user’s underlying intent.

(4) Multi-agent Cross-verification. Multi-agent cross-verification employs several models for mutual critique. Multiagent debate [139] shows that iterative inter-model deliberation substantially reduces factual errors, and Multi-Agent Verification [362] demonstrates that scaling the number of independent aspect verifiers yields weak-to-strong generalization.

4.6.2 Verification Levels

Verification mechanisms can also be characterized by the stage at which they intervene during the agent’s execution lifecycle.

(1) Step-level Verification. Step-level verification evaluates intermediate reasoning quality at each step, complementing hooks that enforce binary allow-or-block decisions via deterministic rules. Where a hook asks “is this action permitted?”, a step-level verifier asks “is this reasoning step productive toward the goal?”, producing scalar quality signals that guide search and correction rather than gating execution. Building on earlier evidence that training answer verifiers and combining process- with outcome-based feedback improves math reasoning [96, 618], process reward models (PRMs) [363] show that per-step supervision substantially outperforms outcome-only evaluation for complex reasoning, AgentPRM [702] extends this to agentic tasks by scoring each decision on goal proximity and incremental progress, and CRITIC [182] introduces a tool-interactive verify-then-correct loop that establishes external feedback as essential for reliable self-correction. Recent work advances step-level verifiers along three axes. *Generative verification*, which recasts reward modeling as next-token prediction [822], lets GenPRM [857] and ThinkPRM [272] reason step-by-step before scoring, showing that generative verifiers trained on minimal labels can match or outperform discriminative PRMs and rival strong proprietary judges on process benchmarks. *Domain-specialized PRMs* tailor verifiers to concrete agent domains: Web-Shepherd [49] builds structured subgoal checklists for web navigation at substantially lower cost than GPT-4o-based verification, while SWE-PRM [163] and SWE-TRACE [203] deliver trajectory-level course-correction and rubric-based dense feedback for software-engineering agents. *Efficient supervision* concentrates labels where they matter: CSO [329] restricts preference learning to verified critical steps that demonstrably flip task outcomes, requiring supervision at only a small fraction of trajectory steps, and ToolPRMBench [311] contributes the first large-scale PRM benchmark for tool-using settings, showing that reinforcement learning substantially improves robustness and generalization in tool-specialized verifiers. Most recently, step-level verification is moving toward fully *agentic* verifiers that themselves reason and invoke tools while scoring each decision, rather than emitting a single scalar in one pass, as in AgentV-RL [818].

(2) End-of-run Verification. End-of-run verification evaluates agent outputs after task completion. The LLM-as-a-Judge paradigm [867] uses strong models to score final outputs, and its evolution toward agentic judges has proceeded along two lines. *Agent-as-a-Judge* augments the judge with tools and multi-step reasoning: Agent-as-a-Judge [898] equips the evaluator with tool-augmented assessment for substantially higher human agreement, and a recent paper [775] systematizes how agentic judges combine planning, tool-augmented verification, and persistent memory. *Multi-agent collaborative evaluation* leverages inter-model deliberation: CollabEval [489] runs a three-phase initial-evaluation, discussion, and consensus process that outperforms competitive debate; a complementary route instead scales test-time compute for judging itself, which we discuss under verifier strategies below. *Self-correction and environment grounding* close the loop after execution: Reflexion [544] performs verbal self-reflection with episodic memory across trials, though CorrectBench [605] finds that self-correction yields only limited gains on reasoning models at significant cost; environment-based frameworks such as WebArena [885] and SWE-bench [260] verify success through functional correctness of final states, and SWE-RM [546] complements test-suite checks with an execution-free reward model that measurably lifts SWE-bench Verified accuracy. Chain-of-Verification [121] decomposes a response into atomic claims for independent post-hoc checking. Moving beyond static post-hoc scoring, agentic verifiers increasingly probe the environment after a run: verifying a GUI agent through online, proactive interaction lets the judge actively confirm claimed outcomes rather than trust the final transcript alone [102].

4.6.3 Verifier Strategies

Orthogonal to temporal granularity, verifier quality can be improved through training-free architectural designs and learning-based methods, which we group into three families.

(1) Automated labels and implicit rewards. Math-Shepherd [643] automates process-label generation via Monte Carlo estimation, eliminating costly human annotation, and Implicit PRM [787] shows that process rewards can be extracted from outcome-trained models at no additional cost. Self-consistency [659] offers a complementary training-free signal through majority voting across diverse reasoning paths.

(2) Tree search with verifier guidance. A family of methods couples Monte Carlo tree search with LLM self-

evaluation for structured exploration. LATS [875] and rStar-Math [191] combine MCTS with model-generated value estimates, AB-MCTS [436] adds adaptive branching that decides whether to “go wider” or “go deeper,” and ReST-MCTS* [805] develops reinforced self-training that harvests higher-quality reasoning traces through iterative process-reward-guided tree search.

(3) Compute-optimal Verification Scaling. Test-time compute scaling [550] shows that verifier-guided search lets smaller models match substantially larger ones, and DeepSeek-R1 [112] demonstrates that self-verification can emerge from reinforcement learning with rule-based rewards alone. RLV [521] unifies LLM reasoners with learned value functions, improving MATH accuracy while enabling markedly more efficient scaling, while MCTS-Judge [665] carries test-time scaling into the LLM-as-a-Judge setting, markedly improving code-correctness accuracy through System-2 reasoning. Extending this scaling to specialized domains, tool-integrated verification interleaves reinforcement learning with external tools so that the verifier can certify multi-step reasoning that pure model judgment cannot [812].

5 Optimization: Internalizing Long-Horizon Capability

Overview. In Section 4, we describe how long-horizon capability can be supplied by the harness that specifies, steers, and consolidates each run. This section turns to *optimization*: how such capabilities can be progressively internalized into model parameters. We organize the section along the *training pipeline*. We begin with the *architectural substrate* (Section 5.1), which determines whether long-horizon training and inference are computationally feasible in the first place: a policy that cannot represent and efficiently process a long trajectory cannot be effectively trained on one. We then examine six stages of capability development: data and environment synthesis (Section 5.2), pre-training and mid-training (Section 5.3), fine-tuning (Section 5.4), reinforcement learning (Section 5.5), on-policy distillation (Section 5.6), and self-evolution (Section 5.7). For each stage, we identify the long-horizon capabilities it develops, the mechanisms through which they are acquired, and the limitations that remain. Figure 11 provides an overview of the pipeline, while Table 5 summarizes the characteristics, mechanisms, and representative works associated with each stage.

The Architectural Substrate and Six Training Stages

1. **Architectural Substrate** (Section 5.1): makes long trajectories efficient to train on and decode from.
2. **Data and Environment Synthesis** (Section 5.2): constructs verifiable long-horizon experience, including tasks, environments, and trajectories, that later stages can use for training and feedback.
3. **Pre-/Mid-training** (Section 5.3): builds action-observation and reasoning priors, so agentic behavior becomes native rather than prompted.
4. **Fine-tuning** (Section 5.4): instills behavioral and tool-use discipline from curated agentic trajectories.
5. **Reinforcement Learning** (Section 5.5): optimizes whole trajectories and confronts long-horizon credit assignment under sparse, delayed reward.
6. **On-policy Distillation** (Section 5.6): consolidates the policy on its own state distribution, reducing distribution shift and stabilizing earlier learned behaviors.
7. **Self-evolution** (Section 5.7): turns self-generated experience, feedback, and task-environment updates into continuing policy improvement.

5.1 Architectural Substrate

A long-horizon agent can internalize only what its architecture can represent: a policy that cannot ingest, retain, and efficiently use a long interaction history, such as observations, tool results, and constraints that accrue across many steps, cannot be trained or served on long trajectories. The architectural substrate therefore sets the ceiling for every later training stage, which is why we begin here. From an architecture view, the central question is how the model keeps and uses a long history. We group the main approaches into three representational classes: explicit-context architectures, compressed-state architectures, and hybrid

Table 5 The agentic training pipeline by stage. Each row is the architectural substrate or one of the training stages. **Representative mechanisms** lists the stage’s sub-mechanisms (one per line, following its subsections), and **representative works** pairs each sub-mechanism with two concrete works on the same line.

Stage (§)	Internalized capability	Representative mechanisms	Representative works
Architectural Substrate (§5.1)	Affordably representing and serving long histories	• Explicit-context • Compressed-state • Hybrid memory • High-throughput	Longformer [32], Big Bird [792] Mamba [186], RWKV [476] Jamba [300], Kimi Linear [842] EAGLE [352], GQA [5]
Data & Environment Synthesis (§5.2)	Executable, verifiable long-horizon experience	• Task synthesis • Environment synthesis • Trajectory synthesis	TaskCraft [540], WebShaper [590] SWE-Gym [467], EnvScaler [555] TOUCAN [737], AgentGym-RL [701]
Pre-/Mid-training (§5.3)	Reasoning and perception priors	• Reasoning priors • Long-context priors • Multimodal priors • Data-mixture design	DeepSeek-V3 [110], Qwen3-Coder [46] YaRN [477], LongLoRA [78] Qwen2-VL [644], InternVL3 [892] DoReMi [715], OPUS [649]
Fine-tuning (§5.4)	Behavioral and tool-use discipline	• Selection & mixing • Curriculum • Distillation	AgentTuning [794], LIMi [711] ScalingInter-RL [701], E2H Reasoner [469] Agent Distillation [269], Agent-R [789]
Reinforcement Learning (§5.5)	Feedback-driven long-horizon decisions	• Credit assignment • Policy optimization • Sampling strategy • Interaction patterns	ToolRL [487], RuscaRL [887] DAPO [780], GiGPO [156] Tree-GRPO [250], TCOd [636] GLIDER [232], Memory-R1 [743]
On-policy Distillation (§5.6)	Consolidating behavior on the policy’s own states	• Teacher-guided • Self-improving	GKD [3], Dagger-LLM [309] π -Play [838], Self-Distillation Zero [216]
Self-evolution (§5.7)	Cross-task experience accumulation	• Offline bootstrap • Online interaction • Co-evolution	STaR [793], rStar-Math [191] R-Zero [233], Agent0 [704] Env Tuning [401], Agent-World [131]

architectures. These are not strict model families, but different ways to represent past information within the model. We discuss high-throughput mechanisms separately as a serving-oriented axis rather than as a fourth representation class. This view follows the main split in efficient Transformer and state-space papers [591]. We do not treat Mixture-of-Experts (MoE) as a fourth representational class. MoE routes tokens to different feed-forward experts, as in GShard and Switch Transformer [152, 301]. It primarily changes model capacity, but it does not by itself decide how the sequence history is stored. Accordingly, MoE models can still use explicit attention, latent attention, or hybrid blocks [111, 300, 601].

5.1.1 Explicit-Context Architectures

Explicit-context architectures keep history as a token sequence. The model then uses attention to route information across these tokens. Full attention gives the cleanest form of this design, because any token can attend to any other token [620]. Long-context Transformer variants reduce the cost with local attention, sliding windows, block-sparse attention, or global tokens. Longformer and Big Bird are classic examples of this direction [32, 792]. Mistral 7B uses sliding-window attention and grouped-query attention (GQA) for lower-cost decoding [252]. Gemma 2 and Gemma 3 use local-global attention patterns. Gemma 3 uses more local attention layers to reduce KV-cache cost at long context [595, 596]. Efficient Transformer papers give a broader view of this design space [591]. More recent work makes sparsity *natively trainable* rather than an inference-time approximation, making long-horizon training affordable: DeepSeek NSA couples a hardware-aligned, natively trainable sparse-attention design with efficient kernels [786], and MoBA applies a mixture-of-block-attention routing that lets each query attend to a learned subset of context blocks [398].

This class is useful for long-horizon agents because earlier states remain explicitly represented. The model can, in principle, look back at an old observation, a tool result, or a user constraint. These objects are common in reasoning-and-acting agents and web agents [761, 885]. The cost is the main weakness. As the context grows, attention compute and key-value (KV) cache size also grow [5, 620]. If the model uses a sparse pattern, it may also miss a remote token that matters. Long-context benchmarks and analyses show that locating useful evidence inside long inputs remains hard [28, 378]. Many open base model reports still fit this class. Qwen2.5 and Qwen3 mainly use decoder-only Transformer architectures; Qwen3 adds both dense and MoE variants,

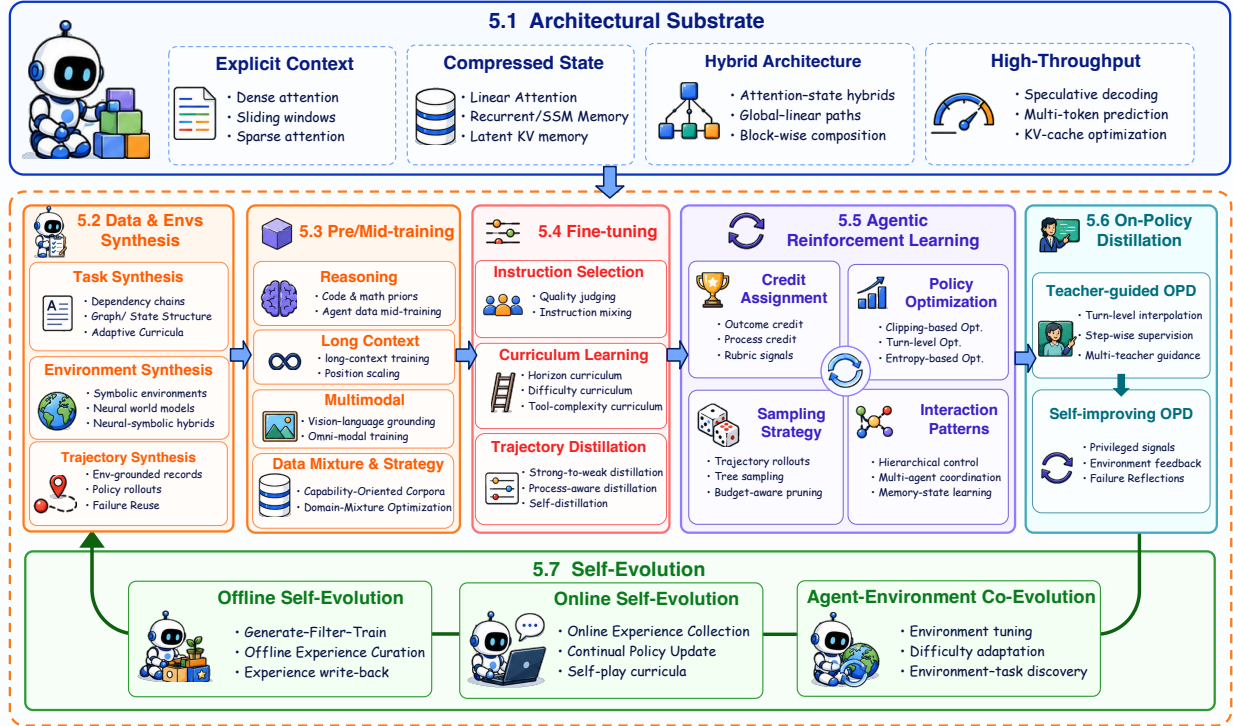


Figure 11 The agentic training pipeline for internalizing long-horizon capabilities. As the internalized counterpart to the externalized harness described in Section 4, the pipeline builds on an architectural substrate that supports long-sequence representation and proceeds through six stages. Each stage either internalizes a capability previously provided at runtime or generates the experience required for its subsequent internalization.

but the history is still accessed through attention [601, 744]. The GLM series follows the same pattern. The ChatGLM report describes the path from GLM-130B to GLM-4. GLM-4.5 is an MoE foundation model for agentic, reasoning, and coding tasks, but it is still an attention-based Transformer family [178, 795].

5.1.2 Compressed-State Architectures

Compressed-state architectures do not keep all pairwise token interactions. They instead encode history into a more compact internal representation. Some methods compress the attention map or the token set, as in low-rank attention and kernelized linear attention [92, 651]. Other methods use a recurrent or state-space form. This includes Retentive Network (RetNet), Receptance Weighted Key Value (RWKV), structured State Space Models (SSMs), and selective SSMs such as Mamba [186, 187, 476, 576].

This class fits the stream-like nature of agent work. In many agent settings, a model observes, reasons, acts, and observes again [761, 885]. A compressed state can be updated step by step, so the runtime memory can stay small [186, 187, 476, 576]. The weakness is precision. Once old tokens are folded into a state, the model may not recover a rare fact or an exact old string. This is a core trade-off between efficient recurrence and content-level access [473, 606]. This is a problem when an agent must obey a past constraint exactly, as in tool-use and web tasks that carry instructions across steps [761, 885]. A partial, and importantly different, base-model version of this idea is Multi-head Latent Attention (MLA), which compresses the KV cache into a latent form. DeepSeek-V2 introduced MLA together with DeepSeekMoE, and DeepSeek-V3 kept this design at larger scale [110, 111]; Kimi-K2 likewise pairs a large MoE model with MLA [597]. Unlike recurrent or state-space compression, however, MLA keeps a per-token latent and still attends over all past tokens, so it preserves per-token addressability and does not incur the exact-recall loss described above. It is therefore best read not as a recurrent compressed state but as a low-rank KV compression of explicit context, sitting between the two classes and doubling as a throughput mechanism (Section 5.1.4).

5.1.3 Hybrid Architectures

Hybrid architectures combine explicit attention with compressed or recurrent state [300, 842]. The goal is to combine precise content access with efficient long-range state tracking. Attention can keep precise access to important tokens [378, 620]. A compressed state can carry long-range signals at low cost [186, 606]. In this view, attention is used for exact evidence, while the state module is used for long-term continuity [300, 842]. Jamba is a direct example: it mixes Transformer layers, Mamba layers, and MoE layers [300]. Kimi Linear is another clear example. It combines Kimi Delta Attention units with a global MLA layer. The model therefore keeps both a linear-attention path and an explicit global attention path [842].

This makes hybrid design a plausible direction for long-horizon agents. Agent runs need both exact recall and cheap long-term tracking, as shown by web-agent and long-context tasks [28, 885]. Pure attention gives direct access but can be expensive at long context [591, 620]. Pure compressed state is cheaper but may lose details [473, 606]. A hybrid model can assign these roles to different layers, heads, or blocks, as in Jamba and Kimi Linear [300, 842]. The reports above show the same direction at system scale: strong models often combine attention, KV-cache reduction, and MoE routing [110, 597, 601]. These parts solve different problems and should not be collapsed into one label [5, 152, 301].

5.1.4 High-Throughput Mechanisms

High-throughput mechanisms form a second axis under architecture [5, 111, 180, 302, 535]. They do not decide how the model stores long history; they reduce the serial work or cache traffic needed to decode from that history [5, 111, 180, 302, 535]. A complementary, kernel-level route keeps attention *exact* but IO-aware: FlashAttention tiles the computation to avoid materializing the full attention matrix in high-bandwidth memory, cutting memory traffic and enabling longer sequences without altering the model’s outputs [108]. One group reduces serial dependence in the output path through parallel generation with diffusion language models [559] or through autoregressive lookahead [43, 180, 302, 352]. Multi-token prediction trains the model to predict several future tokens at each position, so the model learns a longer local prediction horizon [180]. Speculative decoding uses a draft model to propose several tokens and lets the target model verify them in parallel [302]. Medusa keeps the draft process inside one model by adding multiple decoding heads [43]. EAGLE moves the draft signal into feature space and treats feature uncertainty as central to speculative sampling [352]. DeepSeek-V3 uses multi-token prediction as an auxiliary training objective and also presents it as a path to speculative decoding [110].

The other group reduces the cost of reading and writing the KV cache [5, 111, 535]. Multi-query attention shares key and value projections across query heads to speed decoding [535]. Grouped-query attention keeps more key-value groups than multi-query attention, but still reduces the cache cost relative to full multi-head attention [5]. MLA compresses the KV cache into a latent form, so it is both a compressed-state design and a throughput design [110, 111]. These mechanisms matter for long horizon agents because long runs often produce many intermediate tokens after a large context has already been built [180, 302, 761, 885]. They reduce the serving cost of long decoding, but they do not remove the need for explicit recall, compressed state, or hybrid memory when old evidence must be found accurately [28, 180, 302, 378].

5.2 Data and Environment Synthesis

Long-horizon behavior is learned from interaction experience that is executable, feedback-bearing, and reusable. Unlike static instruction–response pairs, such experience records how an agent decides, observes feedback, corrects errors, and accumulates information over a long trajectory. Deployment logs, demonstrations, and benchmark rollouts offer natural sources of such data, but their direct use is constrained by acquisition cost, privacy, and limited reproducibility. Controlled data synthesis has therefore become an important route for constructing long-horizon training experience. It centers on three coupled objects: *tasks* define goals and capability pressures, *environments* instantiate execution and verification, and *trajectories* turn interaction histories into trainable signals.

5.2.1 Task Synthesis

Task synthesis determines the objective distribution for long-horizon agents. A task defines the target behavior through goals, constraints, initial and terminal states, and success criteria. Task generation sits at the entry of interactive experience collection, since a task definition largely fixes horizon, dependency structure, and failure modes [238, 808]. Existing methods mainly construct tasks through dependency chains, structured information spaces, state transitions, and capability-adaptive curricula.

Dependency-chain Construction. The most direct way to lengthen horizon is to chain subgoals so that early decisions constrain the actions later available. Long-horizon diagnostics relate failures to intrinsic horizon and compositional depth [658]; horizon-length studies further show that longer tasks aggravate optimization instability and credit assignment [279]. AgentSynth composes easy-to-generate subtasks into longer computer-use tasks, while TaskCraft expands atomic tasks into multi-hop, multi-tool objectives [540, 713]. The horizon here grows through necessary intermediate dependencies rather than through mere interaction length.

Graph-structured Information Construction. When horizon arises from evidence rather than from actions, tasks are drawn as paths through web pages, documents, tools, or evidence graphs, and the difficulty shifts to selecting relevant evidence under branching, incomplete, or masked observations. WebWalker and WebDancer construct multi-step information-seeking tasks from traversal, crawling, and backward question evolution [689, 690]. WebSailor uses knowledge-graph walks and information masking, while WebShaper formulates information seeking through set and knowledge projection to control evidence structure, answer space, and verifiability [327, 590]. WebSailor-V2 and WebWeaver further extend this line to more open-ended information-seeking tasks [326, 356].

State-transition Construction. A third view frames a task as the gap between an initial and a target state, or as recovery from an erroneous one. In software settings, SWE-Gym, R2E, and SWE-smith reconstruct tasks from repositories, issues, commits, and tests [248, 467, 748]. Reverse synthesis makes this state-transition view explicit: DIVE generates tasks from executed evidence traces, TASTE evolves valid tool-sequence skeletons, and CLI-Gym derives repair tasks from abnormal command-line states and historical errors [53, 271, 369]. Execution traces and state gaps are thereby recast as tasks with verifiable outcomes.

Curriculum-driven Construction. The routes above fix a task distribution in advance; curricula instead let difficulty track the current policy, reducing the mismatch between static task sets and evolving capability. WebRL generates new web tasks from failed attempts, and Learn-by-interact reconstructs instructions and supervision from environment documents and interaction histories [484, 562]. AgentFrontier targets the model’s zone of proximal development, while CuES uses exploration and curiosity signals to synthesize environment-grounded RL tasks [75, 417]. Self-evolving agent papers view such mechanisms as part of joint agent–task evolution [165].

These routes impose complementary capability pressures on plan maintenance, evidence selection, and state recovery, whereas curricula act orthogonally, tuning how fast difficulty grows rather than what it tests. Which pressure to emphasize depends on the target deployment, since a distribution poorly matched to the policy tends to either saturate or destabilize training.

5.2.2 Environment Synthesis

Environment synthesis supplies the training ground where long-horizon agents accumulate the interaction experience that static corpora cannot provide. Whereas tasks specify what to solve, environments determine how actions are executed, observed, and rewarded, closing the action–observation–feedback loop that turns a passive predictor into an active decision maker. A synthesized environment makes this loop reproducible, resettable, parallelizable, and verifiable, so that agents can act, fail safely, and receive checkable feedback at scale. Real repositories, browsers, APIs, databases, desktop systems, and game simulators provide fidelity [145], but only controlled synthesis makes such interaction cheap and reliable enough for repeated rollout. We organize environment synthesis into three families according to how their transitions and feedback are produced: symbolic environments, neural environments, and neural–symbolic hybrids.

Symbolic Environments. Symbolic environments implement transitions and feedback through code, tests, databases, or business logic, which yields stable execution and verifiable hard signals. This family has developed along several technical routes, distinguished by the substrate they make executable. A first route builds *repository-level software environments*: SWE-Gym packages real repositories, issues, dependencies, and tests into executable tasks, and follow-up systems scale repository construction, test generation, and task expansion [248, 467, 748, 797, 856]. A second route targets *terminal and command-line environments*, where Terminal-Bench standardizes hard, realistic shell tasks and later work co-derives instructions, sandboxes, and teacher trajectories to scale terminal-agent training [82, 86, 369, 428]. A third route provides *GUI, web, and game execution environments*, represented by WebArena, OSWorld, and GameWorld and extended toward broader interface and generation settings [251, 461, 695, 716, 852, 885]. A fourth route constructs *tool- and domain-specific environments*, ranging from multi-step tool substrates such as AgentScaler to grounded scientific and medical settings [148, 538, 555, 626, 729]. Across these routes, explicit rules stabilize verification but remain less effective under ambiguity, dynamic interfaces, and cross-tool side effects.

Neural Environments and World Models. Neural environments approximate transitions, observations, or feedback with LLMs, visual generators, or latent dynamics, reducing the cost of repeated interaction. These methods treat the model as a *world action model*: conditioned on the interaction history and the agent’s action, it simulates the next round of environmental feedback, so that rollout no longer touches the real system. Three technical routes differ mainly in the representation over which the next state is predicted. The first route learns *pixel-level world models*, represented by DIAMOND and GameNGen, which generate action-conditioned visual futures and have since been extended to embodied and open-world settings [2, 6, 89, 170, 196, 249, 619, 769, 839]. The second route builds *text-level simulators* better suited to web, GUI, tools, and code, where WebDreamer predicts future web states and later systems structure UI, webpage, or software states [47, 149, 188, 572, 661, 712, 870]; recent language world models push this route toward general agents, learning next-state prediction across many agent domains and coupling generative simulators with agents in difficulty-aligned co-evolution [195, 905]. The third route learns *latent world models* in the style of joint-embedding prediction, which compress dynamics for cheaper long-horizon prediction [24, 25, 30, 176]. Neural environments scale cheaply, but they may introduce hallucination, drift, unreliable feedback, and sim-to-real gaps.

Neural-Symbolic Hybrid Environments. Neural-symbolic hybrid environments keep hard state and final verification in databases, file systems, programs, or APIs, while using models for user simulation, language interfaces, or hard-to-specify observations. They can be grouped by where the model is inserted into an otherwise symbolic loop. One route places models at the *user and application interface*, as in τ -bench, which combines stateful applications, user simulation, and final-state checks [399, 614, 762]. A second route standardizes *tool-server backends* through MCP [299, 411, 668, 700]. A third route pairs *generated content with symbolic constraints*, coupling generated webpages with state-machine constraints and completion checks [50, 695, 852]. A fourth route grounds *domain environments* in real data, retrieval, or computation while using models to expand tasks and feedback [538, 626, 804, 894]. The design choice throughout is where to draw the boundary between model-based simulation and symbolic verification.

Across these families, feedback and verification quality remain central. Software tasks can rely on tests and execution; tool tasks use persistent database, file, or API states; web and GUI tasks often require scripts, page-state checks, visual-semantic matching, or expert rules; and open-ended tasks depend more heavily on rubrics, LLM judges, or hybrid verifiers. Dense step-level signals can improve credit assignment, but stronger automation also raises judge bias, verifier loopholes, and reward-hacking risks [238, 402]. Synthesized environments must therefore balance correctness, diversity, complexity, and fidelity.

5.2.3 Trajectory Synthesis

For long-horizon agents, trajectory synthesis internalizes interaction as trainable behavioral experience: it records how a policy acts, observes, errs, and recovers over an environment, and converts that history into supervision, preferences, rewards, and memory. What can be recorded is bounded by the environment’s observability and verification, so trajectory quality inherits the strengths and gaps of the synthesized environment. A useful trajectory keeps the cross-step dependencies, delayed feedback, and recovery decisions

that plain instruction–response data discard. Prior work develops this idea along four directions: grounding records in real environments, aligning rollouts with the trained policy, assigning step-level credit while reusing failures, and compressing long histories into reusable experience.

Environment-grounded Execution Records. Environment-grounded records capture state changes, tool side effects, and feedback produced during execution. Textual ReAct traces or function-call transcripts provide useful templates, but they often omit post-action states and failure signals. TOUCAN synthesizes large-scale tool-agentic trajectories in real MCP environments; OpenResearcher anchors deep-research trajectories in a reproducible offline corpus; and ASTRA constrains multi-turn tool trajectories with tool-call graphs [354, 604, 737]. WebSTAR, ANCHOR, and HATS further tie GUI and computer-use trajectories to interface states, branch points, action ambiguity, and step-level feedback [215, 531, 679].

Policy-aligned Rollouts. Policy-aligned rollouts reduce distribution shift between collected trajectories and the policy being trained. Expert, human, or strong-model trajectories provide useful cold-start priors, whereas on-policy or near-on-policy rollouts expose the current policy’s own exploration, failures, and recovery states. AgentGym-RL, AgentRL, and ASearcher combine multi-turn rollout, horizon scheduling, and asynchronous sampling; DPEPO uses parallel trajectories for cross-path exploration; and Agent Lightning decomposes execution records into RL-consumable units [166, 409, 701, 811, 820].

Step-level Credit Assignment and Failure Reuse. Step-level credit assignment exposes local correctness, redundant actions, and critical errors that are hidden by final success labels. WebSTAR filters learnable GUI actions with a process reward model; ANCHOR expands and cleans post-branch trajectories; and HATS targets difficult segments under contextual and visual ambiguity [215, 531, 679]. CSO identifies outcome-changing critical steps, VPR constructs verifiable turn-level rewards, and GEAR reallocates trajectory-level advantages to segments [329, 333, 785]. The unit of supervision shifts from whole trajectories to steps, segments, and critical decisions. Failure trajectories provide complementary training signals for planning, evidence selection, tool use, state recovery, and harness interaction. CSO generates critical-step replacements; AgentHER relabels failed attempts under reachable alternative goals; and AOI and PIVOT use failed records for correction and trajectory refinement [125, 329, 751, 826]. HORIZON and HarnessFix further show that failures may stem from tools, context management, verifiers, or harnesses rather than model decisions alone [64, 658]. Useful trajectories therefore retain errors, control flow, state changes, and verification provenance.

History Compression and Experience Reuse. History compression addresses the context and credit-assignment difficulties caused by long interaction records. ReSum trains over summary-bounded segments; Context-Folding and AgentFold organize trajectories into foldable subprocesses; and Memory-as-Action and Memory-R2 treat memory edits as trajectory decisions with credit-assignment consequences [573, 693, 742, 768, 846]. Long-horizon trajectories can thus be segmented, summarized, folded, converted into memory states, and abstracted into reusable experience.

In summary, trajectory synthesis is moving from text-only demonstrations toward environment-grounded, verifiable, and reusable agentic experience. Task synthesis sets the capability pressure, environment synthesis supplies the interaction loop, and trajectory synthesis turns execution histories into trainable signals.

5.3 Pre-training and Mid-training

Pre-training and mid-training are the earliest stages for internalizing the capabilities required by long-horizon agents. Pre-training establishes broad linguistic, factual, coding, mathematical, and reasoning priors through large-scale self-supervised learning, while mid-training further reshapes these priors before instruction tuning or reinforcement learning. For long-horizon agents, these stages are important because later supervised fine-tuning (SFT), alignment, and policy optimization can only elicit and refine capabilities that are already sufficiently represented in the base model. We therefore discuss pre-training and mid-training from four aspects: reasoning capability, long-context state modeling, multimodal perception, and the data mixture and training strategies that support these capabilities.

5.3.1 Reasoning Priors

Reasoning is a foundation for long-horizon agency because agents must decompose goals, maintain intermediate hypotheses, evaluate partial results, and revise plans over multiple steps. Code and mathematics are representative sources for injecting such priors during pre-training and mid-training, since they expose models to structured dependencies, executable programs, multi-step derivations, and verifiable intermediate states. Recent model reports increasingly emphasize coding, mathematics, reasoning, and agentic capabilities as part of their training recipes, including Qwen2.5 [744], DeepSeek-V3 [110], GLM-4.5 and GLM-5 [178, 179], and Kimi K2 [597]. Qwen3-Coder-Next [46] makes this direction more explicit through targeted mid-training toward code reasoning, repository-level understanding, and agent-style interaction patterns. These structured domains make reasoning supervision closer to long-horizon agent workloads than ordinary short-form text, because they require cross-step consistency, error checking, and compositional problem solving.

5.3.2 Long-Context State

Long-context capability provides the substrate for maintaining task state across extended interactions. For a long-horizon agent, context is not merely a longer input window; it serves as a serialized working state that may contain goals, observations, retrieved evidence, tool outputs, failed attempts, intermediate plans, and revised decisions. A first line of work extends the usable window by rescaling rotary position embeddings: Position Interpolation linearly rescales positions to fit a target length [66], YaRN improves this with an NTK-aware, frequency-dependent schedule for cheaper extension [477], and LongLoRA makes such extension parameter-efficient through shifted sparse attention during fine-tuning [78]. A complementary line studies the training recipe rather than the positional scheme. ProLong shows that long-context capability depends on continued-training data composition, training sequence length, and instruction tuning [171]; UltraLong pushes context to the million-token regime through continued pre-training and instruction tuning [721]; and LongWriter shows that long input windows do not by themselves induce long-output behavior without long-output training examples [29]. Recent frontier models also operationalize this direction. GLM-5.2 couples a 1M-token context window with specialized training for long-horizon coding-agent scenarios [179]. What matters for agents is therefore whether extended context is actually used for state tracking and evidence integration, not the maximum length a model nominally accepts [133, 625].

5.3.3 Multimodal Perception

Long-horizon agents increasingly operate in environments where task state is distributed across webpages, screenshots, videos, and audio streams; their perceptual priors must therefore extend beyond linguistic shortcuts that can mask basic visual failures [61, 502, 608, 624]. Open vision-language models establish the basic substrate: Cambrian-1 explores vision-centric representations and visual grounding [607]; Qwen2-VL perceives images and long videos at varying scales through dynamic resolution and multimodal rotary position embedding [644]; and the InternVL3 series advances open pre-training for high-resolution, long-context visual understanding [654, 892]. Recent industrial reports push these priors toward agent-centric perception, with the common thread being that visual grounding and GUI control are trained as native capabilities rather than added after a text-only stage: Seed1.5-VL and the subsequent Seed1.6 series pre-train on large-scale interleaved vision, text, and interaction data to sustain long-video understanding and GUI control across evolving screens [42, 194], while MiMo-VL treats visual grounding and GUI localization as first-class targets on a reasoning-optimized backbone [393, 394]. On the omni-modal side, Qwen3.5-Omni and Kimi K2.5 extend joint text, vision, and audio pre-training toward multimodal agentic intelligence [498, 598]. The shared implication is that long-horizon capability depends on persistent multimodal state tracking and cross-modal evidence grounding, not textual reasoning alone.

5.3.4 Data Mixture and Training Strategies

The above capabilities are shaped by how training data are selected, mixed, and structured. Recent work shifts pre-training from generic web-scale language modeling toward capability-oriented data distribution design. DataComp-LM [316], FineWeb [475], Dolma [552], and OLMo [184] make pre-training data construction more reproducible. Beyond static corpus construction, DoReMi [715] and Data Mixing Laws [767] show that domain proportions can be optimized or predicted with proxy experiments, while OPUS [649] performs

principled data selection at each pre-training iteration by scoring sample utility in the optimizer-induced update space, shifting pre-training from more tokens to better tokens and improving both from-scratch and continued pre-training under a fixed compute budget. For long-horizon agents, such data strategies act as inductive biases for reasoning, state modeling, and compositional problem solving, rather than merely as mechanisms for scaling factual knowledge. A further strategy is to structure training data as trajectories rather than isolated text samples. AgentTuning [794], Agent-FLAN [79], and AgentBank [557] show that interaction trajectories can improve generalized agent abilities such as planning, memory, and tool use. Since high-quality long-horizon trajectories are scarce, synthetic and verifier-guided data are also used to control horizon length, task difficulty, feedback structure, and failure-recovery patterns [29, 46]. Here the decisive factor is not what the data contain but whether they preserve temporal dependencies, interaction feedback, and recovery processes.

Taken together, pre-training and mid-training form the first stage of internalizing long-horizon capability. Reasoning-oriented data provide structured problem-solving priors, long-context training supports state maintenance over extended interactions, multimodal training enables persistent observation across heterogeneous evidence, and data mixture strategies determine how these signals are balanced and selected. These latent priors are then elicited, sequenced, and optimized by SFT and RL into reliable agentic behavior.

5.4 Fine-tuning

Fine-tuning converts the latent priors acquired during pre-training and mid-training into controllable long-horizon behavior. Whereas pre-training shapes broad parameter-level capabilities, supervised fine-tuning (SFT) teaches the model how these capabilities should be expressed during interaction: how to decompose goals, call tools, interpret observations, update state, recover from errors, and terminate with a final answer. For long-horizon agents, SFT is therefore process-level imitation over trajectories that contain intermediate reasoning, actions, tool results, and state transitions, rather than instruction-following over isolated input–output pairs. It typically serves as a cold-start stage that constrains the model to valid, task-relevant interaction patterns before any online optimization.

The central issue is how to construct a supervised trajectory distribution that is diverse enough to teach general agent behavior, structured enough to ensure valid tool use, and progressive enough to prepare the model for later stages. We organize this section around three questions: which demonstrations to select and how to mix them, how to stage them through a curriculum, and how to obtain them through distillation.

5.4.1 Instruction Selection and Mixing

Trajectory Quality Over Quantity. Fine-tuning long-horizon agents requires data that expose not only correct answers, but also the intermediate decisions connecting goals, observations, actions, and recovery behaviors across many turns. AgentTuning [794] constructs AgentInstruct, a compact set of high-quality interaction trajectories, and mixes it with general instruction data to strengthen agent skills while preserving general ability. FireAct [54] shows that trajectories generated by stronger models improve agent behavior beyond prompt-only adaptation, and LIMI [711] argues that a small set of strategically curated demonstrations can elicit strong agency-oriented behavior. Rather than cloning full trajectories, ATLaS [81] identifies the *critical steps* of planning, subgoal reasoning, and strategic decision making, and fine-tunes only on them, mitigating expert bias and improving cross-environment generalization. These works reframe agent SFT as an instruction-selection problem: prioritizing trajectory quality, decision-point coverage, and task diversity over raw demonstration count.

Compact Reasoning Demonstrations. A complementary line explains why small trajectory sets can be effective. LIMO [770] elicits complex mathematical reasoning from a strong base model with only 817 curated demonstrations, treating them as cognitive templates over knowledge already encoded during pre-training; s1 [442] builds a 1K-example set under quality, difficulty, and diversity criteria; and LIMR [344] extends the less-is-more observation to reinforcement learning through learning-impact-based sample selection. Unified Data Selection [342] sharpens this idea by selecting samples around high-entropy tokens, which mark critical branching points in reasoning trajectories. This “less-is-more” view is not unchallenged: Ding et al. [122] report that, in an SFT-then-RL pipeline, data scale still governs the attainable performance ceiling while

trajectory difficulty acts as a multiplier, suggesting that selection should maximize decision coverage rather than merely shrink the dataset.

Executable Tool-use Data and Mixture. For tool-using agents, data value further depends on whether actions are syntactically valid, executable, and grounded in observations. CodeAct [656] represents actions as executable Python code, enabling code execution, observation feedback, and self-debugging; APIGen [392] generates function-calling data verified through format, execution, and semantic checks; ToolACE [385] scales synthetic tool-learning data with a self-evolving API pool and multi-agent dialogue generation; and TOUCAN [737] synthesizes large-scale multi-tool, multi-turn trajectories from real-world Model Context Protocol (MCP) environments. Selection also matters at scale: EDGE [844] picks informative multi-turn samples according to guideline effectiveness. Because such corpora remain fragmented across incompatible formats, the Agent Data Protocol [558] introduces a unified schema that standardizes heterogeneous agent datasets into a common trajectory representation; SFT on the resulting cross-domain mixture yields broad gains and avoids the negative transfer induced by single-domain tuning.

5.4.2 Curriculum Learning

Staging by Horizon. Long-horizon trajectories are difficult to imitate directly because errors compound over time and the model must maintain state across many steps before receiving any signal of success or failure. Curriculum learning addresses this by staging the training distribution rather than exposing the model to full-length tasks at once, with the interaction horizon as a natural first axis. ScalingInter-RL, introduced in AgentGym-RL [701], begins with short-horizon exploitation to establish stable foundational policies and then progressively enlarges the interaction budget, mitigating the collapse that plagues naive long-horizon optimization. Studying horizon in isolation, Kim et al. [279] construct controlled task suites that vary only the required action-sequence length and show that horizon itself is a training bottleneck: reducing the horizon first and then extending it stabilizes learning and induces *horizon generalization*, where competence acquired on short goals transfers to unseen longer ones at inference time.

Staging by Difficulty. A second axis orders tasks from easy to hard. E2H Reasoner [469] schedules problems probabilistically from easy to difficult, and shows both empirically and through approximate-policy-iteration bounds that gradually fading out easy tasks prevents overfitting and lets small models acquire reasoning skills that vanilla training cannot reach. Curricula for long-horizon agents can be graded along several dimensions at once, including horizon length, task difficulty, tool complexity, feedback density, and recovery demand, so that a staged cold-start distribution first instills basic interaction competence before later stages optimize decisions under delayed rewards.

5.4.3 Distillation

Because high-quality long-horizon demonstrations are scarce, many fine-tuning pipelines obtain supervision by distilling interaction processes from stronger models, scaffolded agent systems, or executable environments. This is better understood as trajectory distillation rather than answer distillation: the student learns not only final responses, but also planning steps, action formats, observation interpretation, and recovery strategies. We use *distillation* here in its offline sense, where supervision comes from fixed teacher or past-model trajectories consumed as static SFT data, in contrast to the on-policy distillation of Section 5.6 and the closed-loop self-evolution of Section 5.7.

Strong-to-weak Distillation. *Strong-to-weak distillation* transfers agentic behavior from powerful teacher models or expensive agent systems into smaller, deployable students, reusing costly execution as supervised traces. Because agent competence lives in full reason-act-observe trajectories, recent work supervises structure rather than final answers. Agent Distillation [269] transfers retrieval and code-execution trajectories from a large teacher into sub-3B students, using a first-thought prefix and self-consistent action generation to keep tiny agents competitive with far larger chain-of-thought-distilled baselines. Structured Agent Distillation [376] segments each trajectory into [REASON] and [ACT] spans with segment-specific losses, preserving both reasoning fidelity and action validity under compression. Pushing the teacher side further, AgentArk [410] distills a

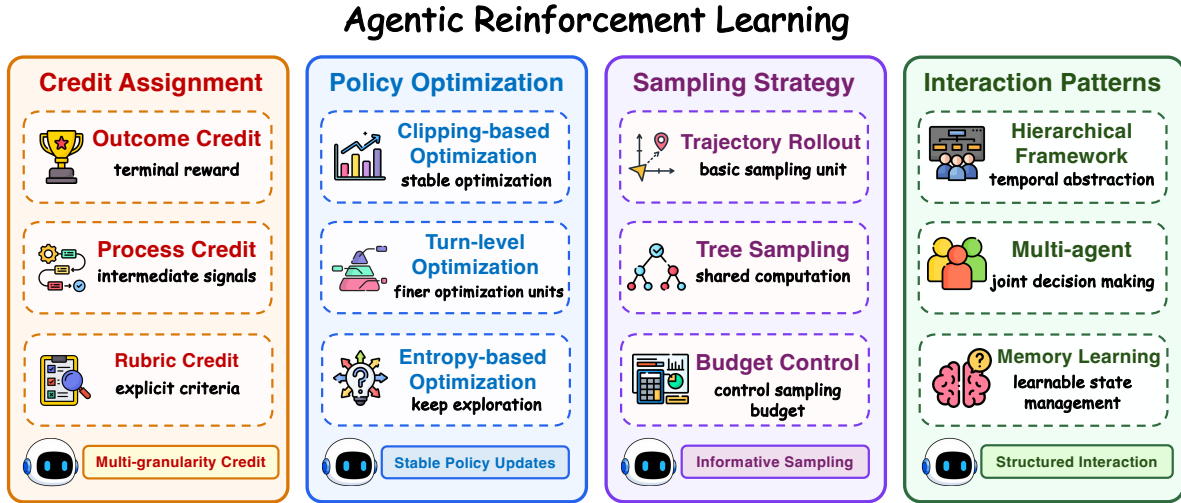


Figure 12 Agentic reinforcement learning through four key questions. These questions correspond to the four components discussed in Section 5.5: credit assignment, policy optimization, sampling strategy and interaction structure.

multi-agent debate system into a single model, turning expensive test-time interaction into implicit self-correction ability. Across these methods, effective distillation aligns supervision with trajectory structure rather than compressing outputs alone.

Self-distillation and Reflective Self-improvement. *Self-distillation* removes the external teacher: the model generates its own trajectories, filters or reflects on them, and trains back on the curated result. Since many long-horizon failures are local, such as choosing a wrong tool, misreading an observation, or failing to recover after an execution error, the central design choice is how to turn raw rollouts into clean, corrective supervision. Watch Every Step [719] explores along expert trajectories, collects step-level process feedback, and learns from contrastive action pairs. Agent-R [789] uses Monte Carlo Tree Search to locate the first erroneous step in a failed rollout and splices it onto an adjacent correct path, so that iterative fine-tuning teaches timely on-the-fly correction rather than end-of-episode fixes. Reflection enriches the signal further: RESD [845] converts failed rollouts into retrospective diagnoses and a reusable “playbook” of lessons, while HERO [374] turns each observation into a turn-level hindsight reflection used to re-score the student’s own actions. These reflection-driven variants shade into the on-policy self-distillation of Section 5.6; used offline, their filtered traces still supply denser process supervision than final-answer imitation.

Both regimes share a lesson: distillation for long-horizon agents must preserve *process* structure, whether transferring capability from strong teachers or recycling an agent’s own history into cleaner supervision. Both can also amplify biases and environment-specific habits, so execution-based verification, semantic filtering, and mixture with general instruction data remain essential for robust, transferable behavior.

Overall, fine-tuning determines how long-horizon priors surface as interactive behavior: instruction selection defines the supervised behavior distribution, curriculum learning stages it by horizon and difficulty, and distillation transfers complex interaction processes from stronger or self-generated sources. It improves controllability, tool-use validity, state tracking, and recovery, yet stays bounded by the coverage of supervised demonstrations, leaving decisions under delayed rewards and compounding errors to later online optimization.

5.5 Agentic Reinforcement Learning

Compared with supervised fine-tuning, Agentic Reinforcement Learning (Agentic RL) further optimizes the policy against feedback from the agent’s own interaction. This extends the optimization targets of LLMs from single-turn responses to long-horizon interaction trajectories. As illustrated in Figure 12, the core challenges involve four aspects: sampling informative multi-turn trajectories, deriving multi-granularity credit signals

from sparse feedback, integrating these signals into stable policy updates, and organizing training interaction patterns such as hierarchy, tool use, multi-agent collaboration, and memory. In the following subsections, we elaborate on credit assignment, policy optimization, and sampling methods in Agentic RL, as well as several unique interaction patterns of Agentic RL. Representative works for each part are shown in Table 6.

5.5.1 Credit Assignment

Credit assignment is one of the core challenges for Agentic RL. It concerns how to attribute credit to heterogeneous decision units when an agent completes long-horizon tasks. This section reviews credit assignment from three perspectives: outcome credit assignment, which scores only the final result; process credit assignment, which shapes the intermediate decision process; and rubric credit assignment, which expresses quality through explicit semantic criteria.

Outcome Credit Assignment. Outcome credit assignment is the most direct form of credit assignment, where the reward is determined only by the final result, such as exact match, F1, task success, final-state checks, unit-test results, or retrieved-answer correctness. Its appeal lies in being automatable, cheap to verify, and easy to scale to large online rollouts. This paradigm was first validated by verifiable terminal rewards in mathematical and code reasoning [112, 345, 533]. It was subsequently carried into interactive settings by a line of web and information-seeking agents led by Search-R1, which reuse task success, environment-state checks, or retrieved-answer correctness as terminal rewards across long-horizon application environments [60, 256, 261, 681, 871]. However, this form of credit is coarse-grained. A single terminal score is shared across the entire trajectory, conflating key exploration, redundant actions, and early mistakes into one learning signal. As a result, the policy struggles to discriminate the quality of individual local decisions.

Process Credit Assignment. Process supervision predates agentic RL in mathematical reasoning, where process reward models (PRMs) score intermediate steps rather than only final answers [363, 643]; reinforcement learning with verifiable rewards then scaled outcome-based optimization for reasoning [112, 533]. Agentic credit assignment inherits this lineage and extends it to multi-step, tool-using trajectories. Process credit assignment extends the evaluation target from the final answer to the intermediate process. Existing methods follow two complementary routes. The first decomposes the quality of an individual decision: a group of tool-use-oriented methods represented by ToolRL break tool invocation into separately scoreable dimensions, including output format, tool selection, and parameter names and values [129, 487, 834]. The second distributes denser reward across the interaction process rather than scoring a single decision, spanning a spectrum of granularity: some works such as AdaTIR and AgentPRM assign reward to intermediate units of the trajectory rather than to its endpoint alone [77, 150, 318, 554, 682, 702, 837]. Among them, some methods fold process signals such as interaction cost and query quality into trajectory-level rewards, while others attach explicit step- or turn-level credit via process reward models or advantage redistribution. Across both routes, the shared goal is not merely to make rewards denser, but to align credit signals with the true decision structure of the agent.

Rubric Credit Assignment. Rubric credit assignment expresses answer or trajectory quality through explicit, multi-dimensional semantic criteria (e.g., factuality, completeness, reasoning soundness, evidence grounding, and safety) when a single verifiable metric is insufficient; a recent paper separates such rubrics from reward models, verifiable rewards, and LLM-as-a-judge [386]. Methods differ along two axes: how rubrics are constructed, and how they are turned into credit.

On construction, rubrics are produced along an increasingly automated spectrum: (1) strong LLMs or human experts author checklists used directly as scoring criteria or as training rewards [534], as in Rubrics as Rewards and the rubric bank of Reinforcement Learning with Rubric Anchors [192, 240]; (2) rubrics are induced from preference contrasts, where OpenRubrics extracts rules and implicit principles filtered by preference-label consistency and Chasing the Tail targets the high-reward tail to curb over-optimization [384, 819]; and (3) rubrics are made context-grounded and online, as when Agentic Rubrics lets an expert agent inspect a repository before generating criteria and DR Tulu co-evolves positive and negative rubrics with the policy during deep-research training [500, 532].

On conversion, rubrics become credit signals (1) as direct RL rewards, aggregated by weights or an LLM judge and optionally decomposed into fine-grained signals such as answer correctness, step validity, and premise

Table 6 Representative works in agentic reinforcement learning. Rows are grouped by the four parts of Section 5.5: credit assignment, policy optimization, sampling strategy, and interaction patterns; **Type** names the mechanism family, **Key Mechanism** gives its one-line idea, and **Resource** links to a public code repository where available.

Method	Type	Key Mechanism	Resource
Credit Assignment			
DeepSeekMath [533]	Outcome Reward	Terminal reward for math reasoning	 GitHub
Search-R1 [261]	Outcome Reward	Answer correctness as terminal reward	 GitHub
DeepRetrieval [256]	Outcome Reward	Retrieval-metric terminal reward	 GitHub
ToolRL [487]	Process Reward	Decomposes tool calls into scored aspects	 GitHub
Tool-Star [129]	Process Reward	Monitors answer and tool usage	 GitHub
CriticSearch [837]	Process Reward	Retrospective critic for fine-grained credit	—
RaR [192]	Rubric Reward	Expert checklists used directly as rewards	—
OpenRubrics [384]	Rubric Reward	Rubrics induced from preference contrasts	—
DR Tulu [532]	Rubric Reward	Co-evolving rubrics in deep research	 GitHub
RuscaRL [887]	Rubric Reward	Rubrics as decaying exploration scaffolds	 GitHub
Policy Optimization			
REINFORCE++ [222]	Ratio Clipping	Global advantage normalization	 GitHub
DAPO [780]	Ratio Clipping	Decoupled clip-higher/lower	 GitHub
Dr.GRPO [389]	Ratio Clipping	Removes length/std normalization bias	 GitHub
GSPO [865]	Ratio Clipping	Sequence-level ratio and clipping	—
Turn-PPO [321]	Credit Granularity	Turn-level advantage estimation	—
StepPO [628]	Credit Granularity	Step-aligned advantage	 GitHub
GiGPO [156]	Credit Granularity	Episode- and step-level advantage	 GitHub
Clip-Cov [103]	Entropy Regularization	Covariance-gated updates control entropy	 GitHub
CURE [330]	Entropy Regularization	Critical-token-guided re-concatenation	 GitHub
EPO [732]	Entropy Regularization	Trajectory entropy regularization	 GitHub
Sampling Strategy			
R1-searcher [553]	Trajectory Rollout	On-policy search-agent trajectory rollout	 GitHub
WebDancer [689]	Trajectory Rollout	Web-search trajectory pipeline	 GitHub
WebRL [484]	Trajectory Rollout	Self-evolving curriculum web trajectories	 GitHub
Tree of Thoughts [760]	Tree Sampling	Organizes thoughts as a search tree	 GitHub
RAP [204]	Tree Sampling	Planning as tree search with world model	 GitHub
LATS [875]	Tree Sampling	Language-agent tree search over actions	 GitHub
TreePO [348]	Tree Sampling	Reuses inference compute across tree paths	 GitHub
LiteResearcher [334]	Budget-Aware Pruning	Difficulty/pass-rate-based pruning	 GitHub
TCOD [636]	Budget-Aware Pruning	Trajectory-depth curriculum	 GitHub
Interaction Patterns			
GLIDER [232]	Hierarchical Framework	Grounds LLMs as decision-making agents	 GitHub
SkillRL [705]	Hierarchical Framework	Evolves a skill library from failures	 GitHub
THOR [52]	Hierarchical Framework	Joint episode- and step-level optimization	 GitHub
AT-GRPO [861]	Multi-agent Framework	Per-agent, per-turn advantage grouping	 GitHub
MATPO [439]	Multi-agent Framework	Multiple roles in one LLM over joint rollouts	 GitHub
SPIRAL [371]	Multi-agent Framework	Role-conditioned advantages	 GitHub
Memory-R1 [743]	Memory-state Learning	Separates memory manager and answer agent	 GitHub
Memory-as-Action [846]	Memory-state Learning	Working-memory editing as policy action	 GitHub
Agent Workflow Memory [674]	Memory-state Learning	Induces reusable workflows	 GitHub
Agent Lightning [409]	Memory-state Learning	Traces workflows into trainable samples	 GitHub

utilization [164, 192]; (2) as supervision for reward models or verifiers, including Prometheus, the OpenRubrics reward model, and the interpretable scorer R3 [19, 278, 384]; and (3) as guidance for policy behavior rather than only final scoring, as in Rubric-Scaffolded RL, which injects rubrics as decaying exploration scaffolds during rollout and as judge references during exploitation [887]. Rubric credit is especially useful in Agentic RL, where context dependence, tool interaction, and side effects are easily obscured by a single composite reward; its open challenges remain rubric quality, judge bias, criterion weighting, and reward hacking.

5.5.2 Policy Optimization

Policy optimization is where credit signals are turned into parameter updates. In Agentic RL, this step is difficult because the updates must be computed from multi-turn trajectories that mix tool calls, environmental observations, GUI states, and delayed terminal rewards [261, 321, 628, 681]. We organize the discussion around three practical questions: how to make sparse-reward updates stable, which interaction unit should be optimized, and how to preserve exploration over long trajectories.

Clipping-based Policy Optimization. The first problem is how to make PPO- or GRPO-style RLVR reliable when trajectories are long and rewards are sparse or binary [460, 524, 525, 533]. The main difficulty is the mismatch between noisy group statistics and long sequence-level gradients. This mismatch appears as length bias, local normalization bias, homogeneous all-correct or all-wrong groups, unstable token-level importance ratios, and reward-scale drift. Existing methods fall into two broad categories. One category makes group-relative updates less biased through normalization, clipping, and sampling design; representative examples include REINFORCE++, DAPO, Dr.GRPO, CISPO, and GPPO [222, 389, 435, 565, 780]. GSPO defines the importance ratio and applies clipping at the sequence level rather than per token [865]. The other category changes the estimation unit or objective to reduce variance and constrain policy drift, with representative examples such as BNPO [95, 205, 391, 710].

Turn-level Policy Optimization. The second problem is the granularity of optimization. In agent-environment interaction, the meaningful decision unit is often a trajectory, turn, step, subgoal, or branch rather than a single response. Early agentic RL systems therefore lift optimization from response-level training to full interaction trajectories, as in web and search agents such as WebAgent-R1 [261, 681]. A more fine-grained line then aligns advantages with explicit segments, steps, or turns, represented by SegPO, StepPO, and Turn-PPO [321, 628, 901]. More recent methods make the comparison set itself more local and informative. Some anchor comparisons to shared states or trajectory structures, as in GiGPO and related methods [156, 212, 318, 349]. Others use shared branches, subgoal segments, or hindsight information to ask which local choice changed the downstream trajectory, as in Tree-GRPO and its extensions [250, 478, 582, 687].

Entropy-based Policy Optimization. The third problem is exploration. Long-horizon RL can quickly over-reinforce early successful patterns, causing entropy collapse and reducing the diversity of future trajectories. Recent work therefore treats entropy as a decision- or trajectory-level control variable rather than only a token-level regularizer. One group modifies the update rule to avoid excessive probability concentration: Clip-Cov and KL-Cov gate over-concentrating updates, and OPEFO balances entropy-increasing and entropy-decreasing gradients [103, 725]. Another group injects diversity at critical decision points, for example CURE, which re-concatenates rollouts around critical (high-entropy) tokens to prevent entropy collapse [330]. Other methods, such as AEnt and AER, attempt to stabilize policy entropy within a bounded interval to alleviate entropy collapse [536, 830]. For multi-turn agents, the same idea becomes trajectory-level exploration control: EPO regularizes trajectory entropy under sparse rewards [732]. ARPO and AEPO branch from high-entropy positions after tool feedback to trade off rollout diversity against gradient distortion [128, 130].

5.5.3 Sampling Strategy

In long-horizon agent tasks, the sampling target is multi-turn interaction trajectories encompassing tool calls, environmental observations, memory operations and final rewards [808, 821]. Unlike credit assignment and policy optimization, the sampling strategy decides which trajectories are generated in the first place. We organize this from three angles: trajectory-level rollout that fixes the basic sampling unit, tree-structured

sampling that shares computation and produces relative signals, and budget-aware pruning that keeps sampling informative under a fixed cost.

Trajectory-level Rollout. The basic sampling unit in Agentic RL is a multi-step agent-environment trajectory: at each step the model acts on observations, history, and task goals, and the environment returns new observations and reshapes the future state distribution. Early systems represented by ReAct established this reasoning-acting-observing loop, together with tool use, verbal reflection, and skill reuse, mostly through prompting rather than training [523, 544, 630, 761]. Subsequent RL work turned such loops into the rollout unit: a line of information-seeking agents led by Search-R1 folds query generation, evidence acquisition, and stopping decisions into on-policy trajectories [256, 261, 553, 871], while web and tool agents organize browser operations or executable tool composition as multi-turn online trajectories [484, 653, 681, 701, 834]. At larger scale, deep-research systems represented by WebDancer and the WebSailor family build end-to-end pipelines that combine synthetic data, tool-based web environments, and trajectory-level rollout [326, 327, 689]. On top of such pipelines, WebResearcher, ReSum, and WebWeaver further address long-context research, context summarization, and evidence structuring during sampling [356, 490, 693].

Tree-structured Sampling. Independent full-trajectory rollout is expensive and, when trajectories are homogeneous, yields weak relative learning signals. Tree-structured sampling addresses both problems by sharing prefixes across rollouts and contrasting sibling branches. The idea originates from organizing thoughts or actions as trees during reasoning and planning, as in Tree of Thoughts, RAP, and LATS [204, 760, 875], and is brought into RL training by Tree-GRPO, which compares sibling branches under a fixed token or tool-call budget [250]. A subsequent group refines where and how the tree expands: AT2PO, TreePO, and PORTool guide expansion toward uncertain turns, reuse inference computation, or organize shared and divergent tool-use paths [348, 687, 900], while branching methods such as BranPO and Tool-Light branch selectively from high-entropy positions, late critical decisions, or otherwise informative states to maximize rollout contrast [76, 860].

Budget-aware and Pruning Strategies. A third angle keeps sampling informative under a fixed rollout budget, either by pruning or reshaping the trajectory distribution or by allocating the branching budget adaptively. One line prunes or reshapes the trajectory distribution: representative mechanisms include experience reuse and confidence-based filtering in Tool-R1 and WebRL, difficulty- or pass-rate-based pruning in LiteResearcher, and trajectory-depth curricula in TCOD [334, 484, 636, 834]. These strategies discard low-signal rollouts, concentrate budget on contrastive or appropriately difficult cases, and bound trajectory length. They reduce the cost of producing each informative training sample in long-horizon settings. Another research approach makes branching itself budget-aware: ARPO triggers branch sampling only at high-entropy tool-call steps, reducing the tool-use budget of trajectory-level RL, while AEPO adaptively allocates the budget between global and branch sampling [128, 130].

5.5.4 Interaction Patterns

Interaction patterns concern how to organize optimizable decision-making structures during training. We discuss them from three perspectives: hierarchical interaction frameworks, multi-agent interaction frameworks, and memory-state learning.

Hierarchical Interaction Frameworks. Hierarchical frameworks exploit the temporal abstraction that naturally exists in long-horizon tasks, and split along two threads. The first is explicit planner-executor decomposition, where the policy is separated into a high-level module that sets subgoals and a low-level module that executes them, so that subgoal boundaries become new optimization units. Representative instances include HiPER, GLIDER, and STEP-HRL, which differ mainly in whether the low-level layer is a task-agnostic skill set or a progress-compressed executor [232, 478, 863]. The second thread pushes hierarchy toward experience abstraction and capability reuse: building on the observation that procedural skills in agentic exploration originally rely on external skill libraries and prompting. SkillRL evolves a skill library from validation failures during RL and Skill0 internalizes skills into model parameters via in-context agentic RL [405, 705]. Analogous high-low decompositions appear in tool-augmented reasoning, where THOR jointly optimizes episode-level problem solving and step-level tool or code operations [52].

Multi-agent Frameworks. Multi-agent frameworks extend the training target from a single policy to a joint decision-making system. Their central difficulties are role non-stationarity, enlarged joint action spaces, and credit assignment under shared rewards. One thread formalizes LLM collaboration as cooperative MARL in Dec-POMDP settings and adapts group-relative training accordingly. MAGRPO uses centralized group-relative advantages with decentralized execution, while Stronger-MAS (AT-GRPO) restores variance reduction by grouping samples per agent and per turn [382, 861]. To assign credit across heterogeneous roles, M-GRPO computes separate group-relative advantages for a planner and its tool-executing sub-agents under decoupled cross-server training, whereas MATPO trains the planner and worker within a single LLM by normalizing over their joint rollouts [218, 439]. Dr.MAMR instead pushes credit to the step level with a Shapley-inspired causal-influence estimate to counter lazy-agent collapse [850]. A second thread covers self-play, debate, and role specialization, which inherit classical MARL principles such as centralized training with decentralized execution, counterfactual baselines, value decomposition, and MAPPO [396, 506, 776]. Within this thread, recent systems instantiate complementary System 1/System 2 policies for deep research, variance-reduced group-relative training with evolving memory, and role-conditioned advantage estimation in zero-sum language games [56, 201, 371]. Multi-agent debate adds a further variant, where competitive interaction provides additional preference, refutation, and verification signals for the final answer [139, 358].

Memory-state Learning. Memory-state learning turns runtime structures that are usually fixed by an external harness, including context, workflows, long-term memory, tool-use histories, and reusable experience, into learnable policies. This shifts Agentic RL from optimizing isolated LLM calls to training state management over long horizons. One thread makes memory operations themselves a learning target: Memory-R2 observes that memory writes place trajectories in different effective environments and uses LoGo-GRPO to combine a global objective with local comparisons from the same memory state, Memory-R1 separates a Memory Manager from an Answer Agent, and Memory as Action treats working-memory editing as an explicit policy action handled by Dynamic Context Policy Optimization [742, 743, 846]. A parallel thread lifts the same idea to the organization of the task process: Agent Workflow Memory induces reusable workflows from successful tasks, FlowSteer optimizes a workflow director with structural and task-level rewards, and Agent Lightning offers a system-level route that traces existing workflows as spans and transitions and converts them into trainable samples [409, 674, 823].

5.6 On-Policy Distillation

Alongside the emergence of Agentic RL, recent research has increasingly explored alternative training paradigms tailored to long-horizon trajectory optimization. As interaction horizons grow, agent training becomes increasingly challenged by distribution shift, error accumulation, and sparse, delayed supervision [377]. Existing training paradigms only partially address these challenges. Off-policy imitation, such as behavioral cloning and offline knowledge distillation [609], relies on static expert trajectories and therefore fails to expose agents to their own induced state distributions. Reinforcement learning [94], in contrast, optimizes on on-policy trajectories but often suffers from sparse rewards, costly exploration, and poor sample efficiency.

On-Policy Distillation (OPD), first formalized for language models by GKD [3], combines the high-quality supervision of distillation with the state coverage of on-policy learning [309, 635]. Instead of learning solely from fixed expert demonstrations or sparse outcome rewards, OPD trains the student on trajectories generated by its current policy and provides additional supervisory signals on these self-generated trajectories. As a result, OPD exposes the student to self-induced state distributions while providing richer supervision than sparse outcome rewards. This delimits OPD from its neighbors: unlike the offline trajectory distillation of Section 5.4, which learns from a fixed set of teacher or past-model traces, OPD draws supervision on the student’s own current rollouts; unlike the self-evolution of Section 5.7, it keeps the task set and environment fixed and only densifies the supervision signal. These properties make OPD particularly attractive for long-horizon agents, where robustness to distribution shift, recovery from compounding errors, and access to dense supervisory signals are all critical for effective learning [697, 838].

5.6.1 Teacher-guided OPD

In teacher-guided OPD, one or more teacher models are queried on states visited by the student policy to provide supervisory signals for the generated trajectories [309, 312, 635, 697]. Unlike conventional offline

distillation that relies on fixed expert demonstrations, teacher-guided OPD generates supervision adaptively online according to the student’s evolving trajectories, allowing teachers to provide guidance tailored to the student’s current behaviors and mistakes. Existing methods can be characterized by two complementary design choices: how supervision is organized over trajectories and how teacher signals are constructed.

Supervision Scheduling. A first question is how teacher signals should be organized and delivered throughout long interaction trajectories. Representative methods such as DAgger-LLM introduce turn-level interpolation and step-wise supervision to provide dense corrections on states visited by the student policy [309, 873]. Other approaches further incorporate adaptive curricula and preference-based guidance, progressively adjusting the timing, granularity, and difficulty of supervision during training [313, 636, 835]. What emerges is that teacher-guided OPD hinges not only on the quality of teacher signals, but also on when and at what resolution they are injected during multi-turn interactions.

Teacher Signals. A second question is how to construct richer and more informative supervision for on-policy distillation. A series of methods represented by MAD-OPD leverage multiple agents or agent evolvers to aggregate complementary expertise and generate more reliable supervision than a single teacher policy [635, 809]. Along a related line, KAT-Coder-V2 [312] exploits specialist agents with complementary capabilities and distills their collective knowledge into a unified policy. Retrieval-augmented approaches such as LiteGUI [697] instead enrich teacher supervision with oracle trajectories and dynamically retrieved references, providing structured guidance in multi-solution interaction environments. Across these efforts, the recurring lesson is that supervision improves when teacher signals are made more diverse, complementary, and context-aware.

5.6.2 Self-improving OPD

Self-improving OPD removes direct reliance on external teacher models and instead constructs its own supervisory signals from auxiliary information available during training. Such information generally arises from two complementary sources: privileged information that is accessible only during training [838] and adaptive feedback generated through repeated interactions with the environment [216, 242, 845].

Privileged Information. One source of self-supervision is information that is accessible during training but unavailable at deployment time. Some methods leverage privileged annotations or modalities, such as outcome-aware information and fine-grained visual details, to provide dense supervision beyond what the agent’s own observations reveal [788]. Others derive privileged information from the interaction trajectories themselves, treating intermediate construction processes, latent skills, or other trajectory-level abstractions as additional supervision that exposes the hidden structure of successful behaviors [631, 838]. Augmenting sparse interaction signals with training-only information in this way helps the student acquire more robust and reusable long-horizon behaviors.

Environment Feedback. The other source of self-supervision is the interaction outcomes and verifier signals returned by the environment. Some methods transform sparse environmental feedback, such as rewards, textual critiques, runtime errors, and verifier outputs, into dense self-supervision by conditioning the policy on feedback-informed predictions and distilling them back into the model [216, 242]. A complementary view treats failures as supervision, generating retrospective reflections and accumulating reusable lessons that guide future interactions [757, 845]. Beyond obtaining feedback, a further line studies how to use it well during distillation, for example by emphasizing informative signals, learning feedback-aware representations, and mitigating the instability caused by noisy or stale supervision [72, 161, 735, 782]. The decisive factor is therefore not whether environmental feedback is available, but how it is transformed, accumulated, and reused across long-horizon training.

5.7 Self-evolution

This subsection discusses self-evolution as a training mechanism for internalizing experience generated by the agent itself. The preceding stages mainly rely on prepared data, task sets, environment collections, or trajectories. Self-evolution adds a feedback loop: the current policy produces new attempts, failures,

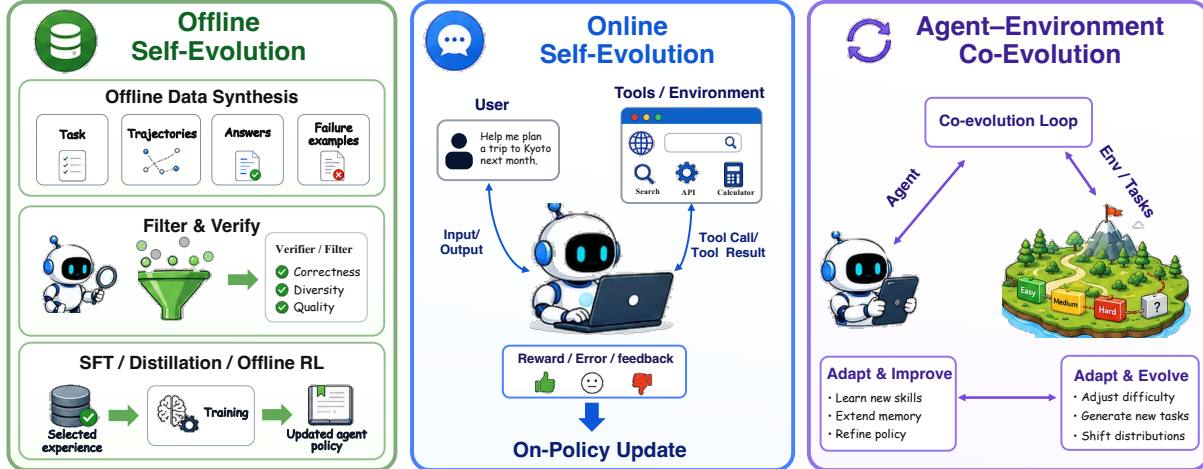


Figure 13 Three modes of self-evolution for long-horizon agents. *Offline self-evolution* learns from selected self-generated experience through a generate-filter-train pipeline; *online self-evolution* learns from live environment interaction on the agent’s own induced state distribution; and *agent-environment co-evolution* jointly adapts the agent and its task/environment distribution to maintain a learnable capability boundary.

and feedback in its environment, and training writes selected records back into the model. This is what distinguishes it from the two distillation stages above: whereas fine-tuning distillation (Section 5.4) and on-policy distillation (Section 5.6) both consume a supervision source that is held fixed within a training round, self-evolution treats the data, tasks, and even the environment as objects that the loop itself updates. This has recently grown into a fast-moving research area, and dedicated papers now catalog how agents can autonomously evolve their models, memories, tools, and workflows toward lifelong adaptation [147, 165]. As illustrated in Figure 13, we organize this line of work into three forms: offline self-evolution, online self-evolution, and agent-environment co-evolution.

5.7.1 Offline Self-evolution

Offline bootstrapped self-evolution typically follows a generate-filter-train pattern. The model generates candidate tasks, answers, reasoning traces, code patches, tool calls, or interaction trajectories; verifiers, tests, reward models, teacher models, or self-evaluation mechanisms select successful samples, informative failures, and correctable fragments; training then turns these samples into SFT, distillation, preference optimization, or offline RL objectives. STaR bootstraps reasoning from successful rationales [793], and subsequent work extends the same idea to self-edits, tool-augmented reasoning, multi-agent demonstrations, and code-execution feedback [310, 517, 859, 906]. More recent work emphasizes how to turn search processes and failures into stable training assets. SOAR alternates evolutionary search with hindsight learning in program synthesis, converting search attempts into fine-tuning pairs; SAMULE synthesizes reflection data at the trajectory, task, and cross-task levels, and trains a retrospective model from failure analysis [173, 481]. A parallel search-based route runs the generate-filter-train loop for several rounds: rStar-Math uses Monte Carlo tree search to synthesize step-by-step verified reasoning trajectories and iteratively co-evolves a policy model together with a process preference model, so that each round’s verified rollouts become the next round’s supervision [191]. This route is attractive because data can be audited, denoised, and relabeled before an update, making it suitable for cold start and capability consolidation. Its main risk is that candidate data still largely comes from the existing model distribution, which can reinforce known behaviors while doing little to discover out-of-distribution strategies.

5.7.2 Online Self-evolution

Online interactive self-evolution interleaves experience collection with policy updates, training the model on state distributions induced by its own behavior. For long-horizon agents, difficult states are often not static

benchmark inputs; they arise from early decisions, tool side effects, recovery attempts, and environment state changes. Recent work therefore incorporates web, code, search, tool-use, and multi-agent game environments into training, using rollout, environment feedback, and reinforcement learning to continually update the policy [484, 672, 701, 799, 871]. A complementary line reduces external data seeds by using self-play, challenger-solver, or proposer-solver structures to create training pressure, so that the model obtains learning signals by proposing tasks, solving them, and verifying outcomes [233, 371, 854]. Recent agentic instantiations push this toward tool use and multi-step reasoning: Agent0 co-evolves a curriculum agent and a tool-using executor from the same base model with zero external data, so that a stronger executor pressures the curriculum agent to propose harder, tool-aware tasks [704]. Compared with offline bootstrapping, online self-evolution is more likely to move beyond the ceiling of existing data, but it is also more sensitive to the quality of environments and verifiers: delayed rewards, credit-assignment errors, policy collapse, verifier loopholes, and exploration cost are all amplified in long rollouts.

5.7.3 Agent-Environment Co-evolution

Agent-environment co-evolution further treats the training environment as an object of update. A fixed environment can quickly drift away from the model’s capability boundary: if it is too easy, it only reinforces existing strategies; if it is too hard, it yields little usable learning signal. A first line makes the environment itself tunable to the policy: Environment Tuning uses curricula, environment augmentation, and progress feedback to mitigate cold start and sparse rewards in tool-use training, while RLVE and GenEnv align the difficulty of verifiable or generated environments with the policy’s current capability, and Agent-World closes the loop by discovering environment-task pairs, exposing capability gaps through dynamic task synthesis, and improving the policy through multi-environment RL [131, 195, 401, 798]. In multi-agent settings, the environment may also be constituted by the policies of other agents; CoMAS uses interaction rewards to jointly update multiple agent policies, showing that peer interaction can serve as a training signal for self-evolution [738]. Socratic-Zero pushes this idea in a data-free direction, letting a Teacher, a Solver, and a Generator co-evolve so that the Teacher continually crafts questions targeting the Solver’s weaknesses while the Generator distills its curriculum-design strategy, effectively turning the task distribution itself into a co-adapting counterpart [648]. From this perspective, the environment is not merely a source of samples, but a dynamic curriculum, feedback generator, and probe of the capability boundary: improvements in the agent change future reachable states and failure modes, while environment updates reshape the next round of exploration and optimization.

Overall, training self-evolution is not unconstrained self-training, but an experience loop jointly constrained by environments, verifiers, and optimization. External memories, skill libraries, and workflow traces can serve as intermediate forms of accumulated experience, but they constitute training self-evolution only when their stable patterns are absorbed through distillation, fine-tuning, reinforcement learning, or trainable modules [705]. The risks of this loop arise from its bootstrapping structure: self-generated data may shrink the distribution, weak verifiers may induce reward hacking, and dynamic curricula may reinforce flawed capabilities. Long-horizon agent self-evolution therefore requires strong verifiers, process-level feedback, and a combination of offline audit with online exploration.

6 Application: Long-Horizon Agents in Practice

The preceding sections examined the general ingredients of long-horizon agency, including persistent state, feedback-driven adaptation, and mechanisms for maintaining alignment over extended trajectories. We now turn to the settings in which these requirements arise in practice. Long-horizon agents operate across a wide range of environments, from code repositories and web documents to graphical interfaces, multimodal streams, and open-ended research workflows. These settings differ not only in task duration, but also in the interface through which an agent observes, acts, and receives feedback from the world, which in turn shapes what information is available during execution, what errors can be detected locally, and what capabilities the agent must add to compensate for what the environment does not provide. Therefore, we organize this section by the interface between the agent and its environment rather than by overall difficulty.

Figure 14 groups representative applications into five broad categories, distinguished by the interface through which each agent observes and acts:

Five Representative Applications of Long-Horizon Agents

1. **Software Engineering Agents** (Section 6.1): act over code repositories and execution environments, where edits can often be checked through tests, compilers, and runtime outputs.
2. **Information-seeking Agents** (Section 6.2): interact with web documents, search engines, and knowledge sources, where the challenge is to gather and synthesize evidence without a direct execution oracle.
3. **Computer-use Agents** (Section 6.3): operate graphical interfaces through screenshots, accessibility trees, and low-level actions, inferring and manipulating partially observable interface state.
4. **Multimodal Agents** (Section 6.4): reason over long visual, auditory, and textual streams, where relevant information is distributed across time and modalities.
5. **General-purpose Agents** (Section 6.5): combine several such interfaces within a single workflow, drawing on heterogeneous tools, permissions, and external services.

For each category, we ask three questions:

- What information and actions does the interface expose?
- What kinds of state and feedback must persist across a long trajectory?
- What capabilities must the agent or harness add to compensate for what the environment does not provide directly?

The following subsections address these questions by discussing the core capabilities required in each domain and representative systems that instantiate them.

6.1 Software Engineering

Software engineering agents are one of the most foundational and representative classes of long-horizon agents. The agent interacts with its environment through code: it acts by writing and running code, obtains feedback from how that code behaves, and uses this feedback to advance its task. A defining feature of this paradigm is unusually strong executable feedback: through tests, compilers, and runtime traces, the environment can often tell the agent automatically whether an action has succeeded, which is what distinguishes software engineering from many other long-horizon tasks. These signals nonetheless remain incomplete proxies for intended behavior: passing tests and clean compiles cannot always certify that the real issue is resolved, especially where hidden requirements, security, maintainability, and regression risk are concerned [499, 671, 801].

Turning this feedback into reliable long-horizon behavior, however, takes more than the signal itself. We organize long-horizon software engineering around three capabilities that an agent must add. The agent first has to work out *where* to act, building a usable picture of a codebase too large to fit in its context window; we call this *repository grounding*. It then has to keep the overall objective intact as it proceeds, ordering its edits and tracking progress rather than chasing one fix at a time, which is *workflow-level planning*. Finally, it has to use the feedback it receives to actually correct its mistakes, the task of *feedback-driven repair*. Together, these capabilities mark the step up from fixing isolated bugs to repository-scale work [124, 749], including implementation, migration, and pull-request preparation; commercial systems such as Devin² brought such end-to-end long-horizon coding agents into production. The next subsections examine each capability in turn.

6.1.1 Repository Grounding

Before any code is written, the agent must decide *where* to act. A natural-language issue typically says little about which files, dependencies, or project conventions are relevant, so long-horizon systems need a way to progressively construct and reuse a working model of the repository.

²Devin (Cognition): <https://devin.ai>.

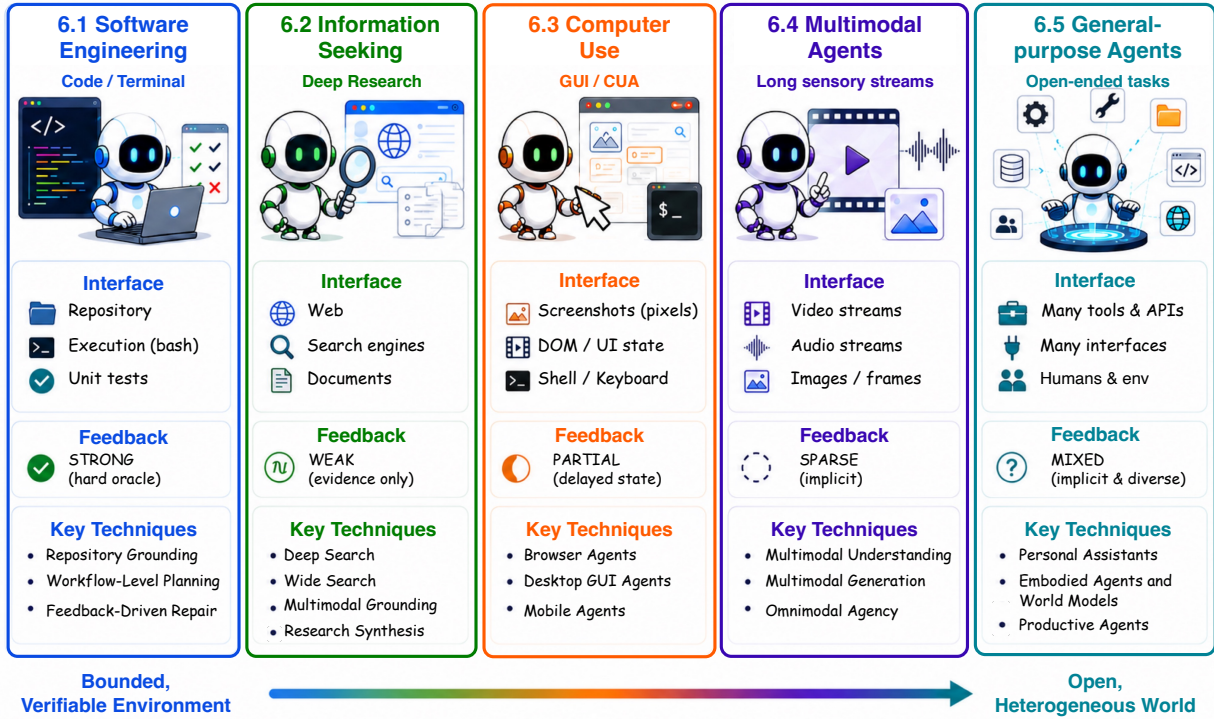


Figure 14 Representative long-horizon agent applications grouped by the structure of the agent-environment interface. Different interfaces expose different forms of observation, action, state, and feedback, leading to distinct capability requirements for long-horizon behavior.

Existing designs largely follow three strategies. The first is to improve *interactive navigation*: SWE-agent, for example, exposes search, navigation, and editing commands through an agent-computer interface tailored to LLM use [747], while structure-aware systems such as AutoCodeRover search over a program’s class and method hierarchy rather than over raw text [843]. The second is to maintain a *persistent workspace*, so that file, shell, and browser state can accumulate across steps rather than being repeatedly reconstructed; OpenHands is representative of this approach [657]. The third is to externalize repository structure in compact, reusable artifacts, such as Aider’s repository map,³ IDE-level project rules,⁴ and AGENTS.md-style instructions.⁵ Although implemented differently, these approaches share a purpose: turning a large, partially observed codebase into stable task-relevant state that can guide later decisions. Related work extends this idea by constructing executable environments from arbitrary repositories [248], or by training models to internalize more repository-level structure directly [46].

6.1.2 Workflow-level Planning

Knowing where to act does not by itself determine how to sequence the work. Feature development, dependency migration, experiment reproduction, and pull-request preparation require the agent to decompose an objective, track intermediate progress, manage artifacts, and decide when available evidence is sufficient to stop [117, 142, 260, 317, 437]. The challenge here is to preserve the relationship between each local action and a global engineering objective.

Long-horizon systems increasingly externalize this relationship through explicit workflow state. One line of work structures execution around branches, plans, checkpoints, and approval gates, so that progress can be reviewed and resumed rather than remaining implicit in a conversation history.⁶ A second line factors

³Aider: <https://github.com/Aider-AI/aider>.

⁴Cursor Rules: <https://cursor.com/docs/rules.md>.

⁵AGENTS.md: <https://agents.md/>.

⁶Codex, Agent Approvals & Security: <https://developers.openai.com/codex/agent-approvals-security>. Anthropic, Claude

recurring orchestration functions, such as planning, file-system state, tool invocation, and delegation, into reusable runtime components, as in OpenClaw and Deep Agents [293, 457]. A third line extends the workflow beyond a single coding loop by coordinating specialized agents across browsing, terminal use, file manipulation, context management and code editing [157, 381]. At deployment scale, managed execution systems further separate session-level reasoning from sandboxed execution, allowing long-running work to be supervised, audited, and resumed [18, 561]. Across these designs, the common goal is to make the task trajectory itself a persistent object of control rather than an unstructured sequence of model actions.

6.1.3 Feedback-driven Repair

Within each step of such a plan, the agent must turn execution feedback into revisions without mistaking local evidence for complete correctness. Tests and compiler errors make failures visible, but they do not fully specify the intended behavior, and an agent may repeatedly patch the most recent symptom, overfit to visible tests, or introduce a regression elsewhere in the repository.

Systems address this problem at both training and execution time. At training time, executable environments and repair trajectories teach agents to associate failures with plausible diagnosis-and-revision behavior, as in SWE-Gym and SWE-smith [467, 748]. At execution time, the central principle is to make repair *inspectable and reversible*: agents preserve diffs, logs, execution traces, and isolated workspaces so that failed attempts can be diagnosed rather than simply overwritten [657, 747]. Production-oriented systems extend this loop with branch isolation, hooks, approval boundaries, rollback, and final verification gates.⁷ The result is more than iterative debugging: the agent accumulates evidence across failed attempts while retaining enough control to revise or discard earlier decisions. Not all of this requires an open-ended agent loop, however: Agentless shows that a fixed localize-then-repair pipeline can rival more elaborate agentic systems [703].

6.2 Information Seeking

Information-seeking agents are a prominent class of long-horizon agents, because answering complex information needs often requires progressively accumulating evidence over extended retrieval trajectories, where relevant information is distributed across heterogeneous sources and only partially observable at any individual search step. We organize information-seeking agents around four capabilities: *deep search* sustains a single line of inquiry across many retrieval hops; *wide search* coordinates numerous parallel investigations under a shared objective; *multimodal grounding* extends retrieval beyond text to evidence in documents, figures, and images; and *research synthesis* organizes dispersed evidence into coherent, well-attributed research reports.

Together, these capabilities define the progression from retrieval-centric systems to long-horizon information-seeking agents. Rather than locating isolated facts, such agents must continuously acquire, compress, verify, and integrate evidence across extended trajectories [284]. They coordinate multiple investigations, maintain evolving research state, synthesize findings into structured outputs, and ground conclusions in heterogeneous sources. The resulting challenge is no longer search alone, but the end-to-end management of evidence throughout the entire research lifecycle [239].

6.2.1 Deep Search

Deep search concerns sustained investigation along a single line of inquiry, where relevant evidence may only emerge after many rounds of retrieval, navigation, and reasoning [223, 883]. As search trajectories lengthen, maintaining useful state becomes increasingly important. Deep search agents can be broadly categorized into two paradigms according to how they optimize long-horizon search behaviors. The first paradigm consists of RL-based deep search agents, which explicitly optimize multi-step search policies through reinforcement learning [367]. Existing efforts mainly improve these systems from two complementary directions. One line focuses on constructing challenging long-horizon search tasks that encourage models to acquire complex search and reasoning behaviors [586, 689], while another develops more effective reinforcement learning algorithms to stabilize exploration, credit assignment, and long-term planning [120, 130, 682]. The second paradigm comprises workflow-based deep search agents, which rely on carefully designed reasoning and retrieval

Code documentation: <https://code.claude.com/docs/en/overview>.

⁷Anthropic, *Claude Code Hooks* reference: <https://code.claude.com/docs/en/hooks>.

pipelines rather than policy optimization. A series of methods represented by ReSum therefore compress accumulated interaction histories into compact summaries or latent states that can be repeatedly reused across long trajectories [530, 693], while another class of approaches focus on condensing retrieved documents into task-relevant evidence representations for downstream reasoning [106, 336]. Long search chains also make retrieval failures harder to distinguish from reasoning failures [772] and amplify the risk of propagating incorrect assumptions or low-quality evidence [480]. Representative systems such as WebSeer continuously reassess intermediate conclusions, identify unsupported claims, and trigger additional retrieval when evidence remains incomplete or inconsistent [207, 332, 746]. Through these mechanisms, retrieval becomes a persistent evidence-accumulation process that supports deep investigation over extended horizons.

6.2.2 Wide Search

Wide search addresses research objectives whose scope exceeds a single search trajectory. Rather than pursuing one chain of evidence to completion, agents must coordinate many partially independent investigations while maintaining coverage of a large information space. A common strategy is task decomposition, where broad objectives are recursively broken into manageable research questions, evidence requirements, or execution plans. Explicit planning further enables agents to revise information needs and adapt research trajectories as new evidence emerges during execution [227, 292, 650, 746]. Increasingly, decomposition is coupled with parallel exploration. Different agents or search branches investigate complementary hypotheses, domains, or source collections before aggregating findings into a shared research state. Systems such as FlowSearch demonstrate that parallel search can substantially improve coverage and reduce the effective depth of individual trajectories [227, 241, 295, 556, 736]. Beyond parallel exploration, recent methods increasingly organize the search process as structured graphs, trees, or tables, which explicitly maintain search states and support dynamic expansion and revision as exploration proceeds [227, 289, 292]. The resulting challenge shifts from discovering isolated facts to coordinating large-scale evidence collection under a unified objective.

6.2.3 Multimodal Grounding

Multimodal grounding extends information seeking beyond textual retrieval. Many long-horizon research tasks require agents to gather evidence scattered across images, videos, and visually rich documents [114, 414, 714], where multilingual visual-text comprehension, document parsing, and OCR provide the perceptual access needed when text-based retrieval alone is insufficient [159, 290, 459, 584]. Recent work has therefore expanded information-seeking agents toward image search, enabling agents to retrieve, localize, and interpret visual evidence embedded in web-pages or documents, such as screenshots, diagrams, charts, and figures, while grounding subsequent reasoning in visual observations rather than textual descriptions alone [67, 138, 175, 346, 802, 825]. Another line of work focuses on video search, where agents actively explore long videos to identify answer-critical segments through iterative reasoning and interaction, replacing exhaustive frame-by-frame parsing with adaptive evidence seeking over temporal visual content [146, 361, 372]; the underlying active-perception and video-memory systems are treated in Section 6.4. These advances extend information seeking from text-centric retrieval toward unified search over heterogeneous sources.

6.2.4 Research Synthesis

Research synthesis focuses on transforming large collections of retrieved evidence into coherent research artifacts. In long-horizon settings, information gathering alone is insufficient; agents must organize findings accumulated across many searches, identify relationships among sources, and construct narratives that remain faithful to the underlying evidence. Its central application is deep-research report generation, where agents synthesize retrieved evidence into comprehensive reports for domains such as open-domain research, finance, and business analysis [136]. Beyond research prototypes, widely deployed commercial systems such as OpenAI Deep Research, Gemini Deep Research, and Perplexity have brought this capability to broad audiences. Representative systems progressively externalize intermediate artifacts, including outlines, section plans, research notes, and persistent workspaces, before composing final reports [337, 356, 827, 891]. They further maintain explicit evidence attribution so that readers can trace individual claims back to their sources, improving both faithfulness and auditability [292, 347, 621]. The core challenge is therefore not merely to summarize retrieved documents, but to preserve coverage, narrative coherence, factual support [434, 677], and source-level traceability across a long search trajectory.

6.3 Computer Use

Computer-use agents interact with stateful software systems in which actions have persistent consequences and task completion requires sustained execution across extended interaction trajectories. These agents autonomously manipulate desktops, browsers, and applications through heterogeneous interfaces including screenshots, DOMs, accessibility trees, shells, executable code, and file systems [126, 588]. Recent systems such as Anthropic’s Computer Use and OpenAI’s Operator support planning, multi-tool coordination, and persistent execution state across long-horizon tasks [12, 452, 745].

We organize long-horizon computer use according to the execution environment: *browser agents*, *desktop GUI agents*, and *mobile agents*. Browser agents focus on sustained task execution within web environments through navigation and information extraction. Desktop GUI agents emphasize reliable visual grounding and interaction with operating systems and desktop applications. Mobile agents further extend long-horizon execution to smartphone applications, where agents must coordinate touch-based interaction across diverse mobile interfaces. Together, these directions characterize the evolution from browser automation to general-purpose computer use. Beyond executing actions within a single application, modern computer-use agents must coordinate perception, reasoning, and execution across heterogeneous software environments while maintaining persistent state and coherent progress toward long-running objectives.

6.3.1 Browser Agents

Browser agents investigate long-horizon interaction within web environments through navigation, information extraction, and form manipulation. Benchmarks such as WebArena, Mind2Web, and ClawBench establish browser interaction as a representative long-horizon computer-use setting, requiring agents to interpret page content, choose actions, and maintain progress across evolving interface states [116, 847, 885]. To support sustained execution, many systems expose browsers through reusable action primitives such as clicking, typing, scrolling, navigation, and form completion, allowing agents to reason over browser interaction as a structured action space rather than raw interface manipulation [210, 226, 404, 443, 781]. At the same time, accumulated observations, retrieved information, and intermediate decisions are increasingly externalized through memories, notes, summaries, or reusable knowledge stores that persist across long interaction trajectories [637, 781]. These developments progressively transform browser interaction from isolated page-level decisions into persistent task execution over extended horizons.

6.3.2 Desktop GUI Agents

Desktop GUI agents study how autonomous systems interact with operating systems and graphical desktop environments through multimodal perception and action. Benchmarks such as OSWorld and Workflow-GYM, together with representative systems including UI-TARS, have established desktop interaction as a realistic long-horizon setting, requiring agents to perceive interface state, execute multi-step workflows, and recover from execution failures across diverse applications [494, 526, 551, 716, 745]. Visual grounding plays a central role in desktop interaction by maintaining reliable correspondence between evolving interface observations and executable actions. Because graphical layouts can be both dense and dynamic across applications and execution steps, agents must identify relevant elements in high-resolution interfaces [324], interpret their functional roles, and update action targets throughout long interaction trajectories [181, 691]. Modern systems further complement multimodal perception with execution verification mechanisms based on screenshot comparison, visual change detection, semantic consistency checks, and artifact inspection [12, 259, 518], together with safety guardrails such as permission boundaries and confirmation policies [12, 455] to improve robustness under execution uncertainty.

6.3.3 Mobile Agents

Mobile agents extend long-horizon computer use to smartphones and tablets, where agents must coordinate interactions across multiple applications through touch-based interfaces [282, 373, 570]. Compared with desktop environments, mobile tasks typically involve long multi-app workflows, limited screen space, frequent context switching, and gesture-based operations, making persistent planning and state management particularly important [667, 766]. Recent systems increasingly adopt hierarchical agent architectures that separate high-level planning from low-level UI execution, allowing complex user requests to be decomposed into executable

subtasks while maintaining progress across extended interaction trajectories [543, 667, 723, 766]. At the same time, long-term memories, reusable interaction skills, and execution verification mechanisms are incorporated to improve robustness and enable continual improvement from previous experiences [105, 571, 723, 766]. These developments transform mobile agents from reactive UI operators into persistent assistants capable of accomplishing complex real-world workflows across diverse mobile applications.

6.4 Multimodal Agents

Multimodal agents are long-horizon agents that must process information from different modalities. In long-horizon scenarios, multimodal evidence may be distributed across long visual contexts or multiple videos [55, 320, 413, 666], requiring agents to locate and acquire relevant information, store it, reason over it, and integrate it into an answer or multimodal output. Recent benchmarks such as AgentVista and OmniGAIA mainly evaluate agents’ ability to solve cross-modal tasks [339, 564, 589], rather than focusing only on isolated visual question answering or reasoning [45, 85, 190, 465].

We organize long-horizon multimodal agents into three categories according to their tasks and capabilities: *multimodal understanding*, *multimodal generation*, and *omnimodal agency*. Multimodal understanding focuses on how to locate useful information over extended timelines, including deciding which segments to inspect and which perception tools to invoke. Multimodal generation focuses on how to understand user intent and conduct multi-round planning, generation, and refinement, a process that relies on external evidence, verification mechanisms, and persistent memory. Omnimodal agency focuses on the coordination of different modalities so that they can jointly accomplish an end-to-end task. Overall, these three types of capabilities enable agents to handle richer multimodal information and interact more deeply with the real world.

6.4.1 Multimodal Understanding

For long-horizon multimodal understanding, agents must possess the ability of *active perception*. Under limited budgets, it is impossible to read all information streams at once; truly important information often appears only in a few frames of a long video, a small region of an image, or a single line of text in a document. Therefore, agents should be able to actively locate useful information within long information streams.

Current designs follow two main lines. The first line studies *tool-augmented active perception*. Works represented by VideoSeek treat long videos and images as explorable spaces rather than inputs that are fed into the model all at once [138, 365, 655, 825, 903]. The agent dynamically invokes browsing and retrieval tools according to the current question and existing clues, progressively locating key evidence in long videos. The second line studies *memory-augmented evidence organization*. Representative works such as DVD [510, 739, 829] construct structured video memory and then retrieve relevant evidence according to task demands. The former emphasizes active exploration strategies, while the latter emphasizes structured representations of long information streams and evidence preservation [796]. Their shared goal is to avoid blindly reading all multimodal inputs under limited budgets.

6.4.2 Multimodal Generation

In long-horizon scenarios, multimodal generation is not a one-step mapping from a prompt to an image or video clip. Agents usually need to understand user intent, plan the generation process, and conduct multiple rounds of generation and verification. Therefore, multimodal generation is a long-term task that must be completed through continuous iteration. Recent multimodal generation models, such as those in the Qwen-Image, Seedream, and SeedEdit series, have significantly improved intent understanding, multimodal creation, image editing, and video synthesis capabilities [73, 115, 172, 528, 600, 645, 855]. However, for long-horizon agents, beyond the generative capability of the model itself, the design of the surrounding harness is more important for ensuring that generated content remains consistent with user requirements.

Therefore, three mechanisms are crucial. *Evidence seeking* provides grounding for generation by retrieving images, documents, and videos. This is especially necessary in knowledge-intensive multimodal generation tasks, where sufficient task-relevant information must be collected. Recent generative agents autonomously invoke retrieval and visual analysis tools to gather task-relevant evidence before generation [68, 209, 255]. *Verification* checks generated results from multiple dimensions to ensure that they remain aligned with

the original goal [574, 741, 831]. *Memory* stores user requirements and evidence at different granularities, enabling subsequent generation to remain consistent with earlier decisions [71, 733]. Recent multimodal generation systems increasingly organize planning, evidence collection, memory, verification, and rendering into unified agentic workflows. For example, PTAH coordinates specialized agents throughout the research and writing lifecycle, maintains structured visual working memory, and enforces cross-modal consistency through verifier-guided report generation [803]. Similarly, long-form audio-video systems such as MUSE employ planning-execution-verification-revision loops, transforming multimodal generation into a long-horizon iterative process [574].

6.4.3 Omnimodal Agency

Omnimodal tasks require agents to coordinate across different modalities, because a single modality or a single model is often insufficient to complete the task. This is closer to real-world scenarios, where inputs may include text, images, speech, audio, video, and other modalities. Therefore, agents need the ability of *cross-modal orchestration*. Specifically, an agent needs to decompose the task, route subtasks to appropriate specialized models, and then verify and integrate their outputs.

Recent work has developed along two complementary directions. One direction focuses on native omnimodal models, such as those in the Qwen-Omni series, which provide unified interfaces for multimodal understanding or generation and usually support streaming interaction and speech-centered multimodal dialogue [498, 726, 727]. The other direction focuses on orchestration-based agents, such as OmniAgent, Orchestra-o1, and OmniGAIA, which explicitly assign subtasks to different modalities and coordinate their execution processes and outputs [137, 339, 364, 718, 728, 806]. In long-horizon omnimodal tasks, information from different modalities may conflict with each other, so explicit verification is required. Omnimodal agent capabilities enable agents to interact more deeply with real-world environments and broaden their range of applications.

6.5 General-purpose Agents

General-purpose agents represent one of the broadest forms of long-horizon agency, because they must pursue open-ended goals across heterogeneous interfaces rather than operate within a single fixed environment. A user request may require an agent to interact with multiple interfaces and invoke different types of tools. The central challenge for such agents is therefore *sustained coordination*: an agent must conduct overall planning for the task, maintain and transfer state across different environments, and recover from failures during execution. Benchmarks such as GAIA and τ -bench reflect a recent shift toward evaluating whether agents can consistently complete tasks in heterogeneous environments [335, 387, 431, 762, 840]; we do not restate the individual environments here, and collect them together with the domain-specific benchmarks reused throughout this section in Table 7.

We organize general-purpose agents into three representative categories according to their deployment scenarios. *Personal assistants* deploy general-purpose agent capabilities in personal computing and office environments, where agents must operate across applications and interact continuously with users. *Embodied agents and world models* extend such capabilities to physical environments, where agents must perceive, reason, and predict the consequences of actions in the world. *Productive agents* apply general-purpose agent capabilities to professional workflows, where outputs must satisfy domain-specific standards and verification requirements.

6.5.1 Personal Assistants

Personal assistants are an increasingly important class of agents designed to operate across everyday digital environments. Unlike traditional conversational assistants, these agents are expected not only to answer user questions, but also to complete continuous tasks across applications and local resources. Personal assistants therefore mark a shift from conversational assistance to persistent computer agents: the agent no longer merely generates text, but can continuously carry out tasks through multi-turn interaction with both the user and the environment.

A capability underlying all personal assistants is *general tool use*: selecting and invoking the right tools to act beyond the model’s parametric knowledge. Early benchmarks assess this ability *without environment state*, testing whether an agent can choose and call the correct function from large, scalable tool sets, as

in ToolBench [492] and the Berkeley Function-Calling Leaderboard [472]. τ -bench [762] then moves tool use into *stateful* environments, where each call mutates a persistent backend and success is judged against realistic multi-turn user interactions rather than isolated call correctness. More recent benchmarks push toward mixed, cross-application settings with increasingly long-tailed tool inventories: Toolathlon [322] spans diverse realistic applications, while MCP-Atlas [31] and MCP-Mark [700] stress-test tool-use competency over large, heterogeneous MCP tool ecosystems. Finally, Claw-Eval [764] and ClawMark [426] evaluate tool use mediated by reusable *skills*, enabling more flexible invocation that better reflects modern agent deployments.

Personal assistants can be broadly divided into four categories. The first targets *office scenarios*, focusing on whether agents can complete tasks in realistic workflows involving documents, emails, calendars, and related office applications. Benchmarks such as OfficeQA, APEX-Agents, PresentBench, and SpreadsheetBench examine capabilities in information retrieval, file and presentation generation, spreadsheet manipulation, and multi-step task execution in such office environments [74, 415, 458, 623]. The second targets *personal computing scenarios*, emphasizing agents’ ability to operate real device interfaces, including browsers, desktop applications, mobile apps, and local file systems; this setting largely reuses the computer-use systems and benchmarks of Section 6.3 (e.g., UI-TARS and OSWorld [494, 716]) and is not detailed again here. The third targets *software development scenarios*, where agents assist developers with coding tasks that require code understanding, modification, and validation. Systems such as Claude Code and Codex are representative examples of this direction [10, 453]. The fourth targets *persistent personal assistant scenarios*, emphasizing agents that run continuously in user-controlled devices or environments and connect local resources, long-term memory, and external tools. Systems such as OpenClaw and Hermes illustrate this direction [420, 457, 513, 893].

6.5.2 Embodied Agents and World Models

Embodied agents extend agent capabilities from digital environments to the physical world, where they must continuously perceive the environment, reason about goals and constraints, and execute actions that alter the world state. Long-horizon embodied tasks therefore require a sustained loop of perception, planning, and action, together with reliable state tracking, safety constraints, and recovery from execution errors. Their capabilities can be broadly divided into *navigation* and *manipulation*. Navigation focuses on understanding spatial layouts, planning paths, and adapting to environmental changes [135, 755, 771], whereas manipulation requires fine-grained reasoning about spatial references, object states, physical properties, and interactions such as grasping, moving, and assembling [93, 684, 872, 878]. Recent vision-language-action models, including Gemini Robotics and Qwen’s robotic VLA models, integrate language instructions, visual observations, and robotic control within a unified action interface, marking a shift from task-specific controllers toward more general-purpose embodied agents [35, 245, 594, 646, 784, 817].

World models provide embodied agents with predictive representations of environment dynamics, allowing them to evaluate possible action outcomes before acting. This capability is particularly important in physical environments, where exploration is costly and execution failures may be difficult to reverse. Recent world-model research can be broadly divided into two main directions. The first is *predictive representation models*, exemplified by V-JEPA 2 and DINO-WM, which compress the environment into abstract latent representations and predict how these representations evolve over time and in response to actions, rather than directly generating complete future images [25, 879]. By omitting visual details such as texture and illumination, these models can support more efficient multi-step prediction and planning. The second direction is *generative world models*, including Cosmos, Cosmos-Predict2.5, and OSCAR, which directly generate predicted outcomes following an agent’s actions to support trajectory comparison, policy training, and risk assessment before real-world execution [2, 449, 699]. Building on these two directions, *world-action models*, such as DreamZero and τ_0 -WM, have recently emerged to unify world prediction and action generation by jointly modeling future states and executable actions [769, 884].

6.5.3 Productive Agents

Productive agents apply general-purpose capabilities to workflows whose outputs must satisfy domain-specific standards. They retrieve authoritative sources, invoke specialized tools, and maintain traceable intermediate and final artifacts over extended trajectories. The central challenge is therefore not simply task automation, but the integration of planning, tool use, memory, and verification into established practice. We group these

applications into two intuitive families: (1) *professional services*, where agents assist established work in finance, law, and medicine; and (2) *scientific discovery and reasoning*, where agents help generate and verify new knowledge.

Professional Services. In finance, agents integrate market data, financial statements, and related evidence to produce traceable analyses, as illustrated by FinRobot and FinSight [34, 263, 886]. Legal agents retrieve authoritative sources, connect facts to rules, and produce outputs that can be reviewed against legal doctrine and procedure; the Legal Agent Benchmarks, together with agents such as Parthenon Law and LawThinker, reflect the shift from static question answering toward long-horizon legal work [174, 185, 315, 756]. Medical agents must further combine patient records, images, and examination reports while respecting safety, privacy, and auditability requirements [21, 904]. MedAgentBench, MedAgentBoard, and PhysicianBench accordingly evaluate sustained clinical reasoning and workflow execution rather than isolated diagnosis [23, 257, 380, 895]. Across these domains, correctness cannot usually be reduced to a single executable oracle, so provenance, procedural compliance, and expert review remain central [474, 752].

Scientific Discovery and Reasoning. Scientific-discovery agents go beyond producing domain-compliant analyses: they generate new claims and test them against computational or empirical feedback. Their workflows connect literature review with expert-level and experimental scientific reasoning [642, 882], hypothesis generation, experimental design, execution, result analysis, and iterative refinement [505, 678, 869]. The AI Scientist and its successor organize machine-learning research into repeated cycles of ideation, experimentation, evaluation, and writing [397, 740]. AlphaEvolve and Kosmos extend this pattern to algorithmic and cross-domain discovery [438, 448], while NovelSeek closes the loop from hypothesis generation to verification across heterogeneous scientific tasks [800]. In empirical settings, Coscientist couples language-model planning with robotic chemical experimentation [37], and Robin and the Virtual Lab connect multi-agent reasoning to biomedical discovery and laboratory validation [177, 581]. Hypotheses, programs, experimental records, and manuscripts consequently become evolving research state that must be revised whenever new evidence arrives.

Formal mathematics represents a complementary form of discovery in which proof assistants can mechanically check each completed proof [615, 866]. Seed-Prover 1.5 combines natural-language reasoning, lemma-level sketches, recursive decomposition, and an agentic Lean prover to construct long proofs under iterative verification [59]. Prover Agent similarly coordinates informal reasoning, auxiliary-lemma generation, formal proof construction, and Lean diagnostics in a feedback-driven refinement loop [26]. Automated Proof Engineering broadens the objective beyond isolated theorem proving to repository-scale work, including feature addition, refactoring, and bug repair across evolving Lean libraries, with evaluation based on both compilation and semantic requirement satisfaction [717]. Together, empirical science and formal mathematics show how productive agents can move beyond summarizing existing knowledge to generating claims that survive domain-specific verification [629].

6.6 Benchmarks and Resources

Progress on long-horizon agents is measured and reproduced through a shared substrate of benchmarks and open resources. Because a reported score reflects the joint contribution of the model, the harness, and the training recipe, long-horizon evaluation is best read as *horizon-aware*, sensitive to how long a goal must be held, and *attribution-aware*, able to separate harness-induced from model-induced variance [123, 286, 468]. The same verifiable, long-horizon environments that serve as benchmarks are also frequently reused as training environments for the optimization methods of Section 5, so progress on evaluation and progress on internalization are tightly coupled. Beyond the domain benchmarks tabulated below, a growing family of suites deliberately stresses multi-hour autonomy. Examples range from machine-learning engineering in MLE-bench [51] to end-to-end replication of research papers in PaperBench [560], complementing the frontier time-horizon measurements of METR [286, 429]; these appear among the productive-agent entries of Table 7.

Table 7 consolidates representative resources for the application domains above, grouped by domain and, within each domain, ordered by the capability taxonomy of Section 6. Each row pairs an evaluation benchmark or a reusable system with the capability it targets and, where available, a link to its public repository or official release page (🔗), offering a concrete entry point into the works discussed throughout this section. The multimodal-understanding entries (e.g., video and streaming comprehension benchmarks) are not long-horizon

agent tasks in the strict sense; we include them because they measure the foundational perception capabilities on which long-horizon multimodal agents build.

Table 7 Representative systems and benchmarks relevant to long-horizon agents, grouped by domain. One work per row, annotated with its capability, type, a short description, an (approximate) release date, and a code repository. *Type* distinguishes evaluation *Benchmarks* from reusable *Systems*; within each domain, rows are grouped by capability (per Section 6) and ordered by release date. The list is representative rather than exhaustive.

Work	Capability	Type	Description	Date	GitHub Repo
Software Engineering					
AutoCodeRover [843]	Repository Grounding	System	Structure-aware fault localization and patching	2024.04	 GitHub
SWE-agent [747]	Repository Grounding	System	Agent-computer interface for repositories	2024.05	 GitHub
OpenHands [657]	Repository Grounding	System	Persistent coding workspace	2024.07	 GitHub
NL2Repo-Bench [124]	Repository Grounding	Benchmark	From-scratch repository generation	2025.12	 GitHub
OctoBench [123]	Repository Grounding	Benchmark	Scaffold-aware instruction following	2026.01	 GitHub
SWE-bench [260]	Workflow-level Planning	Benchmark	Repository issue resolution	2023.10	 GitHub
Claude Code [10]	Workflow-level Planning	System	Autonomous code editing	2025.02	 GitHub
Trae Agent [169]	Workflow-level Planning	System	Ensemble agent for repository issue resolution	2025.07	 GitHub
SWE-bench Pro [117]	Workflow-level Planning	Benchmark	Enterprise repository issue resolution	2025.09	 GitHub
Terminal-Bench 2.0 [428]	Workflow-level Planning	Benchmark	Hard terminal-based tasks	2025.11	 GitHub
Aider ⁸	Feedback-driven Repair	System	Iterative pair-programming loop	2023.06	 GitHub
DebugBench [602]	Feedback-driven Repair	Benchmark	Multi-language bug diagnosis	2024.01	 GitHub
Agentless [703]	Feedback-driven Repair	System	Localize-then-repair pipeline	2024.07	 GitHub
SWE-Gym [467]	Feedback-driven Repair	System	Executable training environments	2024.10	 GitHub
SWE-smith [748]	Feedback-driven Repair	System	Repair-trajectory synthesis	2025.04	 GitHub
Information Seeking					
Search-o1 [336]	Deep Search	System	Retrieval and document reading	2025.01	 GitHub
BrowseComp [676]	Deep Search	Benchmark	Persistent web browsing	2025.04	 GitHub
WebDancer [689]	Deep Search	System	Information-seeking policy	2025.05	 GitHub
Tongyi DeepResearch [307]	Deep Search	System	Agentic deep-research model	2025.10	 GitHub
MiroThinker [599]	Deep Search	System	Interactive-scaling research agent	2025.11	 GitHub
Open Deep Research [292]	Wide Search	System	Evidence aggregation and synthesis	2025.06	 GitHub
WideSearch [685]	Wide Search	Benchmark	Structured information collection	2025.08	 GitHub
FlowSearch [227]	Wide Search	System	Knowledge-flow orchestration	2025.10	 GitHub
GISA [896]	Wide Search	Benchmark	Deep reasoning and broad information aggregation	2026.02	 GitHub
MMInA [603]	Multimodal Grounding	Benchmark	Multi-hop multimodal browsing	2024.04	 GitHub
MM-BrowseComp [331]	Multimodal Grounding	Benchmark	Multimodal evidence seeking	2025.08	 GitHub
WebWatcher [175]	Multimodal Grounding	System	Vision-language deep research	2025.08	 GitHub
WebThinker [337]	Research Synthesis	System	Autonomous research workflow	2025.04	 GitHub
DeepResearch Bench [136]	Research Synthesis	Benchmark	Research-report quality evaluation	2025.06	 GitHub
WebWeaver [356]	Research Synthesis	System	Evidence organization and synthesis	2025.09	 GitHub
DeepConsult [592]	Research Synthesis	Benchmark	Consulting and business deep research	~2025	 GitHub
Computer Use					
Mind2Web [116]	Browser Agents	Benchmark	Cross-domain web demonstrations	2023.06	 GitHub
WebArena [885]	Browser Agents	Benchmark	Realistic interactive websites	2023.07	 GitHub
WebVoyager [208]	Browser Agents	Benchmark	End-to-end live-web agent evaluation	2024.01	 GitHub
browser-use ⁹	Browser Agents	System	Browser control through agent loop	2024.10	 GitHub
BrowserAgent [781]	Browser Agents	System	Human-inspired native browsing	2025.10	 GitHub
OSWorld [716]	Desktop GUI Agents	Benchmark	Desktop task environment	2024.04	 GitHub
UI-TARS [494]	Desktop GUI Agents	System	Native visual GUI perception	2025.01	 GitHub
ScreenSpot-Pro [324]	Desktop GUI Agents	Benchmark	High-resolution professional GUI grounding	2025.01	 GitHub
PointArena [83]	Desktop GUI Agents	Benchmark	Language-guided visual pointing	2025.05	 GitHub
Seed 2.0 [527]	Desktop GUI Agents	System	Frontier multimodal foundation model	2026.01	 GitHub
AndroidWorld [508]	Mobile Agents	Benchmark	Dynamic Android environment	2024.05	 GitHub
AndroidLab [734]	Mobile Agents	Benchmark	Android agent training framework	2024.10	 GitHub
Mobile-Agent-v3 [766]	Mobile Agents	System	Fundamental GUI-automation agents	2025.08	 GitHub

(continued on next page)

Table 7 (continued)

Work	Capability	Type	Description	Date	GitHub Repo
Mobile-Agent-v3.5 [723]	Mobile Agents	System	Multi-platform fundamental GUI agents	2026.01	 GitHub
Multimodal Agents					
EgoSchema [419]	Multimodal Understanding	Benchmark	Egocentric video question answering	2023.08	 GitHub
MVBench [328]	Multimodal Understanding	Benchmark	Comprehensive video understanding	2023.11	 GitHub
VideoAgent [655]	Multimodal Understanding	System	Agentic evidence selection over long video	2024.03	 GitHub
Video-MME [158]	Multimodal Understanding	Benchmark	Long-video understanding	2024.05	 GitHub
LongVideoBench [688]	Multimodal Understanding	Benchmark	Long-video and text understanding	2024.07	 GitHub
StreamingBench [366]	Multimodal Understanding	Benchmark	Streaming-video understanding	2024.11	 GitHub
DVD [829]	Multimodal Understanding	System	Tool-using video deep research	2025.05	 GitHub
VideoSeek [365]	Multimodal Understanding	System	Tool-augmented active perception in long video	2026.03	 GitHub
GenAI-Bench [306]	Multimodal Generation	Benchmark	Text-to-visual generation evaluation	2024.06	 GitHub
MM-StoryAgent [733]	Multimodal Generation	System	Text-image-audio story generation	2025.03	 GitHub
MUSE [574]	Multimodal Generation	System	Plan-execute-verify story-video generation	2026.02	 GitHub
Ptah [803]	Multimodal Generation	System	Verifier-guided multimodal report generation	2026.05	 GitHub
CrayOtter [741]	Multimodal Generation	System	Verification-driven multimodal generation	2026.06	 GitHub
OmniVideoBench [308]	Omnimodal Agency	Benchmark	Audio-visual evaluation	2025.10	 GitHub
Agent-Omni [364]	Omnimodal Agency	System	Omnimodal reasoning agent	2025.11	 GitHub
OmniGAIA [339]	Omnimodal Agency	Benchmark	Cross-modal tool-use reasoning	2026.02	 GitHub
AgentVista [564]	Omnimodal Agency	Benchmark	Multimodal agent evaluation	2026.02	 GitHub
Orchestra-o1 [806]	Omnimodal Agency	System	Cross-modal orchestration agent	2026.06	 GitHub
General-Purpose Agents					
ToolBench [492]	Personal Assistants	Benchmark	Large-scale tool use without env state	2023.07	 GitHub
Open Interpreter ¹⁰	Personal Assistants	System	Local computer and code execution	2023.07	 GitHub
AgentBench [387]	Personal Assistants	Benchmark	Multi-environment agent evaluation	2023.08	 GitHub
GAIA [431]	Personal Assistants	Benchmark	General assistant tasks	2023.11	 GitHub
τ -bench [762]	Personal Assistants	Benchmark	Stateful tool-agent-user interaction	2024.06	 GitHub
AppWorld [614]	Personal Assistants	Benchmark	Stateful app-based personal tasks	2024.07	 GitHub
BFCL [472]	Personal Assistants	Benchmark	Function-calling leaderboard	2024.08	 GitHub
OpenManus ¹¹	Personal Assistants	System	General-purpose agent workflows	2025.03	 GitHub
MCP-Mark [700]	Personal Assistants	Benchmark	Realistic MCP tool-use stress test	2025.09	 GitHub
Toolathlon [322]	Personal Assistants	Benchmark	Diverse realistic tool environments	2025.10	 GitHub
MCP-Atlas [31]	Personal Assistants	Benchmark	Large-scale tool-use competency	2026.02	 GitHub
Claw-Eval [764]	Personal Assistants	Benchmark	Trustworthy autonomous-agent evaluation	2026.04	 GitHub
ClawMark [426]	Personal Assistants	Benchmark	Multi-day multimodal living-world eval	2026.04	 GitHub
Agents' Last Exam [575]	Personal Assistants	Benchmark	Frontier professional-workflow evaluation	2026.06	 GitHub
VLN-CE [283]	Embodied & World Models	Benchmark	Continuous vision-language navigation	2020.04	 GitHub
OpenVLA [277]	Embodied & World Models	System	Open vision-language-action model	2024.06	 GitHub
DINO-WM [879]	Embodied & World Models	System	Feature-space world model	2024.11	 GitHub
EmbodiedBench [753]	Embodied & World Models	Benchmark	Embodied task evaluation suite	2025.02	 GitHub
GR00T N1 [35]	Embodied & World Models	System	Humanoid vision-language-action model	2025.03	 GitHub
$\pi_{0.5}$ [245]	Embodied & World Models	System	Open-world VLA model	2025.04	 GitHub
V-JEPA 2 [25]	Embodied & World Models	System	Latent video-prediction world model	2025.06	 GitHub
Cosmos-Predict2.5 [2]	Embodied & World Models	System	Video world simulation	2025.10	 GitHub
ChemCrow [39]	Productive Agents	System	Tool-augmented chemistry reasoning	2023.04	 GitHub
FinRobot [886]	Productive Agents	System	Financial research and analysis	2024.05	 GitHub
MLE-bench [51]	Productive Agents	Benchmark	Machine-learning engineering tasks	2024.10	 GitHub
LegalAgentBench [315]	Productive Agents	Benchmark	Multi-step legal tasks	2024.12	 GitHub
MedAgentBench [257]	Productive Agents	Benchmark	Clinical EHR workflows	2025.01	 GitHub
METR [286, 429]	Productive Agents	Benchmark	Frontier time-horizon measurement	2025.03	 GitHub
AI Scientist-v2 [740]	Productive Agents	System	Autonomous scientific discovery	2025.04	 GitHub
PaperBench [560]	Productive Agents	Benchmark	End-to-end research-paper replication	2025.04	 GitHub
AlphaEvolve [448]	Productive Agents	System	Algorithmic discovery agent	2025.05	 GitHub
Finance Agent Benchmark [34]	Productive Agents	Benchmark	Real-world financial analysis tasks	2025.08	 GitHub
GDPval [474]	Productive Agents	Benchmark	Real-world economic task value	2025.10	 GitHub

(continued on next page)

Table 7 (continued)

Work	Capability	Type	Description	Date	GitHub Repo
Kosmos [438]	Productive Agents	System	Cross-domain discovery agent	2025.11	 GitHub
APEX-Agents [623]	Productive Agents	Benchmark	Office multi-step task execution	2026.01	 GitHub
OfficeQA Pro [458]	Productive Agents	Benchmark	Enterprise grounded reasoning	2026.03	 GitHub
PresentBench [74]	Productive Agents	Benchmark	Rubric-based slide generation	2026.03	 GitHub
OneMillion Bench [752]	Productive Agents	Benchmark	Human-level economic tasks	2026.03	 GitHub
Workspace-Bench [588]	Productive Agents	Benchmark	Workspace file-dependency tasks	2026.05	 GitHub
YC-Bench [211]	Productive Agents	Benchmark	Long-term planning consistency	2026.06	 GitHub

Taken together, these resources show that long-horizon evaluation is consolidating around stateful, multi-hour, and economically grounded tasks, and that a reported score must be read as a joint property of the model, the harness, and the training recipe rather than of the model alone.

7 Frontier: Open Challenges and Outlooks

The previous sections have organized long-horizon agency as it currently stands, across both pillars and the experience loop that links them. We now present a set of *positions*: claims about the most consequential paradigm-level shifts that we expect to define the next several years of research, together with the open problems each shift entails. In our view, the frontiers that matter most lie in this *in-between* space: capability migrates bidirectionally between the harness and the weights. The harness itself turns from fixed scaffolding into a learnable, portable object. And in noisy, costly, real environments, long horizons invalidate the idealized premises that both pillars rely on. Our central diagnosis is that practical long-horizon progress is increasingly a property of the runtime around the model, not of the model alone. To make these pressures easier to compare, we group the frontiers into four higher-level axes: *evolution*, *effectiveness*, *efficiency*, and *trustworthiness*. Table 8 previews the nine positions under this structure. For each concrete frontier we first state where the field stands and what blocks it (*status and challenges*), then our position on where it should go and the open problems it leaves behind (*outlook*).

7.1 Evolution: Generalization and Learning Over Time

The first frontier axis is evolution: whether an agent’s long-horizon competence can improve over time rather than remain locked to a hand-written scaffold and a frozen policy. This axis raises three linked questions. Can the harness itself self-evolve? Can the resulting competence transfer across harnesses? And can experience compound across episodes so that the agent keeps learning, and even learns for a lifetime, without destroying previously acquired capability?

7.1.1 Self-evolving Harness and Agents

Status and Challenges. The dominant paradigm of *hand-maintained* agents is a transitional artifact, and self-improvement offers a direct route to extending the effective horizon. We distinguish two levels as follow:

The first is the *self-evolving harness*: with little or no model training, the agent keeps rewriting the runtime structure around a fixed policy. This includes skill-driven fast adaptation, where new skills are distilled directly from failure and success trajectories [206, 705, 706]; recursive procedural-memory evolution, where memory and skills co-evolve from execution traces [813, 853]; and topology evolution, where sub-agents are spawned on demand and the multi-agent graph itself becomes a learnable object [101, 652, 670, 791]. Increasingly the harness is treated as a first-class artifact that can be automatically generated and optimized, as in Meta-Harness and AutoHarness [297, 395], and even hosted by agent operating systems such as AIOS that provide a unified, resource-managed runtime across agents and tasks [424]; industrial trace-based iteration treats run traces as the primary evolution signal [293, 765].

⁸Aider: <https://github.com/Aider-AI/aider>.

⁹browser-use: <https://github.com/browser-use/browser-use>.

¹⁰Open Interpreter: <https://github.com/OpenInterpreter/open-interpreter>.

¹¹OpenManus: <https://github.com/FoundationAgents/OpenManus>.

Table 8 Four frontier axes and nine frontiers for long-horizon agents. For each frontier we summarize its *open challenges* and our *outlook*. A recurring thread: the harness is where much of the next advance must happen.

Frontier	Open challenges	Outlook
I. Evolution — generalize across runtimes and keep learning over time		
Self-evolving Harness & Agents	• Hand-set optimization metric • Limited gains on out-of-distribution tasks • Overfitting and drift over long runs	• Adaptively choose optimization objective and scope • Autonomous evolution guided by feedback • Lifelong, non-overfitting evolution
Harness Generalization and Transferability	• Models tied to specific harnesses • Performance varies across model–harness pairings • Portable standards still ad hoc	• Multi-harness (hybrid-harness) training • Portable skills / experience at inference time • A standard harness protocol
Continual & Lifelong Learning	• External experience is noisy and unverified • Catastrophic forgetting during weight updates	• Learning only from validated experience • Safe online learning from billion-user logs
II. Effectiveness — act reliably in realistic, verifiable environments		
Real-world Environment Interaction	• Slow, costly, and irreversible interactions with real-world systems • Synthesized Envs & world-models may be unfaithful	• Measure, audit, and optimize sim-to-real faithfulness • Fusion of symbolic and neural simulation environments
From Digital to Embodied Agents	• Deep planning vs. real-time control • Limited modeling of physical dynamics & laws • Coarse vs. fine-grained feedback	• Async, hierarchical harness (deliberation vs. reflex) • Predict physical outcomes with world models • Integrate feedback across timescales
III. Efficiency — spend compute, context, and modality budgets		
Cost- & Budget-aware Agency	• Model selection is poorly matched to task difficulty • No calibrated cost / infeasibility sense • No runtime enforcement of ceilings • No law linking budget to success	• Perceive cost, difficulty, infeasibility • Act mid-run: shorten, re-route, or stop • Harness-enforced ceilings & cost-aware routing • A budget-vs-success scaling law
Multimodal & Omni Harness	• Multimodality bolted on; no vision-native harness • Visual-token budgeting is heuristic • Cross-modal verification is unreliable	• A vision-native, unified omni harness substrate • Per-modality granularity control • Cross-modal verification & omni-modal routing
IV. Trustworthiness — stay robust and governable as the horizon grows		
Reflection & Error Robustness	• Failures detected late under noisy, delayed feedback • Intrinsic self-correction is unreliable • Errors compound into goal drift	• Early failure detection & path-switching • External-anchored, calibrated self-knowledge • Align to human course-correction
Safety & Governance	• Injected error & hazardous experience reuse • No unified safety-verification standard • Self-evolution erodes invariants; biased self-checks	• Untrusted external input; gate experience reuse • Score action safety beside effectiveness • Independent, invariant-preserving verification

The second, more ambitious level is the *self-evolving agent*, whose goal is to fold harness structure and skills back into the weights: the Darwin Gödel Machine rewrites its own code to improve a coding agent [814], and a small but growing line couples skill discovery with reinforcement learning or routes rich runtime trajectories into training, as in SkillRL and OpenClaw-style RL such as Claw-R1 [457, 627, 705].

The shared limitations are threefold, mirroring the outlook below: **(1)** the optimization objective is still a hand-set benchmark metric rather than something the agent chooses for itself; **(2)** gains remain largely within-distribution while the evolution history stays opaque about *what* changed and *why*; and **(3)** over long runs such systems risk overfitting a single benchmark or drifting.

Outlook. We expect self-evolution to mature into a complex *automated-optimization* problem pursued with minimal human supervision, and we highlight three requirements. **(1)** The agent should *autonomously discover its own optimization objective*. Today this is typically a single task’s benchmark metric or a training-loss level; in the future it will more likely be a joint objective over many tasks. The agent should locate where its own competence is weak on each task and decide accordingly where to optimize. Sometimes it is enough to

adjust one part of the harness (orchestration logic, a memory module, the tool set), while in other cases the harness and the model weights should be optimized together. **(2)** Self-evolution should proceed *autonomously*: the agent should acquire external information, distill reusable skills (for instance, high-value strategies for deep web search), and continuously update its optimization strategy in response to environmental feedback. **(3)** This process should be able to run *continually, even for a lifetime*: driven by a stream of sparse external signals, it should neither overfit to a single benchmark nor drift into random change, which makes stable long-run self-evolution a genuinely long-term goal. Crucially, if self-evolution is to be more than per-benchmark overfitting, the structure it produces must survive being moved to a different runtime. This is the transferability problem we turn to next.

7.1.2 Harness Generalization and Transferability

Status and Challenges. Harness paradigms keep multiplying, spanning early patterns such as ReAct and CodeAct, the Model Context Protocol, and today’s workstation-style harnesses like OpenClaw, Hermes, and Claude Code. As they do, the field is drifting toward a regime in which each strong model is implicitly co-trained with, and bound to, a single harness. Yet what users expect from a capable model is the opposite: that they can drive it from their own agent, and even extend it to a harness they design themselves. We regard this binding as the central scalability risk of the field: a model competitive only inside the harness it was tuned for must be retrained for every new one. It cannot keep pace with a runtime ecosystem that turns over every few months.

Three concrete pressures make the risk tangible. **(1)** Harness-specific tuning effects are large: identical harnesses change rank when ported across providers, sometimes by tens of points, so a model’s measured “ability” is partly an artifact of how well its idioms match the harness [468]. **(2)** Multi-model orchestration is already standard: large models plan while small models execute. Learned preference-based routing [451] and layered model composition [638] serve as building blocks, with systems such as EvoRoute, Chimera, and ORCH offering early systematic studies [447, 810, 880]; any single-harness assumption thus breaks immediately. **(3)** Portable cross-provider standards such as AGENTS.md and Agent Skills are emerging organically rather than from any single vendor [16, 98]. This development elevates *transferability*, the extent to which a harness configuration remains effective when the base model is swapped, to a first-class concern.

Outlook. We see three complementary routes toward harness-robust competence. **(1)** *External transfer*: package experience, and ideally skills, into portable artifacts so that a model can read externally mounted skills at inference time and gradually raise its competence without retraining. **(2)** *Internal generalization*, in our view the more fundamental route: it exposes the model to many harnesses during training by mixing multi-harness trajectories and even running hybrid-harness RL, so that harness-robust behavior is built into the weights rather than bolted on. **(3)** *Standardizing a harness protocol*: just as the Model Context Protocol connected the world’s tools behind one discoverable, authorizable interface and thereby made tool use broadly transferable [100], a standard harness protocol that supports flexible design and registration could become the starting point for world-wide harness generalization.

The concrete open problems are: the minimal portable contract that preserves performance across providers; training recipes that explicitly optimize for cross-harness transfer rather than single-harness fit; robustness under harness drift, when a new server or tool-calling convention silently changes behavior; and portable observability, so that traces from different runtimes can be analyzed in one stack. Transferability concerns whether competence transfers across runtimes at a single point in time; the complementary question is whether competence *accumulates* across time, which is the problem of continual learning.

7.1.3 Continual and Lifelong Learning

Status and Challenges. Continual and lifelong learning are central to long-horizon agents because such agents operate across many tasks, sessions, environments, and users rather than within a single isolated episode. A capable agent should not only solve the current task, but also accumulate experience that improves later behavior while preserving previously acquired abilities. Existing work pursues this along two complementary routes. **(1)** *External* lifelong learning stores experience outside the parameters: Voyager maintains an expanding library of executable skills for open-ended exploration [630], ExpeL extracts natural-language lessons from past

trajectories [853], and Reflexion keeps verbal feedback in episodic memory to improve later trials without weight updates [544]; such methods are cheap, reversible, and quick to adapt. **(2) Internal** continual learning folds new knowledge or behavior into the model itself; this supports broader generalization, but frequent updates remain costly and risk catastrophic forgetting, especially under noisy, shifting deployment data [58, 641].

Outlook. The key challenge is to connect these two forms of learning into a stable lifelong-learning loop: most new experience should first be kept externally as memory, trajectories, or skills, where it can be inspected, revised, and tested across future tasks, and only experience that proves reliable and broadly useful should later be consolidated into weights. This staged process makes lifelong learning less about fine-tuning after every interaction and more about deciding what to remember, reuse, generalize, or forget.

A distinctive new opportunity is *online learning from deployment at scale*: as billion-user agent products such as Cursor¹², Claude Code¹³, and Doubao¹⁴ mature, they accumulate vast streams of user logs and feedback. Turning these traces into safe improvements is a difficult but consequential long-term research challenge. Doing so requires metrics that identify high-value episodes and online-learning algorithms that incorporate them into the agent with minimal human intervention and without regressions. The open problems include how to prevent forgetting during periodic updates, how to extract trustworthy learning signals from noisy trajectories and user feedback, how to transfer lessons across tasks and environments, and how to evaluate improvement over many sessions rather than within a single episode. In this sense, lifelong learning is a necessary condition for long-horizon agency: without it, an agent may act for a long time, yet never truly become better over time.

7.2 Effectiveness: Acting in Realistic Environments

Evolution matters only if the resulting agent remains effective when the environment becomes slow, noisy, partially irreversible, and hard to verify. The second axis concerns whether long-horizon agents can operate effectively in realistic environments. We first consider the digital environments used for training and evaluation, and then turn to the physical world, where challenges arise from real-time perception and control.

7.2.1 Real-world Environment Interaction

Status and Challenges. A great many agent applications are defined by sustained interaction with real-world environments: ordering food or booking a flight, drafting a Feishu or Notion document, running real-time financial analysis and forecasting, or communicating on platforms such as WeChat¹⁵ and X¹⁶. For training long-horizon agents, however, we usually cannot act directly in these live environments because interaction is slow, costly, and often irreversible. The binding constraint becomes the *environment* rather than the model: where an agent can act, whether it can fail safely, and whether it can receive a checkable reward.

Two construction routes are emerging. **(1) Verifiable, executable synthesis** builds environments that behave like the real systems they stand in for and mechanically decide success and failure: realistic testbeds such as WebArena, OSWorld, and TheAgentCompany ground agents in real browsers, operating systems, and multi-day company workflows [716, 722, 885], and automated environment synthesis pushes this toward scale [131, 238, 555]. **(2) Model-simulated environments** treat a large model itself as an implicit world model, as in GenEnv, Qwen-AgentWorld, and reasoning-model simulators such as Simia-SFT [195, 905], trading mechanical guarantees for flexibility and coverage.

Both routes face the same test of *faithfulness*: an environment that is inexpensive to evaluate but unfaithful may train agents that perform well in controlled settings yet fail in deployment. Faithfulness is thus best treated not as one property but as several separately auditable axes: whether interfaces and available actions match the real system; whether identical actions produce the same state change; whether automatic validators agree with real acceptance criteria; whether simulated users behave like real ones; and whether a reset returns the system to a functionally equivalent state rather than one that merely looks similar.

¹²<https://cursor.com>

¹³<https://www.anthropic.com/claude-code>

¹⁴<https://www.doubao.com>

¹⁵<https://www.wechat.com>

¹⁶<https://x.com>

Outlook. We hold that environment construction, not model scale, is the rate-limiting step for the next phase of agent capability, and that “faithfulness” deserves to be measured as explicitly as task success is today. The most promising direction is to *fuse* the two routes above by combining the verifiability of synthesized environments with the flexibility of model-simulated ones. This fusion would allow agents to train broadly while still receiving trustworthy rewards. On the learning side the binding constraint is sample efficiency: real-world rollouts are slow, costly, and sometimes irreversible, so the algorithms that matter extract the most signal per interaction through off-policy reuse, experience replay over returned traces, and curricula in which environment difficulty co-evolves with the policy.

The open problems are: automated construction of environments that are provably checkable and measurably faithful; metrics and audits for the sim-to-real faithfulness of synthetic and model-simulated environments; sample-efficient RL for slow, costly, partially irreversible interaction; and curricula that co-evolve environment difficulty with agent competence. The most demanding realistic environment of all, however, is the physical world rather than the digital one. In this setting, perception and control must run in real time, and many actions cannot be undone. These constraints define the frontier of embodiment.

7.2.2 From Digital to Embodied Agents

Status and Challenges. A natural extension of long-horizon digital agency is embodiment: equipping an agent with real-world perception and control so that it can provide high-level deliberation for an embodied system. The externalization logic of this paper extends directly to embodiment. The digital harness becomes a high-level controller that orchestrates low-level physical skills, recapitulating the harness pattern as a division of labor between the cerebrum and cerebellum. In this arrangement, a high-level LLM/VLM agent interprets goals, decomposes tasks, and replans based on feedback [243, 357], while Vision-Language-Action (VLA) models serve as callable real-time skill modules for atomic manipulation [35, 36, 134, 277, 899]. But embodiment sharpens three challenges specific to the physical world. **(1)** A *timescale conflict*: long-horizon planning is deep but slow, and its deliberation clashes with the millisecond reactive control an embodied system demands. **(2)** A *dimensionality and physics gap*: today’s long-horizon agents reason mostly over language and, increasingly, other digital modalities, whereas the physical world has far higher effective dimensionality, so better modeling of physical laws and dynamics is a key hurdle to migrating a digital long-horizon agent into a body. **(3)** A *feedback-granularity gap*: real environments provide feedback at multiple spatial and temporal scales, from sparse task-level signals to dense control-level signals (pose deviations, contact forces, or tactile mismatches), and an embodied agent must integrate feedback across these scales rather than rely only on the coarse plan-level signals that text harnesses handle well.

Outlook. We expect embodied and digital agent architectures to converge on a shared engineering stack with the long-horizon harness as the deliberative core, and we read the three challenges above as its research agenda. **(1)** For the timescale conflict, the promising route is an asynchronous, hierarchical harness that decouples slow deliberation from a fast reactive control loop, letting the deliberative core replan out-of-band while cached skills and reflexes keep the body responsive. **(2)** For the physics gap, embodied harnesses will likely lean on action-conditioned world models and simulation, so that the agent can predict physical consequences and close part of the sim-to-real gap before acting. **(3)** For the feedback-granularity gap, memory, skill, and protocol designs must carry both coarse subgoals and fine-grained continuous corrections through one interface, with verifiers that operate at both scales.

The open problems are: real-time, safety-critical action verification under latency budgets text harnesses can ignore; memory, skill, and protocol designs for irreversible physical action; closing the sim-to-real gap between simulated and physical deployment; and a unified deliberative harness that drives both digital tools and physical effectors. Whether an agent acts through digital tools or physical effectors, each step also draws down a finite budget of computation, context, and time. Therefore, how these resources should be allocated becomes a first-class frontier.

7.3 Efficiency: Spending Compute, Context, and Modality Budgets

Long horizons turn resources into first-class design variables. An agent must decide not only what to do, but how much computation, context, modality bandwidth, and verification effort each part of the run deserves.

The efficiency frontier spans both classical compute budgeting and the newer problem of scheduling high-cost multimodal evidence.

7.3.1 Cost- and Budget-aware Agency

Status and Challenges. As long-horizon agents move from research demos into ordinary users’ everyday workflows, such as hours-long coding sessions and personal-assistance workflows spanning an entire day, cost and budget cease to be implementation details and become first-class concerns for both usability and economics. Yet today’s agents are largely *budget-blind*: they carry almost no working sense of what a token costs, no notion that context and compute are a finite budget to be spent deliberately, and no felt pressure to finish within a price or time ceiling. In practice they default to the most capable model at every step, re-read context they could have summarized, and keep spending on runs that are already doomed. This is merely wasteful at demo scale, but once such agents run continuously for millions of users it translates into real cost and a degraded day-to-day experience.

The mechanisms to spend more wisely already exist, including compute-optimal test-time allocation [550], adaptive reasoning budgets [442], tunable best-of- N and multi-agent aggregation [323], and cost-aware model routing and cascades [62, 237, 447, 451, 810, 880]. However, agents do not yet use these mechanisms with any awareness of their own spending. Budget-awareness is best seen not as a single capability but as a chain: an agent must perceive what a task will cost, act on that estimate, run inside a harness that can enforce the limits it would otherwise ignore, and ideally be guided by a law that predicts the trade-off.

Outlook. We frame the way forward as four open problems along this chain. **(1) *Perceiving cost, difficulty, and infeasibility.*** An agent should forecast, and keep re-forecasting as a run unfolds, how hard a task is, how much budget remains, and whether the task has already become infeasible; even a coarse predicted-infeasibility signal can save a large fraction of the tokens otherwise burned on doomed runs [370]. The quantity is multi-dimensional and coupled (tokens, money, wall-clock time, sometimes real resources), and the estimate must be *calibrated* rather than optimistic, since only a calibrated forecast tells the agent when and by how much to change course. **(2) *Acting on the estimate.*** A cost estimate is worthless unless it changes behavior: the agent should shorten reasoning on easy subtasks, avoid redundant re-reading, reserve depth for where it changes the outcome, and escalate, stop, or re-route when the forecast turns bad, while avoiding a measurable loss in quality. The difficulty is that these choices are made mid-trajectory under uncertainty, where a false economy early can cost far more later. **(3) *Enforcing budgets from the harness.*** Not all budget discipline should live in the model’s head; the harness should enforce frugality through budget-aware routing and cascades, context compression and eviction, caching and reuse of intermediate results, and scheduling primitives that hold the agent to a hard price, token, or time ceiling even when the model would overspend. Because the harness sees the whole run, it is also the natural place to account for cost across a long, multi-tool trajectory. **(4) *A scaling law for budget and success.*** Further out, is there a predictable relationship linking compute budget, tool calls, and run length to success rate, analogous to a Chinchilla-style law for agency? Without one, budget allocation stays a craft; the first credible candidates should emerge from the long-running-task regime, where budgets are large enough that depth-versus-breadth trade-offs dominate.

In text-only agents these budgets are already difficult; in multimodal agents the dominant cost may be not reasoning depth but which sensory stream to keep, compress, or discard.

7.3.2 Multimodal and Omni Harnesses

Status and Challenges. Current harnesses are still designed primarily around textual inputs and command-line interaction, whereas future long-horizon agents will operate over far richer multimodal information, including images, video, audio, GUI states, and generated media. Current multimodal agents typically treat multimodality as an auxiliary capability rather than as a native substrate. Doubao’s task-assistant mode, for example, uses a text-centric main agent that invokes external tools for image and video generation and perception. Recent work has begun to address this challenge. Examples include efforts toward on-policy data evolution for visual-native multimodal deep-search agents, as well as verifiable multimodal deep research through multi-agent harnesses for interleaved report generation. However, a unified definition and architecture for a multimodal or omni harness is still missing. The technical obstacles are concrete. **(1) *The cost of visual***

tokens: video and image streams cannot be placed into context in full in the same way as text logs, so the agent must decide when to observe, what evidence to retain, and how to compress, retrieve, and memorize multimodal information; today this allocation across text, images, and video remains largely heuristic. **(2) Cross-modal verification**: multimodal evidence often must be transcribed, aligned, or abstracted into another representation before it can be judged, and these intermediate transformations can themselves fail or hallucinate, so adding more verification stages need not improve reliability linearly. **(3) A wider externalization surface**: multimodal externalization changes the design assumptions underlying skill specification, memory indexing, and protocol schemas, opening a substantially wider frontier for what can be externalized at all.

Outlook. The most consequential shift is that a unified, *vision-native* omni harness must emerge, and we frame the agenda as four requirements. **(1) A vision-native substrate.** It should play for real-world visual capability a role analogous to that of Claude Code and OpenClaw for code and the workstation, giving the whole research program a vision-native harness to build on rather than a text harness with modalities attached. **(2) Per-modality, budget-aware granularity.** Future omni harnesses must choose an appropriate information granularity per modality (raw frames preserve the most information at the highest cost, while sampling, clip summarization, or feature compression trade detail for savings) and decide autonomously at what granularity multimodal evidence is retained, compressed, and retrieved. **(3) Early error detection before rendering.** Because the outputs of image and video generators are no longer cheap textual artifacts, harnesses must detect errors and reduce rework *before* full artifacts are produced, through intermediate representations, low-fidelity previews, staged inspection, or constrained generation. **(4) Omni-modal routing.** The harness should decide not only which model and how deep to search, but which modality, temporal segment, and expert model deserve the budget [339, 364]. As agents extend perception and action beyond text, efficiency and trustworthiness become inseparable: every decision to retain, compress, discard, or regenerate multimodal evidence is at once a cost trade-off and a reliability risk.

7.4 Trustworthiness: Robust and Governed Autonomy

The final axis asks how autonomy remains trustworthy as the horizon grows. Long runs make failures cumulative rather than isolated: small early mistakes compound into goal drift, and the more authority a harness is granted, the larger the blast radius of a single wrong action. Accordingly, trustworthiness joins two problems that are usually studied apart: whether an agent can notice and recover from its own errors under noisy feedback, and whether its autonomy can be governed, contained, and audited as it acts on real systems.

7.4.1 Reflection and Error Robustness

Status and Challenges. Error robustness is the first pillar of trustworthy long-horizon autonomy: an agent that runs unattended over many steps can be trusted only insofar as it can tell when it is going wrong and change course, since an undetected early error is silently amplified over the rest of the run. Real tasks rarely succeed on the first attempt, and the feedback from a real environment is delayed, partial, and noisy, so robustness is a property of the *whole trajectory*, not of any single step: the decisive capability is to act sensibly under noisy feedback, detect mid-execution that a path is failing, and stop or switch reasoning paths early rather than committing to one track to the bitter end. The self-correction literature now provides a more qualified account of when these methods are effective. Reflexion and Self-Refine show that iterative critique-and-revise loops can raise reliability [416, 544], and CRITIC shows that tool-grounded critique, which verifies against an external signal rather than the model’s own confidence, is markedly more trustworthy than ungrounded self-reflection [182]. The crucial caveat is that models frequently *cannot* reliably self-correct reasoning from intrinsic signals alone and may even degrade when they try [234], which is why external, harness-supplied verification must anchor reflection, and why over long horizons undetected errors compound into goal drift and inherited error from weaker prior trajectories [20, 427].

Outlook. We see three priorities. **(1) Early detection and path-switching**: under noisy, delayed environmental feedback, the harness should detect a failing trajectory as early and cheaply as possible and abandon it to try an alternative, rather than spending the budget before discovering the run was doomed. **(2) Aligning to human correction**: in practice, a human often steps in partway through a run to redirect the agent, and a central open problem is how to internalize that signal; possible routes include external, human-aligned validation

checkpoints and incorporating human course-correction trajectories into training so that the agent learns *when* and *how* people intervene. **(3) Calibrated self-knowledge:** reflection should be grounded in external verification rather than the model’s own often-miscalibrated confidence, and the agent should carry a calibrated sense of what it does not know, so that it escalates or asks for help instead of confidently compounding an error. But detecting and recovering from one’s own errors is only half of trustworthy autonomy; the other half is ensuring that an agent, however robust, stays governed, contained, and auditable as the authority it wields over real systems grows. This is the problem of safety and governance.

7.4.2 Safety and Governance

Status and Challenges. In many respects, safety is the highest-stakes frontier in long-horizon agency, and three pressures make this challenge particularly acute.

(1) The environment itself can inject error and risk: a real environment may return manipulated or adversarial observations. More subtly, an agent that summarizes its own experience can distill a hazardous heuristic and then reuse it; for example, having once observed that a successful refund elicits a grateful reply, an agent may over-generalize and become predisposed to issue refunds in later tasks. Such experience-reuse effects compound as harness artifacts (skills, servers, shared memory) are shared across teams, creating attack surfaces analogous to a software supply chain [463, 902]. **(2) No unified standard for safety verification:** harness-level verifiers and benchmarks still overwhelmingly treat task success as the primary measure of *effectiveness*, while largely ignoring whether the actions taken along the way were *safe*, so local verifiers can all pass even when the run violates real-world constraints. **(3) Safety under self-evolution:** self-modifying harnesses make it hard to preserve initial safety invariants, and because a self-evolving agent that audits its own modifications is prone to the same biases that produced them, self-checking alone is not a trustworthy guarantee.

Underlying all three is a structural fact: most deployed safety mechanisms (least privilege, approval gates, capability scoping, audit, sandbox isolation) are *harness-level* controls living above the model and outside its weights, so treating safety as a model-only property miscounts the true attack surface.

Outlook. We sketch a technical path for each pressure. **(1)** Against injected error and hazardous experience reuse, the harness should treat external observations as untrusted input by sandboxing them and tracking their provenance, and should gate which distilled experiences may be reused, so that a heuristic must clear a safety review before it can shape future behavior. **(2)** Against the verification gap, the community needs a *unified safety-verification standard* that scores action safety as a first-class axis alongside effectiveness, with external acceptance tests and red-team suites that certify what an agent did, not just whether it succeeded. **(3)** Against self-evolution risk, invariant-preserving evolution should be enforced by *independent* verifiers rather than the evolving agent’s own judgment, ideally with provable guarantees that a modification cannot weaken a fixed set of safety invariants.

More broadly, each safety layer adds latency, token cost, and friction, so the recurring engineering challenge is to quantify the *alignment tax* of a harness design and minimize it without compromising safety [1, 41, 303, 368, 520, 547, 617, 783]. Our position is that the harness, not the model, is the principal alignment and privilege surface for production agents: safety properties that hold robustly in production are almost always harness-imposed, while model-side alignment is necessary but rarely sufficient. This argument only sharpens once the harness leaves the screen: as embodied agents turn digital decisions into physical action, governance stops being a runtime policy and becomes a real-time safety constraint, which makes the harness, not the model, the decisive locus of trust for long-horizon autonomy.

8 Conclusion

In this paper, we review long-horizon AI agents through a central thesis: long-horizon agency emerges from the *co-evolution of externalized harness engineering and internalized model optimization*. From this view, we develop the discussion through six aspects. **Foundation** formalizes the agent as a coupling of a base policy and a surrounding harness, organizes long-horizon tasks into three nested levels (H1 to H3) together with the capabilities they demand (C1 to C3), and characterizes the scaling of the execution time horizon.

Evolution traces how the control surface has progressively widened across three stages: prompt engineering, context engineering, and runtime harnesses. **Harness** takes a full lifecycle view, organizing the runtime by component: the agent loop and workflow, context and memory, tools and skills, orchestration, hooks, and verification. **Optimization** lays out a complete training recipe for internalizing competence, proceeding from an architectural and data–environment substrate through pre-/mid-training, fine-tuning, reinforcement learning, and on-policy distillation to self-evolution. **Application** organizes practice by the interface between the agent and its environment, spanning software engineering, information seeking, computer use, multimodal, and general-purpose agents, paired with horizon-aware evaluation. **Frontier** consolidates the open problems into four axes: evolution, effectiveness, efficiency, and trustworthiness.

Beyond organizing prior work, we distill several emerging directions that define the next stage of long-horizon agent research. We hope this paper lays a solid foundation for future work and serves as a useful reference for researchers and practitioners alike. As agentic systems continue to mature, the long-horizon agent will remain a central and open problem, one likely to play a decisive role in building robust, general, and enduring artificial intelligence.